

RTL Design Sherpa

DDR3 / LPDDR3 Family Controller Micro-Architecture Specification 0.1

October 7, 2026

Table of Contents

1 Scoria Mas Index..... 27

2 Document Information..... 28

2.1 scoria DDR3/LPDDR3 Family Controller Micro-Architecture Specification. .28

2.2 Revision History..... 29

2.3 Document Purpose..... 31

2.4 Intended Audience..... 32

2.5 Related Documents..... 33

3 Block Inventory..... 34

3.1 Top and macro tier..... 35

3.2 FUB tier..... 37

3.3 The three blocks that are not ordinary FUBs..... 40

3.4 How to read Chapter 2..... 41

4 What Changes vs pumice..... 42

4.1 What the MAS adds to the marking..... 45

4.2 One paragraph per changed or new block..... 46

5 Clocks, Reset, and Gearing..... 47

5.1 The two domains..... 47

5.2 The target design point..... 48

5.3 Width gearing..... 49

5.4 Timing floors worth knowing up front..... 50

6 AXI4 Plane (scoria_axi4_plane)..... 51

6.1 Purpose..... 52

6.2 Parameters....53
6.3 Interface....55
6.3.1 Host AXI4 ports....55
6.3.2 Address-mapping CSR ports....55
6.3.3 Write scheduler and commit-data ports....56
6.3.4 Read scheduler and DFI return ports....57
6.3.5 Status port....58
6.4 Microarchitecture internals....59
6.4.1 Instantiation tree....59
6.4.2 Write datapath....59
6.4.3 Read datapath....59
6.4.4 Read-after-write snarf....60
6.4.5 Read-command CAM and return-ring cooperation....60
6.4.6 Flat scheduler vectors and legacy port tie-offs....60
6.5 FSM policy....61
6.6 Timing....62
6.7 Notes....63
7 Scheduler Plane (scoria_scheduler_plane)....64
7.1 Purpose....65
7.2 Parameters....66
7.3 Interface....67
7.3.1 Timing, mode, and maintenance configuration ports....67
7.3.2 CAM scheduler/commit/issue ports (from scoria_axi4_plane)....68
7.3.3 Output command stream (to scoria_dfi_plane)....69

7.3.4 Init, PHY, mode-register shadow, and write-leveling ports....70
7.3.5 Telemetry, stall attribution, and status ports....71
7.4 Microarchitecture internals....72
7.4.1 Instantiation tree....72
7.4.2 Command packing and output FIFO....72
7.4.3 Fixed-delay release....72
7.4.4 busy_o definition....72
7.4.5 Refresh and ZQ gating....73
7.4.6 Page-policy tap....73
7.4.7 Optional command-history checker....73
7.5 FSM policy....74
7.6 Timing....75
7.7 Notes....76
8 DFI Plane (scoria_dfi_plane)....77
8.1 Purpose....78
8.2 Parameters....79
8.3 Interface....81
8.3.1 Controller-domain ports....81
8.3.2 PHY DFI-domain ports....81
8.3.3 Write-leveling crossing ports....83
8.4 Microarchitecture internals....85
8.4.1 Instantiation tree....85
8.4.2 Single CDC....85
8.4.3 Active-gear phase mask....85

8.4.4 Command path....85
8.4.5 Write serializer....85
8.4.6 Read aligner....85
8.4.7 Write-leveling crossing....86
8.5 FSM policy....87
8.6 Timing....88
8.7 Notes....89
9 AXI Burst Chopper (scoria_axi_burst_chopper)....90
9.1 Purpose....91
9.2 Parameters....92
9.3 Interface....93
9.3.1 Host address channel in....93
9.3.2 Sub-command address channel out....93
9.3.3 Aggregation sideband....94
9.4 Microarchitecture internals....95
9.5 FSM policy....96
9.6 Timing....97
9.7 Notes....98
10 Write Splitter (scoria_wr_splitter)....99
10.1 Purpose....100
10.2 Parameters....101
10.3 Interface....102
10.3.1 Host AXI write in....102
10.3.2 Sub-command AXI write out....103

10.3.3 Aggregation sideband....104
10.4 Microarchitecture internals....105
10.5 FSM policy....106
10.6 Timing....107
10.7 Notes....108
11 Write Intake (scoria_wr_intake)....109
11.1 Purpose....110
11.2 Parameters....111
11.3 Interface....113
11.3.1 AXI4 write slave....113
11.3.2 Downstream write-command push....114
11.3.3 Write-data pop....115
11.3.4 Write completion....115
11.4 Microarchitecture internals....116
11.5 FSM policy....117
11.6 Timing....118
11.7 Notes....119
12 Write Data CAM (scoria_wr_data_cam)....120
12.1 Purpose....121
12.2 Parameters....122
12.3 Interface....123
12.3.1 Command insert....123
12.3.2 Fill data....123
12.3.3 Snarf lookup and stream....123

12.3.4 Oldest entry port....124
12.3.5 Scheduler lookup ports....125
12.3.6 Per-entry scheduling vectors....125
12.3.7 Commit stream....126
12.4 Microarchitecture internals....127
12.4.1 Entry state....127
12.4.2 Age ordering and the matrix....127
12.4.3 Snarf lookup....127
12.4.4 SRAM and shared read port....128
12.4.5 Rate-matched commit....128
12.4.6 Eviction and B consolidation....128
12.5 FSM policy....129
12.6 Timing....130
12.7 Notes....131
13 Read Intake (scoria_rd_intake)....132
13.1 Purpose....133
13.2 Parameters....134
13.3 Interface....136
13.3.1 AXI4 read slave....136
13.3.2 Read command miss path....137
13.3.3 Snarf probe to write CAM....137
13.3.4 Snarf data stream....138
13.3.5 DFI read-return stream....138
13.4 Microarchitecture internals....139

13.4.1 Two-stage AR admit pipeline....139
13.4.2 Snarf probe handshake....139
13.4.3 Order FIFO....139
13.4.4 Beat dropping and RLAST generation....140
13.4.5 Source arbiter and read-data FIFO....140
13.5 FSM policy....141
13.6 Timing....142
13.7 Notes....143
14 Read Command CAM (scoria_rd_cmd_cam)....144
14.1 Purpose....145
14.2 Parameters....146
14.3 Interface....147
14.3.1 Insert....147
14.3.2 Scheduler vectors....147
14.3.3 Generic scheduler lookup ports....148
14.3.4 Oldest fallback port....148
14.3.5 Issue notify....149
14.4 Microarchitecture internals....150
14.4.1 Entry fields....150
14.4.2 Ticket carried from insert to issue....150
14.4.3 Age-order matrix....150
14.4.4 Oldest via 1-bit AND-reduce....150
14.4.5 sch_age_exceed_o versus age_thresh_i....150
14.4.6 Unused ports tied off in scoria....151

14.5 FSM policy....152
14.6 Timing....153
14.7 Notes....154
15 Read Return Ring (scoria_rd_return_ring)....155
15.1 Purpose....156
15.2 Parameters....157
15.3 Interface....158
15.3.1 Allocate....158
15.3.2 Issue notify....158
15.3.3 DFI read return....158
15.3.4 Drain and observability....159
15.4 Microarchitecture internals....160
15.4.1 Tickets in AR order....160
15.4.2 Issue-order FIFO....160
15.4.3 DFI returns land in the issue-Q head slot....160
15.4.4 2-deep prefetch skid over BRAM....160
15.4.5 Head freed at last-beat consume....160
15.4.6 Occupancy output....160
15.4.7 DEPTH bounds in-flight reads....161
15.5 FSM policy....162
15.6 Timing....163
15.7 Notes....164
16 Address Mapper (scoria_addr_mapper)....165
16.1 Purpose....166

16.2 Parameters....167
16.3 Interface....168
16.3.1 Address and configuration ports....168
16.4 Microarchitecture internals....169
16.4.1 Byte-offset strip and bank_lsb clamp....169
16.4.2 Field extraction....169
16.4.3 Bank XOR-hash....169
16.5 FSM policy....170
16.6 Timing....171
16.7 Notes....172
17 Page Policy (scoria_page_policy)....173
17.1 Purpose....174
17.2 Parameters....175
17.3 Interface....176
17.3.1 Configuration and command tap....176
17.3.2 Decisions and telemetry....176
17.4 Microarchitecture internals....178
17.4.1 Modes....178
17.4.2 Per-bank idle timeout....178
17.4.3 Background close request....179
17.4.4 Telemetry counters....179
17.5 FSM policy....180
17.6 Timing....181
17.7 Notes....182

18 Bank Timer (scoria_bank_timer + scoria_bank_timers)....183
18.1 Purpose....184
18.2 Parameters....185
18.2.1 scoria_bank_timers....185
18.2.2 scoria_bank_timer....185
18.3 Interface....186
18.3.1 Wrapper event and readiness ports....186
18.3.2 Single-timer ports....186
18.4 Microarchitecture internals....188
18.4.1 Five timers....188
18.4.2 Safe equations....188
18.4.3 Auto-precharge pending....188
18.4.4 Lookahead twins....188
18.4.5 Observability and derived state....189
18.5 FSM policy....190
18.6 Timing....191
18.7 Notes....192
19 Global Timers (scoria_global_timers)....193
19.1 Purpose....194
19.2 Parameters....195
19.3 Interface....196
19.3.1 Timing CSRs and events....196
19.3.2 Readiness and observability....196
19.4 Microarchitecture internals....198

19.4.1 tFAW....198
19.4.2 tRRD....198
19.4.3 tWTR, tRTW, tCCD....198
19.4.4 Next-state alignment....198
19.4.5 Output flops....198
19.5 FSM policy....199
19.6 Timing....200
19.7 Notes....201
20 Command Arbiter (scoria_cmd_arbiter)....202
20.1 Purpose....203
20.2 Parameters....204
20.3 Interface....205
20.3.1 CAM scheduler vectors....205
20.3.2 Policy CSRs and QoS inputs....206
20.3.3 Page-policy inputs....207
20.3.4 Timer readiness....208
20.3.5 Maintenance request/grant....209
20.3.6 Init passthrough....210
20.3.7 Event strobes....210
20.3.8 Command output....211
20.3.9 Stall attribution....211
20.4 Microarchitecture internals....213
20.4.1 Classification....213
20.4.2 Priority chain....213

20.4.3 Policy CSRs....213
20.4.4 Write-batching hysteresis....214
20.4.5 Pick pipeline....214
20.4.6 Final safety gate....215
20.4.7 Fire-stage bug fixes....215
20.4.8 Column spacing and turnaround guards....215
20.4.9 Auto-precharge guard....215
20.4.10 Refresh safety....215
20.4.11 tRFC recovery and tZQCS block....216
20.4.12 Stall attribution....216
20.4.13 Maintenance request/grant semantics and event strobes....216
20.5 FSM policy....217
20.6 Timing....218
20.7 Notes....219
21 Refresh Controller (scoria_refresh_ctrl)....220
21.1 Purpose....221
21.2 Parameters....222
21.3 Interface....223
21.3.1 Timing, mode, and configuration....223
21.3.2 Credit-window and demand-aware policy....223
21.3.3 Arbiter handshake and refresh outputs....224
21.3.4 Observability....225
21.4 Microarchitecture internals....227
21.4.1 tREFI counter and effective interval....227

21.4.2 Credit accumulator....227
21.4.3 Postpone and pull-in limits....227
21.4.4 Idle confirmation and the three-case request....227
21.4.5 Drain burst....228
21.4.6 REFpb rotor....228
21.4.7 Why a credit machine and not an FSM....228
21.5 FSM policy....229
21.6 Timing....230
21.7 Notes....231
22 ZQ Controller (scoria_zq_ctrl)....232
22.1 Purpose....233
22.2 Parameters....234
22.3 Interface....235
22.3.1 Configuration and timing....235
22.3.2 Mode C placement policy....235
22.3.3 Arbiter handshake and telemetry....235
22.4 Microarchitecture internals....237
22.4.1 Interval counter....237
22.4.2 The four FSM states....237
22.4.3 Placement policy....237
22.4.4 Post-grant hold semantics....237
22.5 FSM policy....239
22.6 Timing....240
22.7 Notes....241

23 Init Sequencer (scoria_init_sequencer)....242
23.1 Purpose....243
23.2 Parameters....244
23.3 Interface....245
23.3.1 Clocks, reset, configuration, and DFI status....245
23.3.2 JEDEC init-sequence waits (CSR-backed)....245
23.3.3 Mode-register values, restart, and shadow port....246
23.3.4 Command request and status....247
23.4 Microarchitecture internals....248
23.4.1 FSM state list....248
23.4.2 DDR3 initialization sequence....248
23.4.3 LPDDR3 initialization sequence....248
23.4.4 MR0 DLL reset handling....248
23.4.5 dram_reset_n_o latch....248
23.5 FSM policy....249
23.6 Timing....250
23.7 Notes....251
24 Mode Register (scoria_mode_register)....252
24.1 Purpose....253
24.2 Parameters....254
24.3 Interface....255
24.3.1 Clocks, reset, configuration, and write port....255
24.3.2 Scheduler request channel (v1 unused)....255
24.3.3 Live decoded outputs....256

24.4 Microarchitecture internals....257
24.4.1 Shadow array....257
24.4.2 Live decode equations....257
24.4.3 JESD79-3F MR0 field-map note....257
24.5 FSM policy....259
24.6 Timing....260
24.7 Notes....261
25 Write Leveling Interface (scoria_wrlvl_ifc)....262
25.1 Purpose....263
25.2 Parameters....264
25.3 Interface....265
25.3.1 Mode and host controls....265
25.3.2 Timing windows (runtime CSRs)....265
25.3.3 DFI v3.1 per-CS leveling handshake....266
25.3.4 DRAM answer and telemetry....266
25.4 Microarchitecture internals....268
25.4.1 FSM state list....268
25.4.2 State behavior....268
25.4.3 t_wlmrd_max_i timeout behavior....268
25.4.4 DFI handshake outputs....268
25.5 FSM policy....269
25.6 Timing....270
25.7 Notes....271
26 DFI Command Path (scoria_dfi_cmd_path)....272

26.1 Purpose....273
26.2 Parameters....274
26.3 Interface....276
26.3.1 Controller-side command in....276
26.3.2 DFI command bus....277
26.3.3 Fire strobes....277
26.4 Microarchitecture internals....278
26.4.1 No command pacing....278
26.4.2 Sub-DFI-word burst packing....278
26.4.3 DFI-wire command-history scoreboard....279
26.5 FSM policy....280
26.6 Timing....281
26.7 Notes....282
27 DFI Command Formatter (scoria_dfi_cmd_formatter)....283
27.1 Purpose....284
27.2 Parameters....285
27.3 Interface....286
27.4 Microarchitecture internals....288
27.4.1 DDR3 command truth table....288
27.4.2 Marking against pumice....288
27.4.3 LPDDR3 CA-bus encoding....289
27.4.4 ODT....289
27.4.5 Phase placement....290
27.5 FSM policy....291

27.6 Timing....292
27.7 Notes....293
28 DFI Write Serializer (scoria_dfi_wr_serializer)....294
28.1 Purpose....295
28.2 Parameters....296
28.3 Interface....297
28.4 Microarchitecture internals....298
28.4.1 Stateless multi-burst serializer....298
28.4.2 Owed-burst accounting....298
28.4.3 Mask generation....298
28.5 FSM policy....299
28.6 Timing....300
28.7 Notes....301
29 DFI Read Aligner (scoria_dfi_rd_aligner)....302
29.1 Purpose....303
29.2 Parameters....304
29.3 Interface....305
29.4 Microarchitecture internals....306
29.4.1 Admission and backpressure....306
29.4.2 Stateless rddata_en delay line....306
29.4.3 EN_CYC versus BL_WORDS....306
29.4.4 Enable-window credit....306
29.4.5 Capture and last-word marking....307
29.4.6 Debug probes....307

29.5 FSM policy..... 308
29.6 Timing..... 309
29.7 Notes..... 310
30 DFI Clock-Domain Crossing (scoria_dfi_cdc)..... 311
30.1 Purpose..... 312
30.2 Parameters..... 313
30.3 Interface..... 314
30.3.1 Controller domain..... 314
30.3.2 PHY domain..... 315
30.4 Microarchitecture internals..... 317
30.4.1 FIFO instances..... 317
30.4.2 Payload packing..... 317
30.4.3 Pointer encoding..... 317
30.4.4 Init tokens..... 317
30.4.5 The staged-token invariant..... 318
30.5 FSM policy..... 319
30.6 Timing..... 320
30.7 Notes..... 321
31 The Dormant Pair (scoria_powerdown_ctrl + scoria_dfi_signal_pack)..... 322
31.1 Purpose..... 323
31.2 Parameters..... 324
31.3 Interface..... 325
31.3.1 scoria_powerdown_ctrl..... 325
31.3.2 scoria_dfi_signal_pack..... 326

31.4 Microarchitecture internals....327
31.4.1 scoria_powerdown_ctrl....327
31.4.2 scoria_dfi_signal_pack....327
31.4.3 The four gaps....327
31.5 FSM policy....328
31.6 Timing....329
31.7 Notes....330
32 AXI4 Slave and APB CSR at Pin Level....331
32.1 Clocks and resets....332
32.2 AXI4 slave signals....333
32.3 PeakRDL register cpuif....335
32.4 CSR address-map summary....336
32.5 Register liveness classes....338
32.6 The timing convention: exact per bank, N+1 global....339
33 DFI v3.1 Pin-Level Table....340
33.1 Per-phase packed buses....341
33.2 Reset convention....342
33.3 Clock, reset, and init....343
33.4 Command bus....344
33.5 Write data....346
33.6 Read data....347
33.7 Per-data-phase chip selects (v3.1 added)....348
33.8 Write-leveling handshake (v3.1 added)....349
33.9 Signals not pinned out....350

33.10 References....351
34 Core Signal Contracts....352
34.1 What a contract is here....352
34.2 Post-RTL citation posture....353
34.3 Contract: address decode....356
34.4 Contract: DDR3 command decode and LPDDR3 CA encoding....357
34.5 Contract: maintenance request/grant....358
34.6 Contract: refresh interval/credit....359
34.7 Contract: ZQ calibration window....360
34.8 Contract: bank-timer spacing gates....361
34.9 Contract: global-timers next-state alignment....362
34.10 Contract: read-CAM issue contract....363
34.11 Contract: return-ring input contracts....364
34.12 Contract: write-CAM fill/commit/snarf ordering....365
34.13 Contract: CDC staged-token invariant....366
34.14 Contract: command-path no-pacing....367
34.15 Contract: arbiter final-gate drop-not-hold....368
34.16 Contract: init MR ordering....369
34.17 Contract: write-leveling four-state telemetry....370
34.18 Regenerating the workbook....371

List of Figures

No figures in this document.

List of Tables

Table 1: Table 0.1: Related documents....33
Table 2: Table 1.1: Top and macro tier....35
Table 3: Table 1.2: The FUB tier with plane assignments....37
Table 4: Table 1.3: The marking set, copied from the HAS....42
Table 5: Table 1.4: Clock domains....47
Table 6: Table 1.5: The genesys2_ddr3_800 operating point....48
Table 7: Table 2.1.1: AXI4 plane parameters....53
Table 8: Table 2.1.2: Host AXI4 interface....55
Table 9: Table 2.1.3: Address-mapping configuration ports....55
Table 10: Table 2.1.4: Write scheduler and commit-data ports....56
Table 11: Table 2.1.5: Read scheduler and DFI return ports....57
Table 12: Table 2.1.6: Status port....58
Table 13: Table 2.2.1: Scheduler plane parameters....66
Table 14: Table 2.2.2: Timing, mode, and maintenance configuration ports....67
Table 15: Table 2.2.3: CAM scheduler/commit/issue ports....68
Table 16: Table 2.2.4: Output command stream....69
Table 17: Table 2.2.5: Init, PHY, mode-register shadow, and write-leveling ports. 70
Table 18: Table 2.2.6: Telemetry, stall attribution, and status ports....71
Table 19: Table 2.3.1: DFI plane parameters....79
Table 20: Table 2.3.2: Controller-domain ports....81
Table 21: Table 2.3.3: PHY DFI-domain ports....81
Table 22: Table 2.3.4: Write-leveling crossing ports....83
Table 23: Table 2.4.1: AXI burst chopper parameters....92
Table 24: Table 2.4.2: AXI burst chopper interface....94
Table 25: Table 2.5.1: Write splitter parameters....101
Table 26: Table 2.5.2: Write splitter interface....104
Table 27: Table 2.6.1: Write intake parameters....111
Table 28: Table 2.6.2: Write intake interface....115
Table 29: Table 2.6.3: Write intake FIFOs....116
Table 30: Table 2.7.1: Write data CAM parameters....122
Table 31: Table 2.7.2: Write data CAM interface....126
Table 32: Table 2.8.1: Read intake parameters....134
Table 33: Table 2.8.2: AXI4 read slave interface....136
Table 34: Table 2.8.3: Read command miss path....137
Table 35: Table 2.8.4: Snarf probe interface....137
Table 36: Table 2.8.5: Snarf data stream....138

Table 37: Table 2.8.6: DFI read-return stream....138
Table 38: Table 2.9.1: Read command CAM parameters....146
Table 39: Table 2.9.2: Insert interface....147
Table 40: Table 2.9.3: Flat per-entry scheduler vectors....147
Table 41: Table 2.9.4: Generic scheduler lookup ports....148
Table 42: Table 2.9.5: Oldest fallback port....148
Table 43: Table 2.9.6: Issue notify interface....149
Table 44: Table 2.10.1: Read return ring parameters....157
Table 45: Table 2.10.2: Ticket allocation interface....158
Table 46: Table 2.10.3: Issue-order FIFO interface....158
Table 47: Table 2.10.4: DFI read return interface....158
Table 48: Table 2.10.5: Drain and observability interface....159
Table 49: Table 2.11.1: Address mapper parameters....167
Table 50: Table 2.11.2: Address mapper interface....168
Table 51: Table 2.12.1: Page policy parameters....175
Table 52: Table 2.12.2: Page policy inputs....176
Table 53: Table 2.12.3: Page policy outputs....176
Table 54: Table 2.12.4: Page-policy modes....178
Table 55: Table 2.13.1: Bank timers wrapper parameters....185
Table 56: Table 2.13.2: Single bank timer parameters....185
Table 57: Table 2.13.3: Bank timers wrapper interface....186
Table 58: Table 2.13.4: Single bank timer command inputs....186
Table 59: Table 2.13.5: Bank timer countdowns....188
Table 60: Table 2.14.1: Global timers parameters....195
Table 61: Table 2.14.2: Global timers inputs....196
Table 62: Table 2.14.3: Global timers outputs....196
Table 63: Table 2.15.1: Command arbiter parameters....204
Table 64: Table 2.15.2: CAM scheduler vector ports....205
Table 65: Table 2.15.3: Scheduling policy CSR ports....206
Table 66: Table 2.15.4: Page-policy input ports....207
Table 67: Table 2.15.5: Timer readiness ports....208
Table 68: Table 2.15.6: Maintenance request/grant ports....209
Table 69: Table 2.15.7: Init passthrough ports....210
Table 70: Table 2.15.8: Timer event-strobe ports....210
Table 71: Table 2.15.9: Command output ports....211
Table 72: Table 2.15.10: Stall-attribution ports....211
Table 73: Table 2.16.1: Refresh controller parameters....222
Table 74: Table 2.16.2: Refresh controller ports....225

Table 75: Table 2.17.2: ZQ controller ports....235
Table 76: Table 2.18.1: Init sequencer build parameters....244
Table 77: Table 2.18.2: Clocks, reset, configuration, and DFI status....245
Table 78: Table 2.18.3: JEDEC timing CSR inputs....245
Table 79: Table 2.18.4: Mode-register inputs, restart, and shadow write port....246
Table 80: Table 2.18.5: Command request, legacy handshake, and status outputs 247
Table 81: Table 2.18.6: Init-sequence timing windows....250
Table 82: Table 2.19.1: Mode-register build parameters....254
Table 83: Table 2.19.2: Clocks, configuration, and shadow write port....255
Table 84: Table 2.19.3: Scheduler MR-update request channel....255
Table 85: Table 2.19.4: Live decoded outputs....256
Table 86: Table 2.19.5: Live decode equations....257
Table 87: Table 2.20.1: Write-leveling interface parameters....263
Table 88: Table 2.20.2: Mode and host controls....264
Table 89: Table 2.20.3: Write-leveling timing CSR inputs....264
Table 90: Table 2.20.4: DFI v3.1 write-leveling handshake....265
Table 91: Table 2.20.5: DRAM answer and host telemetry....265
Table 92: Table 2.20.6: Write-leveling timing windows....269
Table 93: Table 2.21.1: scoria_dfi_cmd_path parameters....273
Table 94: Table 2.21.2: Command input group....275
Table 95: Table 2.21.3: DFI command output group....276
Table 96: Table 2.21.4: Data-path strobe group....276
Table 97: Table 2.21.5: Timing enforcement remains upstream of the command path....277
Table 98: Table 2.22.1: scoria_dfi_cmd_formatter parameters....284
Table 99: Table 2.22.2: scoria_dfi_cmd_formatter ports....285
Table 100: Table 2.22.4: MODIFIED command encodings versus pumice....287
Table 101: Table 2.23.1: scoria_dfi_wr_serializer parameters....295
Table 102: Table 2.23.2: scoria_dfi_wr_serializer ports....296
Table 103: Table 2.24.1: scoria_dfi_rd_aligner parameters....303
Table 104: Table 2.24.2: scoria_dfi_rd_aligner ports....304
Table 105: Table 2.24.3: EN_CYC versus BL_WORDS....305
Table 106: Table 2.25.1: scoria_dfi_cdc parameters....312
Table 107: Table 2.25.2: Controller-domain ports....313
Table 108: Table 2.25.3: PHY-domain ports....314
Table 109: Table 2.25.4: Clock-domain-crossing FIFOs....316
Table 110: Table 2.26.1: Dormant pair parameters....323

Table 111: Table 2.26.2: scoria_powerdown_ctrl ports....324
Table 112: Table 2.26.3: scoria_dfi_signal_pack ports....325
Table 113: Table 2.26.4: Why the dormant pair is not wired in the DDR3 design point....326
Table 114: Table 3.1.1: Clocks, resets, and init-done....331
Table 115: Table 3.1.2: AXI4 slave pin groups and widths....332
Table 116: Table 3.1.3: PeakRDL cpuif pins....334
Table 117: Table 3.1.4: CSR address-map groups....335
Table 118: Table 3.1.5: CSR liveness classes....337
Table 119: Table 3.2.1: Clock, reset, and init handshake....342
Table 120: Table 3.2.2: DFI command bus....343
Table 121: Table 3.2.3: DFI write data....345
Table 122: Table 3.2.4: DFI read data....346
Table 123: Table 3.2.5: Per-data-phase chip selects....347
Table 124: Table 3.2.6: Write-leveling handshake....348
Table 125: Table 3.2.7: DFI v3.1 signals not pinned out....349
Table 126: Table 4.1: Anchor map — RTL anchors and formal verdicts....352

1 Scoria Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 scoria DDR3/LPDDR3 Family Controller Micro-Architecture Specification

Property	Value
Document Title	scoria DDR3/LPDDR3 Family Controller Micro-Architecture Specification
Version	0.3
Date	October 10, 2026
Status	v0.1, written against complete RTL. The scoria controller is sim-verified (HAS v0.9 records 221 tests and 9 formal blocks measured 2026-10-01; TASK-001's bounded advanced modes landed afterwards with their own suites). The testbench tier today runs 21 cocotb/pytest modules — 18 FUB, 2 macro, 1 top — from `dv/tests/`, plus the 9 SymbiYosys suites under `formal/scoria/`. Where this book and the RTL disagree, the RTL is the authority and this book is defective.
Classification	Open Source - MIT License

2.2 Revision History

Version	Date	Author	Description
0.2	2026-10-10	RTL Design Sherpa	Design-point clock restated at **75 MHz** per the owner (2026-10-10: “100 MHz was never the target”, closing scoria BUG-003). Table 1.5 (`MC clock (aclk)` row) and the ZQ interval-counter worked example (page 2.17) updated. The PHY/DRAM side — DDR3-800, 400 MHz CK, `DFI_RATE` 4 — is unchanged; the PHY clock is generated by the board oscillator’s PLL and does not scale with `aclk`.
0.1	2026-10-07	RTL Design Sherpa	First issue. The HAS (v0.9) taken one level down after the fact: every active block documented at signal level — interface tables, cycle behavior, FSM policy, and the traps worth institutionalizing — plus the DFI v3.1 pin-level table and the signal-contract anchor map a future kmap book will cite. The dormant pair (`powerdown_ctrl`, `dfi_signal_pack`) shares one page. The MC-001 rename (`scoria_axi4_ifc` → `scoria_axi4_plane`, `scoria_mem_cmd_schedule r` →

Version	Date	Author	Description
			`scoria_scheduler_plane`) is recorded where it matters; older family documents may still use the old names.
0.3	2026-10-20	RTL Design Sherpa	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

2.3 Document Purpose

This Micro-Architecture Specification (MAS) is the implementation-level view of the scoria controller — the HAS (`../scoria_has/`) taken one level down. It is the document an RTL maintainer reads before touching the SystemVerilog and a verification engineer reads before writing checkers.

The sibling books shaped it: pumice's MAS is the mechanistic ancestor scoria's blocks inherit from, and andesite's MAS explicitly references this book for every block andesite inherits unchanged. That makes this book the family's middle reference — the place where the DDR3/LPDDR3 mechanisms are stated definitively.

It covers:

- every active block's interface, parameters, and internal mechanism, including the FSM policy for each: where a state machine is required, where one is forbidden, and what replaces it elsewhere
- the three plane assemblies and how they wire together in `scoria_core`
- the DFI v3.1 pin-level table the HAS's interface chapter promises
- the core signal contracts, each named against its RTL anchor and, where one exists, its formal proof

2.4 Intended Audience

- RTL maintainers modifying or porting the controller (the LPDDR3 wake-up of the dormant power-down pair is the first named candidate)
- Verification engineers building on the cocotb tiers in `dv/tests/` and the formal suites in `formal/scoria/`
- The andesite lane, which inherits this book's blocks and must not re-derive them

2.5 Related Documents

Table 0.1: Related documents

Document	Location	Content
Hardware Architecture Specification	`../scoria_has/`	the v0.9 architecture this document expands; bindings between the two follow the HAS
design-requirements.md	`../design-requirements.md`	the delta analysis against JESD79-3F / JESD209-3C / DFI 3.1; TASK-001 mode detail (§6)
PRD	`../../PRD.md`	product requirements
Family docs	`../../../docs/`	shared-core design (`mem_ctrl_pkg`) and the five family doctrines
pumice MAS	`../../../pumice-ddr2-lpddr2/docs/pumice_mas/`	the mechanistic ancestor; scoria's inherited blocks carry pumice's evidence
andesite MAS	`../../../andesite-ddr4-lpddr4/docs/andesite_mas/`	the downstream consumer of this book's inheritance
CSR reference	`../../regs/generated/docs/scoria_csr.md`	the generated register documentation (PeakRDL output)
Family README	`../../../README.md`	family overview and per-IP status

3 Block Inventory

The controller is a three-tier tree — FUBs under macro planes under a top — mirroring pumice's shape with the `scoria_` prefix. `scoria_top` wraps `scoria_core` (the three planes) plus the PeakRDL-generated `scoria_csr` register block; configuration reaches the core **by name from `hwif_out.*`**, so there are no config ports on the core.

3.1 Top and macro tier

Table 1.1: Top and macro tier

Module	Location	Function	Status
scoria_top	rtl/top/	core + CSR block wrapper; presents host AXI4 and the DFI pin bus	complete, board-exercised in simulation
scoria_top_geared	rtl/top/	width-gearing wrapper: formally verified AXI data-width converters around scoria_top; bypassed (g_direct) when host width equals the DFI word width	complete
scoria_core	rtl/top/	the three-plane assembly, wired bottom-up	complete
scoria_axi4_plane	rtl/macro/	host front-end: burst chopping, intakes, write/read CAMs, read-return ring, snarf	complete (was scoria_axi4_ifc pre-MC-001)
scoria_scheduler_plane	rtl/macro/	the command brain: arbiter, timers, page policy, refresh/ZQ, init/leveling, output FIFO	complete (was scoria_mem_cmd_scheduler pre-MC-001)
scoria_dfi_plane	rtl/macro/	the PHY side: the single CDC, command path,	complete

Module	Location	Function	Status
		write serializer, read aligner	

3.2 FUB tier

Every active FUB gets its own page in Chapter 2, in datapath order. The Parent column is the instantiating plane — the page to read for the wiring around it.

Table 1.2: The FUB tier with plane assignments

Module	Parent	Function (one line)	Page
`scoria_axi_burst_chopper`	`scoria_axi4_plane`	FSM-free AXI address-channel burst splitter	2.4
`scoria_wr_splitter`	`scoria_axi4_plane`	write-side AW chop + WLAST reframing with zero-strobe padding	2.5
`scoria_wr_intake`	`scoria_axi4_plane`	skid buffers, decoded CAM push, ragged-burst reject	2.6
`scoria_wr_data_cam`	`scoria_axi4_plane`	write scheduling window + data SRAM + snarf source	2.7
`scoria_rd_intake`	`scoria_axi4_plane`	AR snarf probe, two-stage admit, AR-order merge back to R	2.8
`scoria_rd_cmd_cam`	`scoria_axi4_plane`	read scheduling window, carries the return-ring ticket	2.9
`scoria_rd_return_ring`	`scoria_axi4_plane`	AR-order tickets over in-flight read data	2.10
`scoria_addr_mapper`	`scoria_axi4_plane` (via the intakes)	byte offset to {rank, bank, row,	2.11

Module	Parent	Function (one line)	Page
		col}, optional bank XOR-hash	
scoria_page_policy	scoria_scheduler_plane	auto-precharge decisions, idle-timeout close, page telemetry	2.12
scoria_bank_timer / scoria_bank_timers	scoria_scheduler_plane	per-(rank,bank) JEDEC countdown timers, no FSM	2.13
scoria_global_timers	scoria_scheduler_plane	rank-global tFAW/tRRD and shared tWTR/tRTW/tCCD, in the fixed form	2.14
scoria_cmd_arbiter	scoria_scheduler_plane	FR-FCFS pick with maintenance preemption and a final safety gate	2.15
scoria_refresh_ctrl	scoria_scheduler_plane	tREFI credit machine: REFab/REFpb, elastic and TCR modes	2.16
scoria_zq_ctrl	scoria_scheduler_plane	periodic ZQCS as maintenance traffic, deferral policy	2.17
scoria_init_sequencer	scoria_scheduler_plane	RESET#/MR order for DDR3 and LPDDR3, ZQCL closes init	2.18
scoria_mode_register	scoria_scheduler_plane	per-rank MR shadows with live decode of CL/CWL/BL/AL/W	2.19

Module	Parent	Function (one line)	Page
		R	
scoria_wrlvl_ifc	scoria_scheduler_plane	firmware-driven write-leveling interface — no search loop	2.20
scoria_dfi_cmd_path	scoria_dfi_plane	abstract commands to DFI phases; does not pace	2.21
scoria_dfi_cmd_formatter	scoria_dfi_plane	op to DFI pins: DDR3 truth table, LPDDR3 CA bus	2.22
scoria_dfi_wr_serializer	scoria_dfi_plane	t_phy_wrlat delay line to dfi_wrdata_en	2.23
scoria_dfi_rd_aligner	scoria_dfi_plane	rddata_en windows, capture credits, return push	2.24
scoria_dfi_cdc	scoria_dfi_plane	the only ctl↔phy boundary: five async FIFOs	2.25
scoria_powerdown_ctrl	— (dormant)	idle-detect → PDE/SR state machine	2.26
scoria_dfi_signal_pack	— (dormant)	registered DFI pack stage owning dfi_dram_clk_disable	2.26

3.3 The three blocks that are not ordinary FUBs

`scoria_csr` is generated from `rtl/macro/scoria_csr.rdl` by PeakRDL. It is not designed, it is compiled; the register truth lives in `regs/generated/docs/scoria_csr.md` and the address-map summary is in Chapter 3. TASK-001's mode-select fields (elastic refresh, TCR derate, ZQ placement) are ordinary fields in it.

`scoria_cmd_history_checker` is a verification-side block that re-derives JEDEC spacing from the issued command stream and fatals on a violation. It is generate-gated inside the scheduler plane behind `CMD_HISTORY_EN` (default off) and exists in both scheduler-plane and DFI-wire variants. It is documented with its host on the scheduler-plane page, not given a page of its own — its mechanism is inherited from pumice, and andesite carries its DDR4 growth.

The dormant pair (`scoria_powerdown_ctrl`, `scoria_dfi_signal_pack`) is real, lint-closed RTL that nothing instantiates — deliberately, and the reasoning is on their shared page (2.26). They are the starting point for LPDDR3 power-down work, and they stay in the tree for that reason.

3.4 How to read Chapter 2

The pages run in dataflow order: the three plane assemblies first (2.1-2.3, the structural map), then the request path from AXI to CAMs (2.4-2.10), the scheduling support blocks (2.11-2.15), the maintenance and init/training blocks (2.16-2.20), the DFI blocks (2.21-2.25), and the dormant pair last (2.26). Every page stands alone; the plane pages are the ones to read first if the question is “who talks to whom.”

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 What Changes vs pumice

The complete marking set, copied from the HAS (Ch 3, Table 3.1) rather than re-derived. Family doctrine 4 makes the markings a contract: this table, the HAS's, and the inventory on the previous page carry the same pairs, and a drift in any of them is a defect.

Table 1.3: The marking set, copied from the HAS

Block	Marking	Cause	MAS page
`scoria_top` / `scoria_top_geared` / `scoria_core`	INHERITED	structure unchanged	2.1-2.3
`scoria_axi4_plane`	INHERITED	the AXI4 host side is untouched by this generation	2.1
`scoria_scheduler_plane`	MODIFIED	must admit ZQ maintenance demand alongside refresh	2.2
`scoria_dfi_plane`	MODIFIED	DFI v3.1 control surface (per-data-phase CS, write-leveling handshake)	2.3
`scoria_init_sequencer`	MODIFIED	`RESET#` is a pin; MR order is MR2-MR3-MR1-MR0; ZQCL closes init	2.18
`scoria_mode_register`	MODIFIED	MR0-MR3 replaces MR0-MR2 plus EMRS3	2.19
`scoria_dfi_cmd_formatter`	MODIFIED	new encodings: `ZQCL`, `ZQCS`, `PREA`	2.22
`scoria_refresh_ctrl`	MODIFIED (v0.9)	the `REFpb` rotor and the JEDEC ±8 credit accumulator are inherited; TASK-	2.16

Block	Marking	Cause	MAS page
		001 Modes A/B (elastic pull-in/postpone streaks, TCR tREFI derate) are real changes behind CSRs	
scoria_zq_ctrl	**NEW**	periodic ZQCS is maintenance traffic; carries the CSR-selectable placement policy (TASK-001 Mode C)	2.17
scoria_wrlvl_ifc	**NEW**	DDR3 adds write leveling; the interface, not the search	2.20
scoria_powerdown_ctrl	INHERITED (dormant)	mechanism unchanged: CKE + SRE/SRX; not instantiated, by decision	2.26
scoria_dfi_signal_pack	INHERITED (dormant)	every signal it packs is unchanged for DDR3; its v3.1 new channels are driven at the plane above	2.26
everything else (20 FUBs: CAMs, intakes, timers, arbiter, splitter, chopper, mappers, CDC,	INHERITED	no functional change; see the inventory table for the full list	2.4-2.15, 2.21, 2.23-2.25

Block	Marking	Cause	MAS page
serdes, aligner, cmd_path)			

4.1 What the MAS adds to the marking

The HAS states the markings; this book is where the marked blocks live at signal level. Three additions are worth calling out on this page because they are easy to miss inside the chapter pages.

The MC-001 rename. The macro tier no longer uses pumice's naming: `scoria_axi4_ifc` is now `scoria_axi4_plane` and `scoria_mem_cmd_scheduler` is now `scoria_scheduler_plane` (trunk commit `3eee953d2`, the scoria half of the family-wide MC-001 convention). Older family documents — including parts of andesite's MAS — still cite the old names; the mapping is one-to-one and nothing else about the blocks changed in the rename.

`global_timers` is inherited in its FIXED form. pumice's original published readiness one cycle late: the registered status flop sampled the current counter state rather than the next, so the flags alone permitted tCCD and tRTW violations. The fix computes one next-state function and feeds both the counter and its status flop from it. This is not a cosmetic detail — a fresh implementation written from a behavioral description reintroduces the defect, because the description does not mention the flop. The page for the block (2.14) states the invariant as a contract; the formal proof (`formal/scoria/scoria_global_timers.sby`) closes it.

The dormant pair is a decision, not dead code. `powerdown_ctrl` and `dfi_signal_pack` stay in the tree uninstantiated because they are the declared starting point for LPDDR3 power-down work (self-refresh is a mobile part's normal idle state, and deep power-down needs the pair together). The DDR3-scoped justification — parity with the LiteDRAM reference, which has no power-down engine at all — does not transfer to LPDDR3. Page 2.26 carries the four-gap table that stands between today and that wake-up.

4.2 One paragraph per changed or new block

Scheduler plane (MODIFIED). The change is one extra maintenance source at the arbiter: ZQ requests and waits on the same request/grant terms refresh always had (family doctrine 2 — request and wait, never preempt). Everything else about the macro is pumice's scheduler with DDR3's timings. Page 2.2.

Init sequencer (MODIFIED). DDR3 brings a `RESET#` pin and a four-register MR set; the FSM walks RESET# low → CKE → tXPR → MR2 → MR3 → MR1 (DLL enable) → MR0 (DLL reset) → ZQCL → tDLLK/tZQinit. LPDDR3 walks MRW MR63 → MRW MR10 → MR1 → MR2 → MR3. Page 2.18.

Mode register (MODIFIED). MR0-MR3 shadows per rank with live decode of CL/CWL/BL/AL/WR; the RTL comment resolving JESD79-3F's prose-versus-Figure-9 inconsistency in favor of the figure is preserved verbatim on the page. Page 2.19.

Command formatter (MODIFIED). Three new encodings — `ZQCL`, `ZQCS`, `PREA` — join the inherited table, and the LPDDR3 side is the JESD209-2F Table 60 CA bus. Page 2.22.

Refresh controller (MODIFIED, v0.9). The inherited rotor and credit machine stand; TASK-001 adds elastic demand-aware pull-in/postpone (Mode A) and the TCR tREFI derate (Mode B), both CSR-gated and off by default. Page 2.16.

ZQ controller (NEW). Periodic `ZQCS` as ordinary maintenance traffic — interval CSR, request/grant to the arbiter, post-grant tZQCS hold, and the Mode C defer-under-demand placement policy. Page 2.17.

Write-leveling interface (NEW). Decision D2 in silicon: firmware drives the search; the RTL provides the DFI v3.1 leveling handshake, the MR1 write path, the `tWL*` window enforcement, and four-state telemetry. No delay-walk states exist anywhere in the block. Page 2.20.

5 Clocks, Reset, and Gearing

5.1 The two domains

scoria has exactly two clock domains, and the boundary between them is a single block.

Table 1.4: Clock domains

Domain	Signals	Clock	What lives there
Controller	`aclk` / `aresetn`	host/MC clock	AXI4 front-end, CAMs, return ring, scheduler (arbiter, timers, refresh/ZQ/init), the controller side of the CDC
PHY / DFI	`dfi_clk` / `dfi_rstn`	PHY clock	DFI command path, write serializer, read aligner, the PHY side of the CDC

Every crossing between them goes through `scoria_dfi_cdc` (page 2.25): five `gaxi_fifo_async` instances for commands, write data, read data, and the init/leveling event tokens. There are no hand-rolled synchronizers anywhere else in the design, and the DFI plane is deliberately never allowed to stall — all JEDEC spacing is enforced on the controller side, upstream of the FIFOs.

The init handshake crosses as rising-edge tokens and latches sticky on the far side: `dfi_init_start_o` (controller → PHY) and `dfi_init_complete_i` (PHY → controller). Software sees the far end of the same handshake as `STATUS.init_done`.

Both resets are active-low and asynchronous-assert, synchronous-deassert in the usual family style; each side of the CDC has its own reset (`ctl_rstn`/`dfi_rstn` derive from them inside the DFI plane).

5.2 The target design point

One operating point is characterized and gated today — the macro-tier consistency test (`test_scoria_dram_config_consistency.py`) cross-checks the framework's cycle-accurate view of every timing against the controller's programmed spacing table, so the numbers below are a gate, not a hope:

Table 1.5: The `genesys2_ddr3_800` operating point

Property	Value
Board / PHY	Genesys 2, K7 DDR3 PHY
Devices	2 x MT41J256M16, 32-bit DQ
Grade	DDR3-800 — 400 MHz DRAM clock, 3200 MB/s peak
MC clock (`aclk`)	75 MHz (13.33 ns) — restated by the owner 2026-10-10; 100 MHz was never the target (scoria BUG-003, closed)
`DFI_RATE`	4 (the RTL default of 2 is overridden at integration)
Burst	BL8
Address geometry	8 banks, row 15 / col 10

5.3 Width gearing

`scoria_core` ties the host AXI data width to the DFI word width (`DW = DRAM_BEAT_WIDTH * DFI_RATE`). When a host needs a different width, `scoria_top_geared` wraps `scoria_top` with formally verified AXI data-width converters (write and read directions) and enforces a power-of-two ratio between `HOST_AXI_DATA_WIDTH` and `DW`. When the widths already match, the converter is bypassed (`g_direct`) and existing builds are bit-identical.

5.4 Timing floors worth knowing up front

Three floor rules are set at integration in `scoria_core`/`scoria_top`, long before any block page gets to them:

- `w_t_rtw_eff` is floored at `t_rddata_en + RD_EN_CYC_TOP - t_phy_wrlat` (pumice BUG-014) so a write can never land inside the controller's own read window.
- `RD_EN_CYC` is overridden explicitly to `ceil(DRAM_BL / DFI_RATE)`; the aligner's parameter default would lose half a burst on narrow devices (page 2.24).
- `CMD_DELAY` defaults to auto (`5 + 2*BURST_WORDS`) — the scheduler's output FIFO releases each command only after its token matures, so write data always reaches the DFI no later than the command (page 2.2).

Two CSR-encoding landmines were disarmed at the top level and are recorded where they belong: the page-policy encoding is translated from software values to the internal enum in `scoria_top` (a raw cast swapped OPEN/CLOSE — issue #42), and `STATUS.init_done` plus the per-bank `OBS_ROW_HIT` registers are actually driven (they weren't, once, and software polled forever — pumice BUG-020).

6 AXI4 Plane (scoria_axi4_plane)

Module: scoria_axi4_plane.sv / **Location:** rtl/macro/ / **Category:** host interface / **Parent:** scoria_core / **Status:** complete and sim-verified.

6.1 Purpose

The AXI4 plane is the host-facing front end. It takes the full AXI4 slave interface, chops each burst into DFI-burst-sized sub-commands, maps each sub-command to `{rank, bank, row, col}`, buffers write data in the write-data CAM, and holds the read-command CAM plus the read-return ring. It exposes the scheduling window as flat per-entry vectors so the scheduler can pick bank-parallel commands without a CAM lookup round-trip.

The block is structural: no FSM, no storage of its own beyond the instantiated CAMs and FIFOs. Its RTL is inherited unchanged from pumice (verified by `test_scoria_pumice_logic_parity.py`).

6.2 Parameters

Table 2.1.1: AXI4 plane parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	Host AXI ID width
AXI_ADDR_WIDTH	int	32	Host address width
AXI_DATA_WIDTH	int	64	Host data width; also the DFI word width here
AXI_USER_WIDTH	int	1	AXI USER width
DRAM_BEAT_WIDTH	int	64	Misnomer; passed as the AXI width to size DRAM_BURST_BYTES only
NUM_RANKS	int	1	DRAM ranks
NUM_BANKS	int	8	DRAM banks
ROW_WIDTH	int	14	Row address width
COL_WIDTH	int	10	Column address width
BYTE_OFFSET_WIDTH	int	3	log2(device word bytes)
AXI_BEATS_PER_BURST	int	4	AXI beats per DFI burst / per sub-command
NUM_ENTRIES	int	8	CAM scheduling-window entries
N_SRAM_SLOTS	int	NUM_ENTRIES	Write-data SRAM slots
N_SCHED_LU	int	4	Scheduler lookup ports (unused in scoria)
AGE_WIDTH	int	16	Free-running age-

Parameter	Type	Default	Description
			counter width
RD_RET_DEPTH	int	32	In-flight read capacity; power of 2

AXI_DATA_WIDTH is the DFI word width because scoria_core instantiates the macro with DRAM_BEAT_WIDTH set to the AXI width. The front end is DFI-word granular: one AXI beat equals one DFI word.

6.3 Interface

6.3.1 Host AXI4 ports

The plane presents a full AXI4 slave. All `s_axi_*` names follow the AXI4 spec; standard signals are grouped by channel below.

Table 2.1.2: Host AXI4 interface

Group	Signals	Width notes
Write address	`s_axi_awid/addr/len/size/burst/lock/cache/prot/qos/region/user/valid`	`AW`, `IW`, 8, 3, 2, 1, 4, 3, 4, 4, `UW`
Write address handshake	`s_axi_awready`	1
Write data	`s_axi_wdata/wstrb/wlast/wuser/wvalid`	`DW`, `DW/8`, 1, `UW`
Write data handshake	`s_axi_wready`	1
Write response	`s_axi_bid/bresp/buser/bvalid`	`IW`, 2, `UW`
Write response handshake	`s_axi_bready`	1
Read address	`s_axi_arid/addr/len/size/burst/lock/cache/prot/qos/region/user/valid`	`AW`, `IW`, 8, 3, 2, 1, 4, 3, 4, 4, `UW`
Read address handshake	`s_axi_arready`	1
Read data	`s_axi_rid/rdata/rresp/rlast/ruser/rvalid`	`IW`, `DW`, 2, 1, `UW`
Read data handshake	`s_axi_rready`	1

6.3.2 Address-mapping CSR ports

Table 2.1.3: Address-mapping configuration ports

Signal	Direction	Width	Description
`bank_lsb_i`	in	5	`ADDR_MAP.bank_lsb`: bit index of bank in decoded address

Signal	Direction	Width	Description
hash_en_i	in	1	ADDR_MAP.hash_en: enable bank hashing
hash_seed_i	in	8	ADDR_MAP.hash_seed: hashing seed

6.3.3 Write scheduler and commit-data ports

Table 2.1.4: Write scheduler and commit-data ports

Signal	Direction	Width	Description
wr_sch_valid_o	out	NUM_ENTRIES	Per-entry valid bits
wr_sch_bank_o	out	NUM_ENTRIES * BKW	Per-entry bank
wr_sch_row_o	out	NUM_ENTRIES * ROW_WIDTH	Per-entry row
wr_sch_col_o	out	NUM_ENTRIES * COL_WIDTH	Per-entry column
wr_sch_older_o	out	NUM_ENTRIES^2	Flattened age-order matrix
wr_sch_age_exceed_o	out	NUM_ENTRIES	Entry age exceeds threshold
wr_sch_qos_o	out	NUM_ENTRIES * 4	Per-entry AXI QoS
wr_sch_head_rel_o	out	16	Relative age of head entry
wr_commit_valid_i	in	1	Scheduler commits a write entry
wr_commit_ready_o	out	1	Write CAM can accept the commit
wr_commit_slot_i	in	PTRW	Slot to commit
wr_cm_rd_valid_o	out	1	Commit-data stream valid
wr_cm_rd_ready_i	in	1	Commit-data stream ready
wr_cm_rd_data_o	out	AXI_DATA_WIDTH	Commit data

Signal	Direction	Width	Description
wr_cm_rd_strb_o	out	AXI_DATA_WIDTH/8	Commit strobe
wr_cm_rd_last_o	out	1	Final beat of the committed burst

6.3.4 Read scheduler and DFI return ports

Table 2.1.5: Read scheduler and DFI return ports

Signal	Direction	Width	Description
rd_sch_valid_o	out	NUM_ENTRIES	Per-entry valid bits
rd_sch_bank_o	out	NUM_ENTRIES * BKW	Per-entry bank
rd_sch_row_o	out	NUM_ENTRIES * ROW_WIDTH	Per-entry row
rd_sch_col_o	out	NUM_ENTRIES * COL_WIDTH	Per-entry column
rd_sch_older_o	out	NUM_ENTRIES^2	Flattened age-order matrix
rd_sch_age_exceed_o	out	NUM_ENTRIES	Entry age exceeds threshold
rd_sch_qos_o	out	NUM_ENTRIES * 4	Per-entry AXI QoS
rd_sch_head_rel_o	out	16	Relative age of head entry
sched_age_thresh_i	in	8	Age-threshold key for both CAMs
rd_issue_valid_i	in	1	Scheduler issues a read entry
rd_issue_ready_o	out	1	Read CAM can accept the issue
rd_issue_slot_i	in	PTRW	Slot to issue
rd_dfi_ret_valid_i	in	1	DFI read return valid
rd_dfi_ret_ready_o	out	1	Read-return ring can accept return

Signal	Direction	Width	Description
rd_dfi_ret_data_i	in	AXI_DATA_WIDTH	Return data
rd_dfi_ret_resp_i	in	2	Return response
rd_dfi_ret_last_i	in	1	Return last beat

6.3.5 Status port

Table 2.1.6: Status port

Signal	Direction	Width	Description
busy_o	out	1	OR of sub-module busy flags

6.4 Microarchitecture internals

6.4.1 Instantiation tree

The plane instantiates seven sub-blocks:

- `scoria_wr_splitter` — chops AW into DFI-burst sub-commands and reframes W.
- `scoria_wr_intake` — skid buffers, decoded push to the write CAM.
- `scoria_wr_data_cam` — write scheduling window and data SRAM.
- `scoria_axi_burst_chopper` — chops AR into DFI-burst sub-commands.
- `scoria_rd_intake` — read snarf probe, decoded miss push, return reorder.
- `scoria_rd_cmd_cam` — read scheduling window.
- `scoria_rd_return_ring` — AR-order in-flight read buffer.

6.4.2 Write datapath

Host write bursts enter through `scoria_wr_splitter`. The splitter uses `scoria_axi_burst_chopper` with `PAD_TO_CHUNK = 1` so every sub-command is declared as a full `AXI_BEATS_PER_BURST` beats. It regenerates `WLAST` every `AXI_BEATS_PER_BURST` beats and pads short host bursts with zero-strobe beats, holding `fub_wready` low during padding.

The split stream reaches `scoria_wr_intake`, which skid-buffers AXI, maps the head address to `{rank, bank, row, col}`, and pushes the decoded command into `scoria_wr_data_cam`. The intake buffers write data and pops it in CAM commit order.

`scoria_wr_data_cam` allocates one entry and one SRAM slot per sub-command. As W beats arrive they fill the SRAM. The scheduler reads the flat `wr_sch_*` vectors, commits an entry with `wr_commit_valid_i`/`wr_commit_slot_i`, and the CAM streams the committed burst data out on `wr_cm_rd_*`. On the final sub-command's last beat the CAM asserts `commit_done_valid_o` plus the stored ID so the intake can generate the host B response.

6.4.3 Read datapath

Host read bursts enter through `scoria_axi_burst_chopper` with `PAD_TO_CHUNK = 0`. Each sub-command carries `m_ax_agg` and `m_ax_last`; the read intake uses `last` to collapse `RLAST` and drops beats beyond the requested `AxLEN`.

`scoria_rd_intake` skid-buffers AR, probes `scoria_wr_data_cam` for a same-ID snarf hit, and admits misses to `scoria_rd_cmd_cam`. A two-stage admit pipeline lets the head AR stay probed while the previous AR is pushed, so admits can occur every cycle. Snarf hits stream directly from the CAM SRAM; misses go through the read CAM and return ring.

6.4.4 Read-after-write snarf

The snarf probe is registered: `scoria_rd_intake` presents `{bank, row, col, id, len}` to `scoria_wr_data_cam` and receives `snarf_hit_o` one cycle later. A hit is accepted only if it matches the youngest valid, fully-written, not-yet-scheduled entry with the same ID and burst length. The read intake's order FIFO tags snarf-sourced reads with `SRC_SNARF` so they merge correctly with DFI-sourced returns in AR order. Snarf is same-ID only.

6.4.5 Read-command CAM and return-ring cooperation

A read is admitted only when both the read CAM and the return ring have room:

```
ar_push_ready = rd_cam_ins_ready && rt_alloc_ready
```

The ring allocates a ticket in strict AR order. The ticket rides into the `scoria_rd_cmd_cam` entry at insert and is forwarded back out on `rd_issue_valid_i`. Once the read issues to DRAM, the CAM entry frees but the ticket follows the data through the return ring. The ring depth, not the CAM depth, bounds how many reads can be in flight.

6.4.6 Flat scheduler vectors and legacy port tie-offs

Both CAMs expose per-entry scheduler vectors (`sch_valid`, `sch_bank`, `sch_row`, `sch_col`, `sch_older`, `sch_age_exceed`, `sch_qos`, `sch_head_rel`) so the arbiter can do bank-parallel classification without a CAM lookup. The generic `sched_lu_*` and `oldest_*` ports are intentionally tied off: `sched_lu_valid_i` is driven to `'0` and the `oldest_*` outputs are left open. Synthesis prunes them.

6.5 FSM policy

There is no FSM in this plane. Burst chopping is combinational plus a single `r_active` bit in the chopper. CAM allocation, fill, and drain are governed by the CAM internal state, not by a plane-level controller.

6.6 Timing

All logic is in the `aclk` controller clock domain. The AXI splitter and chopper accept the first sub-command combinationally and hold `ready` low until the burst drains. The CAMs add one cycle of registered snarf-hit latency and one cycle of synchronous-read BRAM latency on commit-data and snarf-data returns.

6.7 Notes

- `DRAM_BEAT_WIDTH` is a misnomer in this plane. `scoria_core` instantiates the macro with `.DRAM_BEAT_WIDTH(DW)` where `DW` is the AXI data width, not the true DRAM beat width. The parameter survives only to size `DRAM_BURST_BYTES`.
- `AXI_DATA_WIDTH` equals the DFI word width because the front end is DFI-word granular.
- `AXI_BEATS_PER_BURST` is the sub-command granularity, not the JEDEC burst length. The same identifier has been used for three different quantities in the design.
- `scoria_axi4_plane` was renamed from the older `scoria_axi4_ifc`; some andesite pages and earlier scoria docs still use the old name. The module is the same pumice logic carried forward.

7 Scheduler Plane (scoria_scheduler_plane)

Module: scoria_scheduler_plane.sv / **Location:** rtl/macro/ / **Category:** command scheduling / **Parent:** scoria_core / **Status:** complete and sim-verified.

7.1 Purpose

The scheduler plane is the command brain. It wires the init sequencer, mode-register shadow, refresh and ZQ controllers, write-leveling interface, per-bank timers, global timers, page-policy engine, and command arbiter into one macro that emits a single abstract DRAM command per cycle to the DFI plane. All JEDEC spacing is enforced here; the DFI plane downstream is never allowed to stall.

> **HISTORICAL NOTE — MC-001 rename:** older docs, including some andesite pages, call the host plane `scoria_axi4_ifc` and this plane `scoria_mem_cmd_scheduler`. The landed RTL names are `scoria_axi4_plane` and `scoria_scheduler_plane`. The modules are the same logic.

7.2 Parameters

Table 2.2.1: Scheduler plane parameters

Parameter	Type	Default	Description
NUM_RANKS	int	1	DRAM ranks
CMD_HISTORY_EN	int	0	Optional issued-command history scoreboard
HIST_T_RCD … HIST_T_RTW	int	0/8	History-checker JEDEC windows
NUM_BANKS	int	8	DRAM banks
ROW_WIDTH	int	14	Row address width
COL_WIDTH	int	10	Column address width
AXI_ID_WIDTH	int	8	AXI ID width
NUM_ENTRIES	int	8	CAM scheduling-window entries
AGE_WIDTH	int	16	Age-counter width
CMD_FIFO_DEPTH	int	16	Output command FIFO depth
CMD_DELAY	int	6	Fixed release delay in aclk cycles
N_LU	int	NUM_BANKS	Internal lookup width
NUM_CS	int	NUM_RANKS	Chip-select count

7.3 Interface

7.3.1 Timing, mode, and maintenance configuration ports

Table 2.2.2: Timing, mode, and maintenance configuration ports

Signal group	Signals	Width notes
Page policy	page_policy_i	page_policy_e
Core timings	t_rcd_i, t_rp_i, t_ras_i, t_rc_i, t_wr_i, t_rtp_i, t_faw_i, t_rrd_i, t_wtr_i, t_rtw_i, t_ccd_i	8 each
Refresh timing	t_refi_i, refi_reload_i, t_rfc_i, refresh_burst_i, ref_postpone_i, ref_pullin_i, ref_mode_i, ref_trefi_pb_i, ref_trfc_pb_i	16, 1, 16, 4, 4, 4, 2, 16, 8
Init timing	t_init_wait_i, t_dll_wait_i, t_mrd_wait_i, t_rp_wait_i, t_rfc_wait_i, t_xpr_wait_i, t_zqinit_wait_i	16, 16, 8, 8, 8, 16, 16
Mode registers	mr0_i ... mr3_i, init_restart_i	16 x 4, 1
TASK-001 scheduler	sched_order_mode_i, sched_row_sel_i, sched_col_sel_i, sched_access_pref_i, sched_wr_high_wm_i, sched_wr_batch_max_i, sched_wr_low_wm_i, sched_prio_sub_i,	2, 2, 2, 2, 8, 8, 8, 2, 1

Signal group	Signals	Width notes
	sched_qos_en_i	
TASK-001 page/refresh	page_mode_i, page_tr_init_i, ref_elastic_en_i, ref_pullin_idle_streak_i, ref_postpone_demand_streak_i, ref_tcr_en_i, ref_trefi_derate_i	3, 8, 1, 8, 7, 1, 2
TASK-001 ZQ	zq_enable_i, zq_interval_i, t_zqcs_i, zq_placement_i, zq_overdue_max_i	1, 32, 16, 2, 13

7.3.2 CAM scheduler/commit/issue ports (from `scoria_axi4_plane`)

Table 2.2.3: CAM scheduler/commit/issue ports

Signal	Direction	Width	Description
wr_sch_valid_i	in	NUM_ENTRIES	Write CAM per-entry valid
wr_sch_bank_i	in	NUM_ENTRIES * BKW	Write CAM per-entry bank
wr_sch_row_i	in	NUM_ENTRIES * ROW_WIDTH	Write CAM per-entry row
wr_sch_col_i	in	NUM_ENTRIES * COL_WIDTH	Write CAM per-entry column
wr_sch_older_i	in	NUM_ENTRIES^2	Write CAM age-order matrix
wr_sch_age_exceed_i	in	NUM_ENTRIES	Write CAM age-exceed flags
wr_sch_qos_i	in	NUM_ENTRIES * 4	Write CAM per-entry QoS
wr_sch_head_rel_i	in	16	Write CAM head relative age
wr_commit_ready_	in	1	Write CAM can

Signal	Direction	Width	Description
i			accept commit
wr_commit_valid_o	out	1	Commit a write entry
wr_commit_slot_o	out	PTRW	Committed write slot
rd_sch_valid_i	in	NUM_ENTRIES	Read CAM per-entry valid
rd_sch_bank_i	in	NUM_ENTRIES * BKW	Read CAM per-entry bank
rd_sch_row_i	in	NUM_ENTRIES * ROW_WIDTH	Read CAM per-entry row
rd_sch_col_i	in	NUM_ENTRIES * COL_WIDTH	Read CAM per-entry column
rd_sch_older_i	in	NUM_ENTRIES^2	Read CAM age-order matrix
rd_sch_age_exceed_i	in	NUM_ENTRIES	Read CAM age-exceed flags
rd_sch_qos_i	in	NUM_ENTRIES * 4	Read CAM per-entry QoS
rd_sch_head_rel_i	in	16	Read CAM head relative age
rd_issue_ready_i	in	1	Read CAM can accept issue
rd_issue_valid_o	out	1	Issue a read entry
rd_issue_slot_o	out	PTRW	Issued read slot

7.3.3 Output command stream (to `scoria_dfi_plane`)

Table 2.2.4: Output command stream

Signal	Direction	Width	Description
cmd_valid_o	out	1	Command valid
cmd_ready_i	in	1	Command ready

Signal	Direction	Width	Description
cmd_op_o	out	dram_op_e	DRAM opcode
cmd_rank_o	out	RKW	Target rank
cmd_bank_o	out	BKW	Target bank
cmd_row_o	out	ROW_WIDTH	Row address
cmd_col_o	out	COL_WIDTH	Column address
cmd_ap_o	out	1	Auto-precharge flag

7.3.4 Init, PHY, mode-register shadow, and write-leveling ports

Table 2.2.5: Init, PHY, mode-register shadow, and write-leveling ports

Signal group	Signals	Width notes
PHY init	dfi_init_start_o, dfi_init_complete_i, init_done_o, dram_reset_n_o	1 each
MR shadow	cl_o, cwl_o, bl_o, mr_wr_o, wrlvl_en_o	4, 4, 4, 5, 1
WRLVL CSR	wrlvl_strobe_i, wrlvl_cs_sel_i, t_wldqsen_i, t_wlmrd_i, t_wlmrd_max_i, t_wlo_i, t_wloe_i	1, 4, 16, 16, 16, 16, 16
WRLVL PHY	dfi_phylvl_req_cs_n_o, dfi_phylvl_ack_cs_n_i, dfi_phy_wrlvl_cs_n_o, dfi_wrlvl_strobe_o, wrlvl_prime_dq_i	NUM_CS, NUM_CS, NUM_CS, 1, 1
WRLVL status	wrlvl_result_valid_o, wrlvl_result_o, wrlvl_attempts_o, wrlvl_flips_o, wrlvl_timeout_o, wrlvl_ever_done_o, wrlvl_state_o	1, 1, 16, 16, 1, 1, 3

7.3.5 Telemetry, stall attribution, and status ports

Table 2.2.6: Telemetry, stall attribution, and status ports

Signal group	Signals	Width notes
Page stats	`stat_page_hit_o`, `stat_row_hit_o[NUM_BANKS]`, `stat_page_miss_o`, `stat_page_empty_o`	32, 32 x banks, 32, 32
Command stats	`stat_act_o`, `stat_pre_o`, `stat_ref_o`, `stat_ref_busy_o`	32 each
Refresh/ZQ obs	`obs_ref_postpone_events_o`, `obs_ref_pullin_events_o`, `zq_busy_o`, `zq_total_o`, `zq_interval_cnt_o`, `zq_overdue_o`	16, 16, 1, 16, 32, 1
Stall attribution	`stall_bp_o`, `stall_refresh_o`, `stall_turnaround_o`, `stall_tccd_o`, `stall_actlimit_o`, `stall_banktimer_o`, `stall_noreq_o`, `stall_zq_o`	32 each
Status	`busy_o`	1

7.4 Microarchitecture internals

7.4.1 Instantiation tree

The scheduler plane instantiates:

- `scoria_init_sequencer` — JEDEC initialization sequence.
- `scoria_mode_register` — MR0-MR3 shadow and latency decode.
- `scoria_refresh_ctrl` — tREFI tracker and REF/REFpb scheduling.
- `scoria_zq_ctrl` — periodic ZQCS maintenance.
- `scoria_wrlvl_ifc` — DFI v3.1 write-leveling handshake.
- `scoria_bank_timers` — one `scoria_bank_timer` per `(rank, bank)`.
- `scoria_global_timers` — rank-global and global spacing windows.
- `scoria_page_policy` — runtime page-policy decisions and telemetry.
- `scoria_cmd_arbiter` — command pick core.
- `gaxi_fifo_sync` — output command FIFO.
- Optional `scoria_cmd_history_checker` — generate-gated audit block.

7.4.2 Command packing and output FIFO

The arbiter pushes accepted commands into a `gaxi_fifo_sync`. The FIFO data word is packed as:

```
CMD_W = 4 + RKW + BKW + ROW_WIDTH + COL_WIDTH + 1
w_cmd_wr_data = {a_cmd_ap, a_cmd_col, a_cmd_row, a_cmd_bank,
a_cmd_rank, a_cmd_op}
```

The downstream DFI plane unpacks this word onto the DFI command bus.

7.4.3 Fixed-delay release

Every push into the output FIFO also enters a `CMD_DELAY`-stage token shift register. The FIFO head is released only after the token matures, so every command reaches the DFI plane a fixed `CMD_DELAY` `aclk` cycles after arbitration. This keeps write data aligned with its command. Under DFI back-pressure the token counter accumulates matured-but-unpopped tokens.

7.4.4 busy_o definition

`busy_o` is asserted while any of the following is true:

```
busy_o = !init_done || refresh_req || w_cmd_rd_valid || (|
w_bank_row_active[0])
```

7.4.5 Refresh and ZQ gating

Refresh is gated on `init_done`. REFpb is enabled only when `ref_mode_i == 2'd2` and `memtype_i == MEMTYPE_LPDDR3`; otherwise REFpb degrades to REFab. ZQCS is run only for DDR3:

```
w_refpb_en = (ref_mode_i == 2'd2) && (memtype_i == MEMTYPE_LPDDR3)
w_zq_run   = zq_enable_i && init_done && (memtype_i == MEMTYPE_DDR3)
```

7.4.6 Page-policy tap

`scoria_page_policy` watches the arbiter's accepted command (`a_cmd_valid && a_cmd_ready`), not the FIFO output. Tapping the FIFO output would correlate each command with a stale bank image because the FIFO releases `CMD_DELAY` cycles later.

7.4.7 Optional command-history checker

When `CMD_HISTORY_EN != 0`, a `scoria_cmd_history_checker` audits the issued command sequence against JEDEC same-bank windows and emits simulation-only `$fatal` assertions. It is generate-gated OFF by default.

7.5 FSM policy

The scheduler plane has no FSM. It is a structural aggregation block. The arbiter is priority logic plus pipeline registers; the timers are decrement counters; refresh and ZQ are credit/state-counter machines. The init sequencer is the only state machine in this plane.

7.6 Timing

The plane operates entirely on `aclk`. The critical path runs from the CAM vectors through bank/global timer checks, FR-FCFS priority selection, and the final safety gate to the FIFO push. The output FIFO and `CMD_DELAY` token shift register add fixed release latency but no new combinational critical path.

7.7 Notes

- `scoria_scheduler_plane` was renamed from `scoria_mem_cmd_scheduler`; older andesite references use the old name.
- `scoria_init_sequencer` outputs `zqcl_req_o` and `init_busy_o` are tied off here.
- `scoria_mode_register` outputs `al_o`, `drv_strength_o`, `odt_o`, and `mr_req_o` are tied off here.
- The command-history checker is audit-only and must not be relied on for functional timing enforcement.

8 DFI Plane (scoria_dfi_plane)

Module: scoria_dfi_plane.sv / **Location:** rtl/macro/ / **Category:** DFI/PHY interface / **Parent:** scoria_core / **Status:** complete and sim-verified.

8.1 Purpose

The DFI plane owns the only controller-clock to PHY-clock boundary in the design. It carries commands, write data, and read data across asynchronous FIFOs; formats the abstract scheduler command onto the DFI command bus; serializes write data at `t_phy_wrlat`; aligns read data at `t_rddata_en`; and safely crosses the write-leveling handshake. All JEDEC spacing is enforced upstream in the scheduler; this plane is never allowed to stall.

8.2 Parameters

Table 2.3.1: DFI plane parameters

Parameter	Type	Default	Description
NUM_RANKS	int	1	DRAM ranks
NUM_CS	int	NUM_RANKS	Chip selects
NUM_BANKS	int	8	DRAM banks
ROW_WIDTH	int	14	Row address width
COL_WIDTH	int	10	Column address width
DFI_RATE	int	2	DFI phases per clock
DRAM_BEAT_WIDTH	int	64	DRAM beat width
DFI_BEATS_PER_BURST	int	8	DRAM beats per burst
N_SUBCMD	int	1	Sub-DFI-word commands per word
SUB_COL_STRIDE	int	1	Column stride between sub-commands
CMD_FIFO_DEPTH	int	8	Command async FIFO depth
WD_FIFO_DEPTH	int	16	Write-data async FIFO depth
RD_FIFO_DEPTH	int	32	Read-data async FIFO depth
N_FLOP_CROSS	int	2	Pointer synchronizer stages

Parameter	Type	Default	Description
USE_JOHNSON	int	0	0 = Gray pointers, 1 = Johnson
RD_MAX_OUTSTANDING	int	16	Read aligner outstanding depth
RD_EN_CYC	int	BL_WORDS	rddata_en window width
CMD_HISTORY_EN	int	0	DFI-wire command-history scoreboard

RD_FIFO_DEPTH = 32 was chosen because dfi_rddata_valid is fire-and-forget. A depth of 16 dropped beats under paced adjacent bursts.

8.3 Interface

8.3.1 Controller-domain ports

Table 2.3.2: Controller-domain ports

Signal	Direction	Width	Description
`ctl_clk`	in	1	Controller clock
`ctl_rstn`	in	1	Controller reset (active-low)
`cmd_valid_i`	in	1	Command valid from scheduler
`cmd_ready_o`	out	1	Command ready to scheduler
`cmd_data_i`	in	`CMD_DW`	Packed command `{ap, col, row, bank, rank, op}`
`wd_valid_i`	in	1	Write-data valid
`wd_ready_o`	out	1	Write-data ready
`wd_data_i`	in	`WD_DW`	Write data `{last, strb, data}`
`rd_valid_o`	out	1	Read-data valid
`rd_ready_i`	in	1	Read-data ready
`rd_data_o`	out	`RD_DW`	Read data `{last, resp, data}`
`init_start_i`	in	1	Init start level
`init_complete_o`	out	1	Init complete sticky

8.3.2 PHY DFI-domain ports

Table 2.3.3: PHY DFI-domain ports

Signal	Direction	Width	Description
`dfi_clk`	in	1	DFI clock
`dfi_rstn`	in	1	DFI reset (active-low)
`memtype_i`	in	`memtype_e`	DDR3 or

Signal	Direction	Width	Description
			LPDDR3
rd_phase_i	in	PHW	Read-command DFI phase
wr_phase_i	in	PHW	Write-command DFI phase
t_phy_wrlat_i	in	8	WR command to dfi_wrdata_en delay
t_rddata_en_i	in	8	RD command to dfi_rddata_en delay
gear_i	in	2	Active gear ratio selector
n_subcmd_i	in	SUBW_MAX	Active sub-commands per DFI word
sub_col_stride_i	in	COL_WIDTH	Active column stride
sub_phase_stride_i	in	PHW	Active phase stride
dfi_address_o	out	DFI_ADDR_BUS_W	DFI address bus
dfi_bank_o	out	DFI_BANK_BUS_W	DFI bank bus
dfi_cas_n_o	out	DFI_CTRL_BUS_W	DFI CAS#
dfi_ras_n_o	out	DFI_CTRL_BUS_W	DFI RAS#
dfi_we_n_o	out	DFI_CTRL_BUS_W	DFI WE#
dfi_cs_n_o	out	DFI_CS_BUS_W	DFI chip select

Signal	Direction	Width	Description
dfi_odt_o	out	DFI_CS_BUS_W	DFI ODT
dfi_wrdata_o	out	DFI_DATA_WIDTH	DFI write data
dfi_wrdata_en_o	out	DFI_EN_WIDTH	DFI write data enable
dfi_wrdata_mask_o	out	DFI_STRB_WIDTH	DFI write data mask
dfi_rddata_en_o	out	DFI_EN_WIDTH	DFI read data enable
dfi_rddata_i	in	DFI_DATA_WIDTH	DFI read data
dfi_rddata_valid_i	in	DFI_VALID_WIDTH	DFI read data valid
dfi_init_start_o	out	1	DFI init start sticky
dfi_init_complete_i	in	1	DFI init complete level

8.3.3 Write-leveling crossing ports

Table 2.3.4: Write-leveling crossing ports

Signal	Direction	Width	Description
wl_req_cs_n_i	in	NUM_CS	Leveling request from scheduler (active-low)
wl_wrlvl_cs_n_i	in	NUM_CS	WRLVL mode from scheduler (active-low)
wl_strobe_i	in	1	One-cycle strobe from scheduler
wl_ack_cs_n_o	out	NUM_CS	PHY ack to scheduler (active-low)
wl_prime_dq_o	out	1	Prime DQ to scheduler

Signal	Direction	Width	Description
dfi_phylvl_req_cs_n_o	out	NUM_CS	Leveled request to PHY (active-low)
dfi_phylvl_ack_cs_n_i	in	NUM_CS	PHY ack (active-low)
dfi_phy_wrlvl_cs_n_o	out	NUM_CS	Leveled WRLVL mode to PHY (active-low)
dfi_wrlvl_strobe_o	out	1	Leveled one-cycle strobe to PHY
dfi_prime_dq_i	in	1	Prime DQ from PHY

8.4 Microarchitecture internals

8.4.1 Instantiation tree

The DFI plane instantiates:

- `scoria_dfi_cdc` — the single controller/PHY async-FIFO crossing.
- `scoria_dfi_cmd_path` — DFI command bus formatter and fire-strobe generator.
- `scoria_dfi_wr_serializer` — `t_phy_wrlat` delay line to `dfi_wrdata_en`.
- `scoria_dfi_rd_aligner` — `t_rddata_en` delay line and read-data capture.
- `cdc_synchronizer` and `sync_pulse` — write-leveling clock-domain crossings.

8.4.2 Single CDC

All controller-to-PHY traffic crosses through `scoria_dfi_cdc` using `gaxi_fifo_async` Gray-pointer FIFOs. The block carries command, write data, read data, and init start/complete tokens. Level signals are converted to rising-edge tokens, crossed, and latched sticky on the far side. Write data and a one-bit "write burst staged" token are accepted together so a WR command cannot reach the PHY ahead of its data.

8.4.3 Active-gear phase mask

The active DFI rate is `w_active_rate = 1 << gear_i`. The unmasked `dfi_wrdata_en` and `dfi_rddata_en` from the serializer and aligner are gated by `w_phase_active` before reaching the pins. When `gear_i == log2(DFI_RATE)` the mask is all-ones and the outputs are bit-identical to the unmasked FUB outputs.

8.4.4 Command path

`scoria_dfi_cmd_path` owns the DFI command bus and produces `wr_fire_o` and `rd_fire_o` strobes. It does not pace commands; all JEDEC spacing lives upstream. It supports sub-DFI-word burst packing via `N_SUBCMD`, `SUB_COL_STRIDE`, and `SUB_PHASE_STRIDE`.

8.4.5 Write serializer

On `wr_fire_i`, the serializer waits `t_phy_wrlat_i` DFI cycles and then streams one DFI-word per cycle from the write-data FIFO onto `dfi_wrdata_o` until the burst's `last` word. It pops one word per DFI cycle with zero bubbles. `dfi_wrdata_mask_o` is driven as `~wd_strb_i`.

8.4.6 Read aligner

On `rd_fire_i`, the aligner drives `dfi_rddata_en_o` after `t_rddata_en_i` cycles and captures `BL_WORDS` DFI words into the read CDC FIFO. It tracks multiple outstanding reads and uses a

combinational credit counter to reject the PHY preamble valid before the true read window. `rd_last_o` is asserted when the captured word count reaches `BL_WORDS - 1`.

8.4.7 Write-leveling crossing

Write-leveling signals cross here because this plane owns the only `ctl_clk`/`dfi_clk` boundary. Slow levels use `cdc_synchronizer` flop chains; the one-cycle `wl_strobe_i` pulse uses `sync_pulse` so the pulse count survives arbitrary clock ratios. Active-low `_n` signals are inverted to active-high before synchronizing and inverted again at the output, avoiding spurious assertions out of reset.

8.5 FSM policy

There is no FSM in this plane. The CDC, command path, serializer, and aligner are all dataflow blocks. The only sequential state is in the FIFOs, delay-line shift registers, and synchronizers.

8.6 Timing

The command and write-data paths are fully registered on `dfi_clk` once they cross. The read path captures `dfi_rddata_i` combinationally when `dfi_rddata_valid_i` is asserted and the credit counter allows, then pushes into the async FIFO. The command-path fire strobes are combinational from the CDC command FIFO output.

8.7 Notes

- `RD_FIFO_DEPTH = 32` is required because `dfi_rddata_valid` cannot be back-pressured. Depth 16 was proven insufficient under paced adjacent bursts.
- `RD_EN_CYC` defaults to `BL_WORDS`, which is dangerous whenever `DRAM_BEAT_WIDTH > DRAM_DEVICE_WIDTH`. `scoria_core` overrides it explicitly with the true DQ occupancy.
- `scoria_dfi_signal_pack` is not instantiated here; the DFI bus is driven directly by the command and data paths.
- The write-leveling strobe must use `sync_pulse`, not a flop chain, because a one-cycle pulse would otherwise be duplicated or swallowed across an arbitrary clock ratio.

9 AXI Burst Chopper (`scoria_axi_burst_chopper`)

Module: `scoria_axi_burst_chopper.sv` **Location:** `rtl/fub/` **Category:** address-channel burst chopper **Parent:** `scoria_axi4_plane` **Status:** complete and sim-verified

---

9.1 Purpose

The AXI burst chopper splits one host AXI address-channel command into N sub-commands, each no longer than one DRAM burst. The downstream intakes assume one AXI burst equals one DFI burst, so this block is the request-side guarantee: every sub-command that leaves the chopper has at most `AXI_BEATS_PER_BURST` beats and steps to the next `AXI_BURST_BYTES`-aligned boundary.

The module is generic over AW and AR. The write path instantiates it with `PAD_TO_CHUNK=1` so short sub-commands are declared full-length and padded upstream by `scoria_wr_splitter`. The read path uses `PAD_TO_CHUNK=0`; short reads keep their true length because the R-channel framing follows AR.

9.2 Parameters

Table 2.4.1: AXI burst chopper parameters

Parameter	Type	Range	Default	Meaning
AXI_ID_WIDTH	int	—	8	AXI ID width
AXI_ADDR_WIDTH	int	—	32	AXI address width
AXI_USER_WIDTH	int	—	1	AXI USER width
STRB_BYTES	int	—	8	Bytes per AXI beat (AXI_DATA_WIDTH/8)
AXI_BEATS_PER_BURST	int	power of 2, >= 1	1	AXI beats per DFI burst
PAD_TO_CHUNK	int	0..1	0	1 = declare every sub-command as a full AXI_BEATS_PER_BURST beats

AXI_BEATS_PER_BURST must be a power of two; an initial assertion enforces this. The module derives AXI_BURST_BYTES = AXI_BEATS_PER_BURST * STRB_BYTES for address stepping.

9.3 Interface

9.3.1 Host address channel in

Signal	Direction	Width	Description
fub_axid	in	AXI_ID_WIDTH	AXI ID
fub_axaddr	in	AXI_ADDR_WIDTH	AXI address
fub_axlen	in	8	burst length (beats - 1)
fub_axsize	in	3	burst size
fub_axburst	in	2	burst type
fub_axlock	in	1	lock signal
fub_axcache	in	4	cache type
fub_axprot	in	3	protection type
fub_axqos	in	4	QoS value
fub_axregion	in	4	region identifier
fub_axuser	in	AXI_USER_WIDTH	user sideband
fub_axvalid	in	1	valid strobe
fub_axready	out	1	ready strobe

9.3.2 Sub-command address channel out

Signal	Direction	Width	Description
m_axid	out	AXI_ID_WIDTH	sub-command ID
m_axaddr	out	AXI_ADDR_WIDTH	sub-command address
m_axlen	out	8	sub-command length
m_axsize	out	3	sub-command size
m_axburst	out	2	sub-command burst type

Signal	Direction	Width	Description
m_axlock	out	1	sub-command lock
m_axcache	out	4	sub-command cache
m_axprot	out	3	sub-command protection
m_axqos	out	4	sub-command QoS
m_axregion	out	4	sub-command region
m_axuser	out	AXI_USER_WIDTH	sub-command user sideband
m_axvalid	out	1	sub-command valid
m_axready	in	1	sub-command ready

9.3.3 Aggregation sideband

Table 2.4.2: AXI burst chopper interface

Signal	Direction	Width	Description
m_ax_agg	out	1	high when the host command splits into more than one sub
m_ax_last	out	1	high on the final sub-command of the host burst

m_ax_agg is low for a single-sub command and high for every sub of a split. m_ax_last is high only on the last sub. The sideband rides with the address through the intake and into the CAM so the response path can collapse per-sub B/RLAST without a separate counter.

9.4 Microarchitecture internals

The chopper is FSM-free. A single `r_active` bit, a remaining-beat counter, and a running address drive all outputs. The first sub-command is presented combinationally from the host inputs; the host command is accepted when that first sub launches. Subsequent subs are presented from a latched copy while `fub_axready` stays low.

```
w_total = fub_axlen + 1
w_rem   = w_first ? w_total : r_rem
w_this  = min(w_rem, AXI_BEATS_PER_BURST)
w_addr  = w_first ? fub_axaddr : r_addr
m_axlen = PAD_TO_CHUNK ? AXI_BEATS_PER_BURST - 1 : w_this - 1
m_ax_agg = w_first ? (w_total > AXI_BEATS_PER_BURST) : 1'b1
m_ax_last = (w_rem <= AXI_BEATS_PER_BURST)
fub_axready = w_first && m_axready
```

On each accepted sub-command, `r_rem` decreases by `w_this` and `r_addr` advances by `AXI_BURST_BYTES`. When the last sub is accepted, `r_active` clears and `fub_axready` reopens.

9.5 FSM policy

There is no state machine. The only sequential state is the latched command and the running remainder/address. All output muxing is combinational.

9.6 Timing

The first sub-command is valid in the same cycle `fub_axvalid` is high. The critical path is the host input through the remainder calculation and address mux to `m_axvalid`/`m_axready`. Each subsequent sub-command is accepted one cycle after `m_axready` (registered advance). The host is back-pressured for the drain interval.

9.7 Notes

- **PAD_TO_CHUNK asymmetry:** The write side sets `PAD_TO_CHUNK=1` and relies on `scoria_wr_splitter` to inject zero-strobe filler beats. The read side sets `PAD_TO_CHUNK=0`; short reads need no padding because `RLAST` follows the real beat count.
- **Ragged but legal:** The chopper handles any host `AxLEN`, including bursts that are not a multiple of `AXI_BEATS_PER_BURST`. In scoria's normal path, ragged writes are padded by the splitter and ragged reads are rejected downstream; the chopper itself remains general.
- **Power-of-two assertion:** `initial assert (AXI_BEATS_PER_BURST >= 1 && power-of-2)` fires at elaboration if the parameter is mis-set.

10 Write Splitter (scoria_wr_splitter)

Module: scoria_wr_splitter.sv **Location:** rtl/fub/ **Category:** AXI write request reframer
Parent: scoria_axi4_plane **Status:** complete and sim-verified

---

10.1 Purpose

The write splitter sits between the host AXI write channel and `scoria_wr_intake`. It chops AW into DFI-burst-sized sub-commands and reframes the W stream so each sub-command receives exactly one `WLAST` on an `AXI_BEATS_PER_BURST` boundary. This is purely a request-side transform; B responses are consolidated later, on the return path.

The key change from the old shared `axi_master_wr_splitter` is that `WLAST` is regenerated from the W handshake, not from an AW-side beat budget. The previous scheme lost framing for narrow bursts that split into many single-beat DRAM bursts, leaving most sub-bursts without a `WLAST` and stalling the write-data CAM. Counting the W stream directly fixes that for any `AXI_BEATS_PER_BURST`, including one.

10.2 Parameters

Table 2.5.1: Write splitter parameters

Parameter	Type	Range	Default	Meaning
`AXI_ID_WIDTH`	int	—	8	AXI ID width
`AXI_ADDR_WIDTH`	int	—	32	AXI address width
`AXI_DATA_WIDTH`	int	—	64	AXI data width
`AXI_USER_WIDTH`	int	—	1	AXI USER width
`AXI_BEATS_PER_BURST`	int	power of 2, >= 1	1	AXI beats per DFI burst

The derived widths `IW`, `AW`, `DW`, `UW`, and `SW` (`AXI_DATA_WIDTH/8`) are local only.

10.3 Interface

10.3.1 Host AXI write in

Signal	Direction	Width	Description
fub_awid	in	AXI_ID_WIDTH	host AW ID
fub_awaddr	in	AXI_ADDR_WIDTH	host AW address
fub_awlen	in	8	host AW length
fub_awsize	in	3	host AW size
fub_awburst	in	2	host AW burst type
fub_awlock	in	1	host AW lock
fub_awcache	in	4	host AW cache
fub_awprot	in	3	host AW protection
fub_awqos	in	4	host AW QoS
fub_awregion	in	4	host AW region
fub_awuser	in	AXI_USER_WIDTH	host AW user
fub_awvalid	in	1	host AW valid
fub_awready	out	1	host AW ready
fub_wdata	in	AXI_DATA_WIDTH	host W data
fub_wstrb	in	AXI_DATA_WIDTH/8	host W strobe
fub_wlast	in	1	host W last
fub_wuser	in	AXI_USER_WIDTH	host W user
fub_wvalid	in	1	host W valid
fub_wready	out	1	host W ready

10.3.2 Sub-command AXI write out

Signal	Direction	Width	Description
m_awid	out	AXI_ID_WIDTH	sub-command

Signal	Direction	Width	Description
			AW ID
m_awaddr	out	AXI_ADDR_WIDTH	sub-command AW address
m_awlen	out	8	sub-command AW length
m_awsize	out	3	sub-command AW size
m_awburst	out	2	sub-command AW burst type
m_awlock	out	1	sub-command AW lock
m_awcache	out	4	sub-command AW cache
m_awprot	out	3	sub-command AW protection
m_awqos	out	4	sub-command AW QoS
m_awregion	out	4	sub-command AW region
m_awuser	out	AXI_USER_WIDTH	sub-command AW user
m_awvalid	out	1	sub-command AW valid
m_awready	in	1	sub-command AW ready
m_wdata	out	AXI_DATA_WIDTH	sub-command W data
m_wstrb	out	AXI_DATA_WIDTH/8	sub-command W strobe
m_wlast	out	1	regenerated W last
m_wuser	out	AXI_USER_WIDTH	sub-command W user

Signal	Direction	Width	Description
m_wvalid	out	1	sub-command W valid
m_wready	in	1	sub-command W ready

10.3.3 Aggregation sideband

Table 2.5.2: Write splitter interface

Signal	Direction	Width	Description
m_aw_agg	out	1	high when the host AW splits into more than one sub
m_aw_last	out	1	high on the final sub-command

10.4 Microarchitecture internals

The AW path is an instance of `scoria_axi_burst_chopper` with `PAD_TO_CHUNK=1`. The W path is a down-counter plus padding generator.

`r_wcnt` counts beats until the next regenerated `m_wlast`. It reloads to `AXI_BEATS_PER_BURST-1` on every chunk boundary. `r_pad` holds the number of zero-strobe filler beats owed when the host burst ends mid-chunk.

```
w_padding   = (r_pad != 0)
w_short_last = fub_wlast && (r_wcnt != 0)

m_wdata  = w_padding ? 0 : fub_wdata
m_wstrb  = w_padding ? 0 : fub_wstrb
m_wuser  = w_padding ? 0 : fub_wuser
m_wvalid = w_padding ? 1 : fub_wvalid
fub_wready = w_padding ? 0 : m_wready
m_wlast  = w_padding ? (r_pad == 1) : (r_wcnt == 0)
```

While padding, the host is held off (`fub_wready=0`) and the splitter emits locally generated filler beats. `m_wstrb=0` becomes `dfi_wrdata_mask_o = ~strb` in the DFI write serializer, so the device clocks the beat but writes nothing.

10.5 FSM policy

The splitter is FSM-free. The AW side inherits the single-bit active state of the burst chopper. The W side uses two counters: `r_wcnt` for chunk framing and `r_pad` for filler insertion.

10.6 Timing

AW and W are decoupled after the split. The W counter advances on `m_wvalid && m_wready`. Padding beats are inserted immediately when the host `WLAST` arrives before a chunk boundary. The first sub-command's AW is valid combinationally with the host `AWVALID`.

10.7 Notes

- **WLAST from W, not AW:** The regenerated `m_wlast` depends only on the W beat counter. This is the fix for narrow x16 BL4 bursts that previously lost framing.
- **Zero-strobe padding:** Short host bursts are padded to the chunk boundary with `strb=0`. Every legal `AxLEN` therefore works, including `AxLEN=0`.
- **PAD_TO_CHUNK=1:** AW sub-commands are always declared as full `AXI_BEATS_PER_BURST` beats because the W stream will supply the data, real or padded.
- **Backpressure during padding:** `fub_wready` is forced low during filler insertion. The host cannot push additional beats until the current chunk completes.

11 Write Intake (scoria_wr_intake)

Module: scoria_wr_intake.sv **Location:** rtl/fub/ **Category:** AXI4 write intake **Parent:** scoria_axi4_plane **Status:** complete and sim-verified

---

11.1 Purpose

The write intake is the “dumb” AXI4 write slave that sits between the splitter and the write-data CAM. It handles protocol skid buffering, buffers AW metadata and W data in small FIFOs, maps the head address to `{rank,bank,row,col}`, and pushes the decoded command to the CAM. Ragged bursts are rejected here with a self-generated `SLVERR` response; padding is the splitter’s job upstream.

The block intentionally does no scheduling, forwarding, or pointer management. Those live downstream in `scoria_wr_data_cam`.

11.2 Parameters

Table 2.6.1: Write intake parameters

Parameter	Type	Range	Default	Meaning
`AXI_ID_WIDTH`	int	—	8	AXI ID width
`AXI_ADDR_WIDTH`	int	—	32	AXI address width
`AXI_DATA_WIDTH`	int	—	64	AXI data width
`AXI_USER_WIDTH`	int	—	1	AXI USER width
`NUM_RANKS`	int	1+	1	DRAM ranks
`NUM_BANKS`	int	1+	8	DRAM banks
`ROW_WIDTH`	int	—	14	Row address width
`COL_WIDTH`	int	—	10	Column address width
`BYTE_OFFSET_WIDTH`	int	—	3	log2(device word bytes)
`RAGGED_ASSERT`	int	0..1	1	enable `$error` on ragged bursts
`AXI_BEATS_PER_BURST`	int	power of 2	4	AXI beats per DRAM burst
`AW_FIFO_DEPTH`	int	—	4	AW meta FIFO depth
`WDATA_FIFO_DEPTH`	int	—	16	write-data FIFO depth
`B_FIFO_DEPTH`	int	—	4	B response FIFO depth
`SKID_DEPTH_AW`	int	—	2	AW skid buffer depth
`SKID_DEPTH_W`	int	—	4	W skid buffer depth

Parameter	Type	Range	Default	Meaning
SKID_DEPTH_B	int	—	2	B skid buffer depth

11.3 Interface

11.3.1 AXI4 write slave

Signal	Direction	Width	Description
s_axi_awid	in	AXI_ID_WIDTH	AXI AW ID
s_axi_awaddr	in	AXI_ADDR_WIDTH	AXI AW address
s_axi_awlen	in	8	AXI AW length
s_axi_awsize	in	3	AXI AW size
s_axi_awburst	in	2	AXI AW burst type
s_axi_awlock	in	1	AXI AW lock
s_axi_awcache	in	4	AXI AW cache
s_axi_awprot	in	3	AXI AW protection
s_axi_awqos	in	4	AXI AW QoS
s_axi_awregion	in	4	AXI AW region
s_axi_awuser	in	AXI_USER_WIDTH	AXI AW user
s_axi_awvalid	in	1	AXI AW valid
s_axi_awready	out	1	AXI AW ready
aw_agg_i	in	1	aggregation sideband from splitter
aw_last_i	in	1	final-sub sideband from splitter
s_axi_wdata	in	AXI_DATA_WIDTH	AXI W data
s_axi_wstrb	in	AXI_DATA_WIDTH/8	AXI W strobe
s_axi_wlast	in	1	AXI W last
s_axi_wuser	in	AXI_USER_WIDTH	AXI W user

Signal	Direction	Width	Description
s_axi_wvalid	in	1	AXI W valid
s_axi_wready	out	1	AXI W ready
s_axi_bid	out	AXI_ID_WIDTH	AXI B ID
s_axi_bresp	out	2	AXI B response
s_axi_buser	out	AXI_USER_WIDTH	AXI B user
s_axi_bvalid	out	1	AXI B valid
s_axi_bready	in	1	AXI B ready

11.3.2 Downstream write-command push

Signal	Direction	Width	Description
aw_push_valid_o	out	1	push decoded command to CAM
aw_push_ready_i	in	1	CAM ready to accept
aw_push_rank_o	out	RKW	decoded rank
aw_push_bank_o	out	BKW	decoded bank
aw_push_row_o	out	ROW_WIDTH	decoded row
aw_push_col_o	out	COL_WIDTH	decoded column
aw_push_id_o	out	AXI_ID_WIDTH	command ID
aw_push_qos_o	out	4	command QoS
aw_push_err_o	out	1	ragged-burst flag
aw_push_agg_o	out	1	part of a split host burst
aw_push_last_o	out	1	final sub of the host burst

11.3.3 Write-data pop

Signal	Direction	Width	Description
wdata_valid_o	out	1	W data valid to DFI path
wdata_ready_i	in	1	DFI path ready
wdata_o	out	AXI_DATA_WIDTH	W data
wstrb_o	out	AXI_DATA_WIDTH/8	W strobe
wlast_o	out	1	W last

11.3.4 Write completion

Table 2.6.2: Write intake interface

Signal	Direction	Width	Description
wr_done_valid_i	in	1	downstream commit completion
wr_done_id_i	in	AXI_ID_WIDTH	completion ID
wr_done_resp_i	in	2	completion response

11.4 Microarchitecture internals

The intake contains three main FIFOs plus a sideband FIFO for aggregation bits.

Table 2.6.3: Write intake FIFOs

FIFO	Depth	Width	Push / Pop policy
AW meta	`AW_FIFO_DEPTH`	`3+4+IW+AW`	push on `fub_awvalid && fub_awready`; pop to `aw_push_ready_i`
Aggregation sideband	`SKID_DEPTH_AW + AW_FIFO_DEPTH`	2	push on `s_axi_awvalid && s_axi_awready`; pop locked to AW-meta write
Write data	`WDATA_FIFO_DEPTH`	`1+SW+DW`	push on `fub_wvalid`; pop to `wdata_ready_i`
B response	`B_FIFO_DEPTH`	`2+IW`	push on error or `wr_done_valid_i`; pop to AXI B

The `axi4_slave_wr` instance provides the protocol skid buffers. The aggregation sideband `aw_agg_i`/`aw_last_i` arrives aligned with `s_axi_aw` before the skid, so it is captured in a dedicated FIFO and replayed in lockstep with the AW-meta FIFO write.

Address mapping aligns the head address down to the AXI beat before calling `scoria_addr_mapper`:

```
BEAT_BO = $clog2(AXI_DATA_WIDTH / 8)
w_map_addr_wr = {w_head_addr[AW-1:BEAT_BO], {BEAT_BO{1'b0}}}
```

This prevents double-counting byte offsets when a wide AXI beat spans multiple device words. The byte lanes inside a beat are already carried by WSTRB; an unaligned start address must not also shift the column.

Ragged-burst detection is:

```
w_aw_err = ((fub_awlen + 1) != AXI_BEATS_PER_BURST)
```

A ragged burst sets `aw_push_err_o` and pushes a self-generated `SLVERR` into the B FIFO. Downstream drops the command rather than committing it.

11.5 FSM policy

There is no state machine. AW push, W pop, and B pop are independent FIFO handshakes. The only arbitration is the B-response FIFO write: an error B takes priority over a coincident `wr_done_valid_i`.

11.6 Timing

AW and W are buffered independently after the skid. The AW-meta FIFO head is decoded combinationally and pushed when the CAM is ready. The W-data FIFO pops when the downstream DFI path is ready. B responses pop when the AXI B channel is ready.

11.7 Notes

- **RAGGED_ASSERT:** When non-zero, the intake emits `$error` for ragged bursts. This is disabled only for directed SLVERR tests.
- **B-priority collision:** The intake asserts `$error` if an error B and a `wr_done_valid_i` would collide in the same cycle. The RTL gives priority to the error B, but the collision should never happen in normal operation.
- **Parameter-name history:** Comments in the RTL document an earlier confusion between `DRAM_BEAT_WIDTH` and `GEAR`. In scoria, `DRAM_BEAT_WIDTH` is passed as the AXI width and `GEAR` is always one, so the effective beat count is simply `AXI_BEATS_PER_BURST`.

12 Write Data CAM (`scoria_wr_data_cam`)

Module: `scoria_wr_data_cam.sv` **Location:** `rtl/fub/` **Category:** write scheduling window + data SRAM **Parent:** `scoria_axi4_plane` **Status:** complete and sim-verified

---

12.1 Purpose

The write-data CAM holds pending write commands keyed on `{bank,row,col}` and stores each burst's data in a dedicated SRAM slot. It is a scheduling window, a data buffer, a read-your-write bypass, and a rate-matched drain all in one: fill writes W beats into SRAM, snarf streams them back to a matching read, and commit streams them to the DFI write path.

12.2 Parameters

Table 2.7.1: Write data CAM parameters

Parameter	Type	Range	Default	Meaning
NUM_ENTRIES	int	1+	8	CAM scheduling-window entries
N_SCHED_LU	int	1+	4	scheduler lookup ports
NUM_BANKS	int	1+	8	DRAM banks
ROW_WIDTH	int	—	14	row address width
COL_WIDTH	int	—	10	column address width
AXI_ID_WIDTH	int	—	8	AXI ID width
AXI_DATA_WIDTH	int	—	64	AXI data width
AXI_BEATS_PER_BURST	int	power of 2	4	beats per burst
AGE_WIDTH	int	—	16	free-running age counter width
N_SRAM_SLOTS	int	1+	NUM_ENTRIES	SRAM slots (may be less than NUM_ENTRIES)
WR_DRAIN_AHEAD	int	1+	2	bursts allowed in commit-drain queue

12.3 Interface

12.3.1 Command insert

Signal	Direction	Width	Description
ins_valid_i	in	1	allocate a CAM entry
ins_ready_o	out	1	entry can be allocated
ins_bank_i	in	BKW	command bank
ins_row_i	in	ROW_WIDTH	command row
ins_col_i	in	COL_WIDTH	command column
ins_id_i	in	AXI_ID_WIDTH	command ID
ins_qos_i	in	4	command QoS
ins_agg_i	in	1	part of a split host burst
ins_last_i	in	1	final sub of the split host burst

12.3.2 Fill data

Signal	Direction	Width	Description
wd_valid_i	in	1	W data valid
wd_ready_o	out	1	CAM ready for W data
wd_data_i	in	AXI_DATA_WIDTH	W data
wd_strb_i	in	AXI_DATA_WIDTH/8	W strobe
wd_last_i	in	1	W last

12.3.3 Snarf lookup and stream

Signal	Direction	Width	Description
snarf_probe_valid_i	in	1	read probe

Signal	Direction	Width	Description
			valid
snarf_probe_bank_i	in	BKW	probe bank
snarf_probe_row_i	in	ROW_WIDTH	probe row
snarf_probe_col_i	in	COL_WIDTH	probe column
snarf_probe_id_i	in	AXI_ID_WIDTH	read AXI ID
snarf_probe_len_i	in	8	read arlen
snarf_hit_o	out	1	registered hit (one cycle after probe)
snarf_accept_i	in	1	AR admitted as snarf
snarf_rd_valid_o	out	1	snarf data valid
snarf_rd_ready_i	in	1	snarf data ready
snarf_rd_data_o	out	AXI_DATA_WIDTH	snarf data
snarf_rd_last_o	out	1	snarf last beat

12.3.4 Oldest entry port

Signal	Direction	Width	Description
oldest_valid_o	out	1	oldest valid entry exists
oldest_bank_o	out	BKW	oldest entry bank
oldest_row_o	out	ROW_WIDTH	oldest entry row
oldest_col_o	out	COL_WIDTH	oldest entry column
oldest_id_o	out	AXI_ID_WIDTH	oldest entry ID

Signal	Direction	Width	Description
oldest_slot_o	out	PTRW	oldest entry slot

12.3.5 Scheduler lookup ports

Signal	Direction	Width	Description
sched_lu_valid_i	in	N_SCHED_LU	per-port valid
sched_lu_bank_i	in	N_SCHED_LU*BKW	per-port bank
sched_lu_row_i	in	N_SCHED_LU*ROW_WIDTH	per-port row
sched_lu_hit_o	out	N_SCHED_LU	per-port hit
sched_lu_slot_o	out	N_SCHED_LU*PTRW	per-port slot
sched_lu_col_o	out	N_SCHED_LU*COL_WIDTH	per-port column
sched_lu_id_o	out	N_SCHED_LU*AXI_ID_WIDTH	per-port ID
sched_lu_age_o	out	N_SCHED_LU*AGE_WIDTH	per-port age

12.3.6 Per-entry scheduling vectors

Signal	Direction	Width	Description
sch_valid_o	out	NUM_ENTRIES	entry is schedulable
sch_bank_o	out	NUM_ENTRIES*BKW	per-entry bank
sch_row_o	out	NUM_ENTRIES*ROW_WIDTH	per-entry row
sch_col_o	out	NUM_ENTRIES*COL_WIDTH	per-entry column
sch_older_o	out	NUM_ENTRIES^2	flattened age-order matrix
age_thresh_i	in	8	age-threshold boost key
sch_age_exceed_o	out	NUM_ENTRIES	per-entry age-threshold flag

Signal	Direction	Width	Description
sch_qos_o	out	NUM_ENTRIES*4	per-entry QoS
sch_head_rel_o	out	AGE_WIDTH	relative age of oldest schedulable entry

12.3.7 Commit stream

Table 2.7.2: Write data CAM interface

Signal	Direction	Width	Description
commit_valid_i	in	1	scheduler commits a slot
commit_ready_o	out	1	commit accepted
commit_slot_i	in	PTRW	slot to commit
cm_rd_valid_o	out	1	commit data valid
cm_rd_ready_i	in	1	commit data ready
cm_rd_data_o	out	AXI_DATA_WIDTH	commit data
cm_rd_strb_o	out	AXI_DATA_WIDTH/8	commit strobe
cm_rd_last_o	out	1	commit last beat
commit_done_valid_o	out	1	B strobe on final sub evict
commit_done_id_o	out	AXI_ID_WIDTH	B ID

12.4 Microarchitecture internals

12.4.1 Entry state

Each CAM entry holds {valid, bank, row, col, id, qos, age, ptr, pv, fdone, sched, agg, last}. ptr is the SRAM slot, valid after the first fill beat. fdone marks fill complete; only then may the entry be scheduled or snarfed. sched marks scheduler commit and excludes the entry from further scheduling lookups.

12.4.2 Age ordering and the matrix

A free-running r_age_ctr increments every cycle. Relative age is w_rel[i] = r_age_ctr - r_age[i], which is wrap-safe. The scheduler needs the oldest entry, and the snarf path needs the youngest match.

The original scheduler path computed the oldest by a max-reduce over w_rel[], putting r_age_ctr and a chain of AGE_WIDTH subtract-compare-mux units directly on the scheduling critical path. On the first post-mode synthesis it measured 63.6 ns against a 15 ns period — pumice ISSUE-005.

The fix is the registered age-order matrix r_older[i][j]: bit [i][j] is high when entry i was inserted before entry j. Finding the oldest schedulable entry becomes a shallow NUM_ENTRIES^2 AND-reduce of 1-bit compares, exposed as sch_older_o; only the winner's relative age needs the AGE_WIDTH subtract.

```
// on insert of new slot s:
for every j:
    r_older[s][j] = 0
    if j != s: r_older[j][s] = 1
```

12.4.3 Snarf lookup

Read-your-write forwarding is limited to the safe case: the write is not yet scheduled (!r_sched), the AXI IDs match (same-id W-before-R is the only AXI-ordered case where the read must see the write), and the read's arlen equals AXI_BEATS_PER_BURST-1.

The probe from scoria_rd_intake is registered inside the CAM (r_sp_*) before the associative compare, so the cross-module route gets a full cycle and the hit is valid one cycle later. The match is the youngest among valid, filled, unscheduled entries with matching ID, bank, row, and column:

```
w_sn_match[i] = r_valid[i] && r_fdone[i] && !r_sched[i]
                && (r_id[i]   == r_sp_id)
                && (r_bank[i] == r_sp_bank)
                && (r_row[i]  == r_sp_row)
                && (r_col[i]  == r_sp_col)

is_young[i] = w_sn_match[i] &&
```

```
              for all j != i: !(w_sn_match[j] && !r_older[j][i])
snarf_hit_o = r_sp_valid && |is_young && (r_sp_len ==
AXI_BEATS_PER_BURST-1)
```

12.4.4 SRAM and shared read port

The data array is one block RAM of `N_SRAM_SLOTS * AXI_BEATS_PER_BURST` words, each packing `{strb, data}`. It has one write port (fill) and one read port shared by snarf and commit; the read is synchronous, registered in `r_rd_q`. A 2-deep prefetch skid FIFO decouples fetch from consume so beat 0 of the next burst prefetches while the current tail drains. Each skid entry carries a full tag (`iscm`, `slot`, `blast`, `agg`, `slast`, `id`) so the consume side never needs the source-FIFO head.

Because there is only one read port, commit has priority and a sustained drain could starve snarf forever. `r_sn_wait` counts cycles a pending snarf is preempted; at `SN_STARVE_LIMIT = 16` snarf wins the port for one fetch, bounding forwarded-read latency without a second read port.

```
w_sn_starved = (r_sn_wait >= SN_STARVE_LIMIT)
w_sn_fetch = room && sq_valid && (!dq_valid || w_sn_starved)
w_cm_fetch = room && dq_valid && !w_sn_fetch
```

12.4.5 Rate-matched commit

A write may commit only while fewer than `WR_DRAIN_AHEAD` bursts sit in the drain queue. This prevents the command stream from running far ahead of the data path — with the old “room in the FIFO” gate the command stream could run more than a dozen commands ahead. The arbiter decides a write one cycle before it fires, so `commit_ready_o` answers whether a commit decided now will be accepted next cycle, using `w_dq_occ_next`.

```
w_commit_fire = commit_valid_i && w_dq_wr_ready
                && (r_dq_occ < WR_DRAIN_AHEAD)
w_dq_occ_next = r_dq_occ + w_commit_fire - w_dq_pop
commit_ready_o = w_dq_wr_ready && (w_dq_occ_next < WR_DRAIN_AHEAD)
```

12.4.6 Eviction and B consolidation

The CAM entry and its SRAM slot are freed at **fetch-last**, not consume-last. The B strobe and its ID/agg/last tags ride the skid entry, so the response needs no live entry. Fetch-last eviction removes downstream latency from the entry lifetime; with consume-last eviction, eight entries could not sustain a burst every four cycles.

`commit_done_valid_o` fires only on the final sub of a split host burst, consolidating sub-burst responses into one host B:

```
commit_done_valid_o = w_cm_fire && w_hd_blast && (!w_hd_agg ||
w_hd_slast)
```

Non-split bursts (`agg=0`) always strobe. Split bursts strobe only when `slast` is high.

12.5 FSM policy

There is no central state machine. State is the entry array, SRAM occupancy, and the fetch-beat pointers; the active stream is the head of the relevant FIFO.

12.6 Timing

Insert is combinational once a free entry and fill-queue slot exist. Snarf hit is registered one cycle after the probe. SRAM read latency is exactly one cycle. The scheduler critical path is now the age-order matrix reduce, a 1-bit operation instead of a wide age subtract chain.

12.7 Notes

- **BRAM read latency trap:** The read latency is exactly one cycle. Do not add an output-register stage or register the read address without deepening the `r_if_*` tag pipeline to match; a mismatch pairs every captured beat with the wrong tag and causes silent data corruption.
- **Why same-ID snarf only:** AXI ordering guarantees W-before-R visibility only within the same ID. Cross-id reads take the DRAM path.
- **WR_DRAIN_AHEAD=2:** Bounds the command-to-data lead to a fixed pipeline latency rather than a FIFO depth.
- **Fetch-last eviction:** Frees the entry and SRAM slot when the last word is fetched, not when the downstream consumer takes it. This is what lets a small CAM cover the write bandwidth.
- **SN_STARVE_LIMIT=16:** A latency bound, not a bandwidth guarantee. A sustained commit stream can still average one snarf fetch every 16 cycles, which is acceptable because snarf is an optimization, not the primary read path.

13 Read Intake (scoria_rd_intake)

Module: scoria_rd_intake.sv **Location:** rtl/fub/ **Category:** AXI4 host interface **Parent:** scoria_axi4_plane **Status:** complete and sim-verified

---

13.1 Purpose

The read intake converts post-split AXI4 read requests into either a snarf hit against the write CAM or a miss command bound for the read command CAM. It owns the AR protocol skid buffer, address decode, the snarf probe handshake, the order FIFO that preserves AR order, and the source arbiter that merges snarf and DFI return data back onto the AXI R channel. It does not schedule; it only admits, tags, and reorders.

13.2 Parameters

Table 2.8.1: Read intake parameters

Parameter	Type	Range	Default	Meaning
AXI_ID_WIDTH	int	1+	8	host AXI ID width
AXI_ADDR_WIDTH	int	1+	32	host address width
AXI_DATA_WIDTH	int	1+	64	host/DFI word width
AXI_USER_WIDTH	int	1+	1	AXI USER width
NUM_RANKS	int	1+	1	DRAM ranks
NUM_BANKS	int	1+	8	DRAM banks
ROW_WIDTH	int	1+	14	row address width
COL_WIDTH	int	1+	10	column address width
BYTE_OFFSET_WIDTH	int	1+	3	log2(device word bytes)
AXI_BEATS_PER_BURST	int	power of 2	4	AXI beats per DRAM burst
AR_FIFO_DEPTH	int	1+	4	AR meta FIFO depth
ORDER_FIFO_DEPTH	int	1+	8	AR-order / source-tag FIFO depth
RD_FIFO_DEPTH	int	1+	16	read-data FIFO depth
SKID_DEPTH_AR	int	1+	2	AR skid buffer depth
SKID_DEPTH_R	int	1+	4	R channel skid buffer depth

`scoria_axi4_plane` overrides `ORDER_FIFO_DEPTH` to `RD_RET_DEPTH + 8` so the FIFO can hold one slot for every in-flight read plus the snarf hits that never enter the return ring.

13.3 Interface

13.3.1 AXI4 read slave

Table 2.8.2: AXI4 read slave interface

Signal	Direction	Width	Description
aclk	in	1	controller clock
aresetn	in	1	active-low reset
s_axi_arid	in	AXI_ID_WIDTH	AR ID
s_axi_araddr	in	AXI_ADDR_WIDTH	AR address
s_axi_arlen	in	8	AR burst length
s_axi_arvalid	in	1	AR valid
s_axi_arready	out	1	AR ready
ar_agg_i	in	1	high when the host burst split into >1 sub-command
ar_last_i	in	1	high on the final sub-command
s_axi_rid	out	AXI_ID_WIDTH	R ID
s_axi_rdata	out	AXI_DATA_WIDTH	R data
s_axi_rresp	out	2	R response
s_axi_rlast	out	1	R last
s_axi_rvalid	out	1	R valid
s_axi_rready	in	1	R ready
bank_lsb_i	in	5	address-map bank LSB
hash_en_i	in	1	address-hash enable
hash_seed_i	in	8	address-hash seed

13.3.2 Read command miss path

Table 2.8.3: Read command miss path

Signal	Direction	Width	Description
ar_push_valid_o	out	1	miss command valid to read CAM
ar_push_ready_i	in	1	read CAM ready
ar_push_rank_o	out	$clog2(NUM_RANKS) or 1	decoded rank
ar_push_bank_o	out	$clog2(NUM_BANKS)	decoded bank
ar_push_row_o	out	ROW_WIDTH	decoded row
ar_push_col_o	out	COL_WIDTH	decoded column
ar_push_id_o	out	AXI_ID_WIDTH	AXI ID
ar_push_qos_o	out	4	AxQOS to CAM

13.3.3 Snarf probe to write CAM

Table 2.8.4: Snarf probe interface

Signal	Direction	Width	Description
snarf_probe_valid_o	out	1	probe valid
snarf_probe_rank_o	out	rank width	probe rank
snarf_probe_bank_o	out	bank width	probe bank
snarf_probe_row_o	out	ROW_WIDTH	probe row
snarf_probe_col_o	out	COL_WIDTH	probe column
snarf_probe_id_o	out	AXI_ID_WIDTH	read AXI ID
snarf_probe_len_o	out	8	read AxLEN
snarf_hit_i	in	1	registered hit from write

Signal	Direction	Width	Description
			CAM
snarf_accept_o	out	1	AR admitted as snarf

13.3.4 Snarf data stream

Table 2.8.5: Snarf data stream

Signal	Direction	Width	Description
snarf_rd_valid_i	in	1	snarf data valid from write CAM
snarf_rd_ready_o	out	1	snarf data ready
snarf_rd_data_i	in	AXI_DATA_WIDTH	snarf data
snarf_rd_last_i	in	1	snarf last beat

13.3.5 DFI read-return stream

Table 2.8.6: DFI read-return stream

Signal	Direction	Width	Description
dfi_rd_valid_i	in	1	DFI return data valid
dfi_rd_ready_o	out	1	DFI return data ready
dfi_rd_data_i	in	AXI_DATA_WIDTH	DFI return data
dfi_rd_last_i	in	1	DFI return last beat
dfi_rd_resp_i	in	2	DFI return response
busy_o	out	1	OR of internal busy flags

13.4 Microarchitecture internals

13.4.1 Two-stage AR admit pipeline

The snarf probe inside the write CAM is registered, so a probe presented on cycle `c` only produces its hit on cycle `c+1`. The AR being admitted must therefore sit one cycle behind the AR being probed. The old implementation used a single `r_armed` bit on the skid head: probe the head, wait one cycle, admit it. Because the arm cleared on admit and could not re-arm until the next cycle, admits were capped at one every two cycles. One admitted sub-command equals one DRAM burst, so the gate halved read bandwidth on geometries where one burst is one AXI beat — the x16 BL4 case measured on the board (pumice BUG-011).

The fix stages the AR:

```
skid head  -> probed this cycle
stage reg  -> admitted this cycle
```

The skid head advances whenever the stage frees, so a sub-command is admitted every cycle while the downstream path is ready. While the stage is held because downstream is not ready, the probe is re-pointed at the stage itself rather than the head, so the hit is refreshed every cycle. The admit decision is never more than one cycle old, matching the exposure of the original arm bit but without the throughput penalty.

13.4.2 Snarf probe handshake

The probe is combinational out of `scoria_rd_intake` but registered inside `scoria_wr_data_cam`. The hit `snarf_hit_i` belongs to whatever was probed last cycle, which by construction is the AR now in the admit stage. A hit forces the snarf path: the write CAM holds the youngest matching same-ID entry that is fully written, and the read streams from the CAM's SRAM instead of DRAM. Snarf is same-ID only; cross-ID reads are sent to DRAM even if the data is present, because the CAM key does not include ID and allowing cross-ID snarf would forward data belonging to a different transaction.

13.4.3 Order FIFO

Every admitted read pushes one tag into the order FIFO in AR order. The tag is `{agg, last, source, len, id}`:

- `agg` and `last` come from the aggregation sideband and are used to collapse per-sub `RLAST`.
- `source` is `SRC_DFI` (0) for a miss or `SRC_SNARF` (1) for a hit.
- `len` is the sub-command's `AxLEN`.
- `id` is the AXI ID.

The order FIFO is popped when the last beat of the current sub-command is consumed from whichever source the head selects.

13.4.4 Beat dropping and RLAST generation

A DRAM burst always returns `AXI_BEATS_PER_BURST` beats, but the host may have asked for fewer. Beats past the requested `AxLEN` are consumed from the source and dropped rather than returned to the host. Without this, a single-beat read would receive the full DRAM burst and the R channel would never frame correctly.

`RLAST` is asserted on the last requested beat, not on the last returned DRAM beat. For a split host burst (`agg=1`), the host only sees `RLAST` on the last requested beat of the final sub-command (`last=1`); non-split reads pass `RLAST` per sub-command unchanged.

13.4.5 Source arbiter and read-data FIFO

The source arbiter selects between the snarf stream and the DFI return stream based on the order-FIFO head's `source` bit. The selected beat is pushed into the read-data FIFO as `{resp, last, id, data}`. The FIFO decouples the source-side handshake from the final AXI R channel, which is driven from the FIFO head.

13.5 FSM policy

There is no state machine. Control is distributed across the skid buffer, the one-cycle admit stage, the order FIFO, and the source arbiter. The only counter is the per-sub forward counter `r_fwd`, which counts beats forwarded so far and resets on every DRAM burst boundary.

13.6 Timing

AR admission is one cycle from skid head to `ar_push_valid_o` or `snarf_accept_o`. The snarf hit is available one cycle after the probe, matching the write CAM's registered lookup. DFI return data is returned to the host in AR order with one read-data FIFO cycle of elasticity.

The critical throughput path is the AR admit stage: in steady state the stage admits and reloads in the same cycle, so the rate is one sub-command per cycle.

13.7 Notes

- **BUG-011 fix:** the two-stage pipeline replaced the one-cycle arm bit. The old design halved x16 BL4 read bandwidth because one sub-command is one AXI beat there; the board measurement was 291.7 MB/s for reads versus 570 MB/s for writes, consistent with a one-every-two-cycles gate.
- **Probe freshness:** latching the hit would be cheaper but wrong. A latched hit goes stale against writes entering the write CAM behind the probe, which is exactly the RAW-forwarding case the snarf path exists to catch.
- **Same-ID snarf only:** the probe includes the read ID, and the write CAM match requires ID equality. Cross-ID reads always take the DRAM path.
- **Address alignment:** the AR address is aligned down to the AXI beat before address mapping. This prevents double-counting byte offsets when a wide AXI beat spans multiple device words; the byte lanes are already carried by the AXI strobes on the write side.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

14 Read Command CAM (scoria_rd_cmd_cam)

Module: scoria_rd_cmd_cam.sv **Location:** rtl/fub/ **Category:** scheduling window **Parent:** scoria_axi4_plane **Status:** complete and sim-verified; formal suite formal/mc_common/mc_rd_cmd_cam/rd_cmd_cam.sby

---

14.1 Purpose

The read command CAM is the scheduling window for read sub-commands. It holds up to `NUM_ENTRIES` entries keyed by `{bank, row, col}`, tracks the age order of those entries, and exposes the per-entry vectors the scheduler reads directly. An entry lives only from insert to issue; the return-ring ticket that accompanies a read through DRAM is stored in the entry and forwarded to the return ring on issue.

The CAM deliberately does not hold returned data. Decoupling the scheduling window from the in-flight data buffer is what lets the controller support far more in-flight reads than `NUM_ENTRIES`.

14.2 Parameters

Table 2.9.1: Read command CAM parameters

Parameter	Type	Range	Default	Meaning
NUM_ENTRIES	int	1+	8	CAM scheduling-window entries
N_SCHED_LU	int	1+	4	generic scheduler lookup ports
NUM_BANKS	int	1+	8	DRAM banks
ROW_WIDTH	int	1+	14	row address width
COL_WIDTH	int	1+	10	column address width
AXI_ID_WIDTH	int	1+	8	AXI ID width
AGE_WIDTH	int	1+	16	free-running age-counter width
RD_RET_DEPTH	int	power of 2	32	return-ring depth; bounds ticket space

14.3 Interface

14.3.1 Insert

Table 2.9.2: Insert interface

Signal	Direction	Width	Description
ins_valid_i	in	1	allocate a CAM entry
ins_ready_o	out	1	a free slot exists
ins_bank_i	in	$clog2(NUM_BANKS)	decoded bank
ins_row_i	in	ROW_WIDTH	decoded row
ins_col_i	in	COL_WIDTH	decoded column
ins_id_i	in	AXI_ID_WIDTH	AXI ID
ins_qos_i	in	4	AxQOS
ins_ticket_i	in	$clog2(RD_RET_DEPTH)	return-ring ticket from scoria_rd_return_ring

14.3.2 Scheduler vectors

Table 2.9.3: Flat per-entry scheduler vectors

Signal	Direction	Width	Description
sch_valid_o	out	NUM_ENTRIES	per-entry valid
sch_bank_o	out	NUM_ENTRIES * bank_width	per-entry bank
sch_row_o	out	NUM_ENTRIES * ROW_WIDTH	per-entry row
sch_col_o	out	NUM_ENTRIES * COL_WIDTH	per-entry column
sch_older_o	out	NUM_ENTRIES²	flattened age-order matrix
age_thresh_i	in	8	age-threshold boost key
sch_age_exceed_o	out	NUM_ENTRIES	per-entry age-exceed flag

Signal	Direction	Width	Description
sch_qos_o	out	NUM_ENTRIES * 4	per-entry AxQOS
sch_head_rel_o	out	AGE_WIDTH	relative age of oldest schedulable entry

14.3.3 Generic scheduler lookup ports

Table 2.9.4: Generic scheduler lookup ports

Signal	Direction	Width	Description
sched_lu_valid_i	in	N_SCHED_LU	lookup valid
sched_lu_bank_i	in	N_SCHED_LU * bank_width	lookup bank
sched_lu_row_i	in	N_SCHED_LU * ROW_WIDTH	lookup row
sched_lu_hit_o	out	N_SCHED_LU	lookup hit
sched_lu_slot_o	out	N_SCHED_LU * $clog2(NUM_ENTRIES)	hit slot
sched_lu_col_o	out	N_SCHED_LU * COL_WIDTH	hit column
sched_lu_id_o	out	N_SCHED_LU * AXI_ID_WIDTH	hit ID
sched_lu_age_o	out	N_SCHED_LU * AGE_WIDTH	hit relative age

14.3.4 Oldest fallback port

Table 2.9.5: Oldest fallback port

Signal	Direction	Width	Description
oldest_valid_o	out	1	an entry exists
oldest_bank_o	out	bank width	bank of oldest entry
oldest_row_o	out	ROW_WIDTH	row of oldest entry
oldest_col_o	out	COL_WIDTH	column of oldest entry
oldest_id_o	out	AXI_ID_WIDTH	ID of oldest

Signal	Direction	Width	Description
			entry
oldest_slot_o	out	$clog2(NUM_ENTRIES)	slot of oldest entry

14.3.5 Issue notify

Table 2.9.6: Issue notify interface

Signal	Direction	Width	Description
issue_valid_i	in	1	scheduler issued this slot
issue_ready_o	out	1	ring's issue-order FIFO ready
issue_slot_i	in	$clog2(NUM_ENTRIES)	issued slot
iss_valid_o	out	1	ticket forward valid
iss_ready_i	in	1	ring's issue-order FIFO ready
iss_ticket_o	out	$clog2(RD_RET_DEPTH)	ticket forwarded to return ring
busy_o	out	1	any entry valid

14.4 Microarchitecture internals

14.4.1 Entry fields

Each CAM entry stores:

```
{valid, bank, row, col, id, qos, ticket, age}
```

The ticket is the return-ring slot allocated when the read was admitted. It stays with the entry from insert until issue, at which point it is forwarded to the return ring's issue-order FIFO. The CAM entry is freed on the issue cycle; the read's data return is owned entirely by the return ring from that point on.

14.4.2 Ticket carried from insert to issue

`scoria_axi4_plane` allocates a return-ring ticket at the same time it inserts a CAM entry. The ticket is presented on `ins_ticket_i` and latched into `r_ticket[slot]`. On a valid issue fire (`w_issue_fire`), the ticket is driven onto `iss_ticket_o` with `iss_valid_o` high. The issue-ready handshake comes from the return ring's issue-order FIFO, so the CAM cannot issue faster than the ring can accept tickets.

14.4.3 Age-order matrix

Relative age is `w_rel[i] = r_age_ctr - r_age[i]`. The CAM does not expose a wide age key to the scheduler. Instead it maintains a registered `r_older[i][j]` matrix where bit `r_older[i][j]` means entry `i` is older than entry `j`. The matrix is updated only on insert: the new slot is youngest, so it is older than nobody, and every existing slot becomes older than it.

14.4.4 Oldest via 1-bit AND-reduce

The scheduler-side oldest entry is found with a 1-bit AND-reduce over the matrix. For each slot `i`:

```
w_sho_is[i] = r_valid[i] AND (for all j != i: r_valid[j] -> r_older[i][j])
```

The winner is the highest-numbered slot whose `w_sho_is` is true. Only the winner's relative age needs the `AGE_WIDTH` subtract, which is how the matrix replaced the original wide age-compare path (pumice ISSUE-005).

14.4.5 `sch_age_exceed_o` versus `age_thresh_i`

`age_thresh_i` is a runtime CSR from the scheduler. The per-entry flag is registered:

```
r_age_exceed[i] = r_valid[i] && (age_thresh_i != 0) && (w_rel[i] >= {age_thresh_i, 4'h0})
```

The flag is one cycle stale, which is acceptable because the age-threshold order mode uses it as a preference key, not as a safety guard. `sch_age_exceed_o[i] = r_valid[i] && r_age_exceed[i]`.

14.4.6 Unused ports tied off in scoria

The generic `sched_lu_*` lookup ports and the `oldest_*` fallback port are not used by `scoria_cmd_arbiter`. The scheduler reads the flat per-entry vectors directly so it can activate banks in parallel without a lookup round-trip. In `scoria_axi4_plane` the lookup inputs are tied to zero and the outputs are left unconnected.

14.5 FSM policy

There is no state machine. The CAM is a set of registers, comparators, and priority encoders. Insert allocation picks the highest-numbered free slot; issue frees the selected slot and forwards its ticket.

14.6 Timing

Insert is combinational from `ins_valid_i` to `ins_ready_o`. The scheduler vectors are registered fields with one level of derive. Issue is combinational from `issue_valid_i` to `iss_valid_o`, qualified by `issue_ready_o` from the return ring.

14.7 Notes

- **Input-contract assertion at `scoria_rd_cmd_cam.sv:321-322`:** a non-synthesis assertion fires if the scheduler issues a slot that is not valid. This is an input contract on the issuer, not a property of the CAM. It is mirrored here because the block historically had no dedicated DV suite. The same contract is stated as an assumption in `formal/mc_common/mc_rd_cmd_cam/formal_scoria_rd_cmd_cam.sv`, so the formal proof takes it as given.
- **Formal coverage now exists:** the block has a dedicated `scoria_rd_cmd_cam.sby` suite. The simulation assertion remains as an independent cross-check and is guarded from synthesis.
- **CAM depth is not the in-flight limit:** `NUM_ENTRIES` is only the scheduling window. The number of reads the controller can hold in flight is bounded by `scoria_rd_return_ring.DEPTH`, not by this CAM.

15 Read Return Ring (`scoria_rd_return_ring`)

Module: `scoria_rd_return_ring.sv` **Location:** `rtl/fub/` **Category:** in-flight read data buffer
Parent: `scoria_axi4_plane` **Status:** complete and sim-verified; formal suite `formal/mc_common/mc_rd_return_ring/rd_return_ring.sby`

15.1 Purpose

The read return ring decouples the read scheduling CAM from in-flight read data. It allocates a ticket in AR order when a read is admitted, accepts the ticket into an issue-order FIFO when the read issues, lands DFI return beats into the ticket's slot, and drains completed reads back to the read intake in AR order. Because the CAM entry is freed at issue, the number of reads the controller can hold in flight is bounded by this ring's depth, not by the CAM's entry count.

15.2 Parameters

Table 2.10.1: Read return ring parameters

Parameter	Type	Range	Default	Meaning
DEPTH	int	power of 2	32	in-flight reads
AXI_DATA_WIDTH	int	1+	64	data width
AXI_BEATS_PER_BURST	int	1+	4	beats per DRAM burst
TW	int	derived	$clog2(DEPTH)	ticket width
BCW	int	derived	$clog2(AXI_BEATS_PER_BURST) or 1	beat-counter width
OCW	int	derived	$clog2(DEPTH + 1)	occupancy-counter width

15.3 Interface

15.3.1 Allocate

Table 2.10.2: Ticket allocation interface

Signal	Direction	Width	Description
alloc_valid_i	in	1	allocate a ticket
alloc_ready_o	out	1	ring not full
alloc_ticket_o	out	TW	allocated ticket = current tail

15.3.2 Issue notify

Table 2.10.3: Issue-order FIFO interface

Signal	Direction	Width	Description
issue_valid_i	in	1	ticket has issued to DRAM
issue_ready_o	out	1	issue-order FIFO not full
issue_ticket_i	in	TW	ticket that issued

15.3.3 DFI read return

Table 2.10.4: DFI read return interface

Signal	Direction	Width	Description
dfi_ret_valid_i	in	1	DFI return beat valid
dfi_ret_ready_o	out	1	a ticket is at the issue-Q head
dfi_ret_data_i	in	AXI_DATA_WIDTH	return data
dfi_ret_resp_i	in	2	return response

Signal	Direction	Width	Description
dfi_ret_last_i	in	1	last beat of this burst

15.3.4 Drain and observability

Table 2.10.5: Drain and observability interface

Signal	Direction	Width	Description
drain_valid_o	out	1	drain data valid
drain_ready_i	in	1	drain ready
drain_data_o	out	AXI_DATA_WIDTH	drain data
drain_resp_o	out	2	drain response
drain_last_o	out	1	last beat of this read
occ_o	out	OCW	allocated tickets (observability)
busy_o	out	1	ring not idle

15.4 Microarchitecture internals

15.4.1 Tickets in AR order

Allocation always claims the current tail slot. `alloc_ticket_o = r_tail`, and on a valid allocation the tail increments. The head pointer tracks the oldest allocated slot; the slot is freed when its last beat is consumed by the drain. Occupancy `r_occ` increments on allocate and decrements on free.

15.4.2 Issue-order FIFO

When the scheduler issues a read, the CAM forwards the ticket to the ring's issue-order FIFO. The FIFO is sized `DEPTH`; because a ticket is pushed at most once per allocation, it can never fill before the ring itself does. DFI return beats always land in the issue-order FIFO head's slot, so `dfi_ret_ready_o = w_iq_rd_valid`. The FIFO pops when a complete burst (`dfi_ret_last_i`) has been written into the slot.

15.4.3 DFI returns land in the issue-Q head slot

The BRAM data array is indexed by ticket and beat:

```
w_ret_idx = ticket * AXI_BEATS_PER_BURST + r_ret_beat
```

The return beat is written whenever `dfi_ret_valid_i && dfi_ret_ready_o`. The slot's `ready` bit is set and its `resp` field is captured on the burst's last beat.

15.4.4 2-deep prefetch skid over BRAM

The ring uses a fetch pointer `r_fptr` that walks allocated slots in ring order and reads each slot's beats as soon as the slot is complete. A 2-deep skid buffer sits over the synchronous-read BRAM so the next slot can prefetch while the current slot's tail drains. Without the prefetch skid, a one-beat-per-slot geometry would cap the drain at one beat every three cycles.

15.4.5 Head freed at last-beat consume

The head pointer advances only when the drain consumes the last beat of the head slot:

```
w_free = drain_valid_o && drain_ready_i && w_hd_blast
```

This keeps the occupancy count tied to actual data consumption rather than to fetch completion.

15.4.6 Occupancy output

`occ_o` reports the current allocated-but-not-freed count. In `scoria_axi4_plane` this output is left unconnected; it exists for observability and for the formal proof assumptions.

15.4.7 DEPTH bounds in-flight reads

`DEPTH = 32` is a power of 2. It is the return-ring depth, not the CAM depth (`NUM_ENTRIES = 8`), that bounds read bandwidth. With a 27-cycle DRAM round trip, 32 in-flight reads lets the controller sustain one read per cycle across the latency, whereas eight entries held for the full round trip would have capped bandwidth at roughly 180 MB/s on the board.

15.5 FSM policy

There is no state machine. The ring is a head/tail pointer pair, a per-slot ready/resp array, an issue-order FIFO, and a prefetch skid over the BRAM.

15.6 Timing

Allocation is combinational from `alloc_valid_i` to `alloc_ready_o`. DFI return beats are accepted combinationally when a ticket is at the issue-Q head. BRAM read latency is exactly one cycle, matched by the fetch pipeline. Drain is combinational from the skid head to `drain_valid_o`.

15.7 Notes

- **Two non-synthesis input-contract assertions** at `scoria_rd_return_ring.sv:280-283`:
  - `assert (!(dfi_ret_valid_i && !w_iq_rd_valid))` — a DFI return beat arrived with no ticket in flight.
  - `assert (!(issue_valid_i && w_empty))` — a ticket was issued while the ring was empty.
- **Formal treatment:** both assertions are stated as assumptions in `formal/mc_common/mc_rd_return_ring/formal_rd_return_ring.sv`. The proof takes them as given, and the simulation checks remain as independent cross-checks.
- **BRAM latency trap:** the data storage is reset-free so Vivado maps it to Block RAM. Read latency is exactly one cycle; adding an output register without matching the fetch tag pipeline would silently corrupt the data path.

---

16 Address Mapper (scoria_addr_mapper)

Module: scoria_addr_mapper.sv **Location:** rtl/fub/ **Category:** address decode **Parent:** scoria_axi4_plane (instantiated by the intakes) **Status:** complete, sim-verified, formal-proven

---

16.1 Purpose

`scoria_addr_mapper` turns a flat AXI byte address into the `{rank, bank, row, col}` tuple the scheduling CAMs store. It is pure combinational: one cycle of barrel-shift and mask logic, with no state. The mapping is controlled at runtime by a single CSR knob, `bank_lsb_i`, which decides where the bank field sits in the byte-offset-stripped word address. That one knob generates every common mapping scheme, so there is no scheme mux.

The block is instantiated once on the write path and once on the read path, inside `scoria_axi4_plane`.

16.2 Parameters

Table 2.11.1: Address mapper parameters

Parameter	Type	Range	Default	Meaning
AXI_ADDR_WIDTH	int	—	32	host byte-address width
NUM_RANKS	int	1, 2, 4	1	ranks per channel
NUM_BANKS	int	4, 8	8	banks per rank
ROW_WIDTH	int	—	14	row address width
COL_WIDTH	int	—	10	column address width
BYTE_OFFSET_WIDTH	int	—	3	log2(device bytes per beat); 3 for a 64-bit data path

BYTE_OFFSET_WIDTH strips the beat byte offset. What remains is a device-word address; the column walk of one DRAM burst must stay inside a single bank, which is what constrains bank_lsb_i.

16.3 Interface

16.3.1 Address and configuration ports

Table 2.11.2: Address mapper interface

Signal	Direction	Width	Description
axi_addr_i	in	AXI_ADDR_WIDTH	host byte address (AW or AR channel)
bank_lsb_i	in	5	bank field LSB in the word address
hash_en_i	in	1	enable bank XOR-hash
hash_seed_i	in	8	XOR-hash seed
rank_o	out	$clog2(NUM_RANKS > 1 ? NUM_RANKS : 2)	decoded rank
bank_o	out	$clog2(NUM_BANKS)	decoded bank, optionally hashed
row_o	out	ROW_WIDTH	decoded row
col_o	out	COL_WIDTH	decoded column

16.4 Microarchitecture internals

16.4.1 Byte-offset strip and bank_lsb clamp

The mapper first strips the beat byte offset and zero-extends to 32 bits:

```
w_word = axi_addr_i[AXI_ADDR_WIDTH-1 : BYTE_OFFSET_WIDTH]
```

`bank_lsb_i` is clamped to `[0, COL_WIDTH]` so every field slice stays legal:

```
w_blsb = (bank_lsb_i > COL_WIDTH) ? COL_WIDTH : bank_lsb_i
```

`bank_lsb == COL_WIDTH` places the bank above the whole column, giving a row-major mapping. Any value below `COL_WIDTH` splits the column around the bank, producing interleaving.

16.4.2 Field extraction

The word address is sliced into four variable-base fields:

```
col_lo = w_word & ((1 << w_blsb) - 1)
bank   = (w_word >> w_blsb) & ((1 << BW) - 1)
col_hi = (w_word >> (w_blsb + BW)) & ((1 << (COL_WIDTH - w_blsb)) - 1)
row    = (w_word >> (COL_WIDTH + BW)) & ((1 << ROW_WIDTH) - 1)
rank   = (NUM_RANKS > 1) ? (w_word >> (COL_WIDTH + BW + ROW_WIDTH)) :
0
```

The column is reassembled from the low and high pieces:

```
col = col_lo | (col_hi << w_blsb)
```

Row and rank stack above the column region; their LSB positions are invariant. Only the bank position moves with `bank_lsb_i`.

16.4.3 Bank XOR-hash

When `hash_en_i` is high, each bank bit is folded with row bits and the seed:

```
bank_hashed[i] = bank_raw[i] ^ row[i] ^ row[MID] ^ seed[i]
MID = (i + BW < ROW_WIDTH) ? (i + BW) : (ROW_WIDTH - 1)
```

The `MID` index is clamped so it never runs past `ROW_WIDTH`. The fold matches the legacy `XOR_HASH` scheme and breaks power-of-two-stride hot-spots.

16.5 FSM policy

There is no FSM. The block is a single combinational stage.

16.6 Timing

All outputs are combinational from the inputs. The critical path is the barrel-shift/mask chain plus the optional XOR-hash. The block is duplicated on the AW and AR paths, so read and write addresses decode in parallel.

16.7 Notes

- **Bank-LSB bound.** Software must keep `bank_lsb_i >= log2(DRAM_BL)`, where `DRAM_BL` is the JEDEC burst length in device words. That bound keeps one DRAM burst inside a single bank. The RTL only clamps the upper bound to `COL_WIDTH`; the lower bound is a software contract.
- **Measured, not derived.** The lower bound was established by measurement rather than by formula. Two earlier versions of the constraint existed in the tree: `log2(cols/burst)` (too restrictive) and `log2(BL/DFI_RATE)` (too permissive and unsafe). Commit `f60e5ba1d` replaced both with the measured bound `log2(DRAM_BL)`; see the RTL header and the test `dv/tests/fub/test_scoria_addr_mapper.py::minimum_bank_lsb_is_measured`.
- **Formal proof.** The block is proven in `formal/scoria/addr_mapper/scoria_addr_mapper.sby`.
- **Rank degeneration.** With `NUM_RANKS == 1`, `rank_o` is tied to zero.

---

17 Page Policy (scoria_page_policy)

Module: scoria_page_policy.sv **Location:** rtl/fub/ **Category:** scheduling policy / telemetry
Parent: scoria_scheduler_plane **Status:** complete, sim-verified

---

17.1 Purpose

`scoria_page_policy` watches the issued command stream and the per-bank row state, and produces two things: the auto-precharge decision fed back to the arbiter, and a set of page/row telemetry counters. It does not itself issue commands; it only tells the arbiter whether the current mode wants a bank closed, and it counts what happened.

The block is modeless in hardware — it has no FSM — and is selected by the `policy_mode_i` CSR at runtime.

17.2 Parameters

Table 2.12.1: Page policy parameters

Parameter	Type	Range	Default	Meaning
NUM_RANKS	int	1+	1	ranks per channel
NUM_BANKS	int	4..16	8	banks per rank
ROW_WIDTH	int	—	14	row address width
BKW	int	derived	$clog2(NUM_BANKS)	bank index width
RKW	int	derived	$clog2(NUM_RANKS) if NUM_RANKS > 1, else 1	rank index width

17.3 Interface

17.3.1 Configuration and command tap

Table 2.12.2: Page policy inputs

Signal	Direction	Width	Description
`policy_mode_i`	in	3	page-policy mode select
`tr_init_i`	in	8	idle-timeout reload value
`cmd_valid_i`	in	1	arbiter accept strobe: `cmd_valid && cmd_ready`
`cmd_op_i`	in	`dram_op_e`	issued command opcode
`cmd_bank_i`	in	`BKW`	issued command bank
`cmd_row_i`	in	`ROW_WIDTH`	issued command row
`bank_row_active_i`	in	`NUM_BANKS`	per-bank row-valid flag from bank timers
`bank_open_row_i`	in	`NUM_BANKS*ROW_WIDTH`	per-bank open row from bank timers
`demand_i`	in	1	any CAM entry schedulable

17.3.2 Decisions and telemetry

Table 2.12.3: Page policy outputs

Signal	Direction	Width	Description
`ap_mode_en_o`	out	1	mode 1/2/3 override the legacy `w_ap`
`ap_close_o`	out	`NUM_BANKS`	per-bank auto-

Signal	Direction	Width	Description
			precharge close request
timeout_pre_req_o	out	1	background timeout-precharge request
timeout_pre_bank_o	out	BKW	bank selected for timeout precharge
stat_page_hit_o	out	32	every issued column op
stat_page_miss_o	out	32	ACT after a conflict close
stat_page_empty_o	out	32	ACT after a non-conflict close
stat_act_o	out	32	every issued ACT
stat_pre_o	out	32	every issued PRE/PREA
stat_ref_o	out	32	every issued REF
stat_row_hit_o	out	32 × NUM_BANKS	column op with no pending ACT for that bank
stat_ref_busy_o	out	32	REFs issued while demand_i was high

17.4 Microarchitecture internals

17.4.1 Modes

`policy_mode_i` selects among five live encodings; values 4-7 are retired and fall through to the build default.

Table 2.12.4: Page-policy modes

Mode	Name	`ap_mode_en_o`	`ap_close_o`	Close mechanism
0	default / legacy flat	0	0	legacy `page_policy_i` (`OPEN`/`CLOSE`)
1	static_open	1	all 0	none — rows stay open
2	static_close	1	all 1	auto-precharge on every column
3	fixed_open	1	all 0	background idle timeout
4-7	retired	—	—	fall through to mode 0

Modes 4 and 5 (`adapt_time` and `adapt_access`) were retired after silicon measurements showed them equivalent to or worse than the fixed modes. Modes 6 and 7 (`rbl_static` and `rbl_dyn`) were retired after measurements showed them harmful or converging to mode 0. A write to 4-7 now behaves like the build default.

17.4.2 Per-bank idle timeout

In `fixed_open` mode, each open bank runs an 8-bit down-counter:

```
if (row closed or mode off):
    idle = 0; expired cleared
else if (command to this bank):
    idle = tr_init_i; expired = 0
else if (idle != 0):
    idle = idle - 1
    if (idle == 1 and tr_init_i != 0):
        expired = 1
```

`r_expired` is sticky: it raises at zero and holds until the row closes.

17.4.3 Background close request

When `fixed_open` mode is on, the block scans the banks and requests a precharge on the lowest-numbered bank that is both open and expired. The arbiter issues the actual PRE at its lowest priority, gated by `bank_pre_ready_o` and the normal 2-cycle guards. This block never touches the command wires.

17.4.4 Telemetry counters

The counters are free-running and cleared on reset; software subtracts two reads to form a window.

- `stat_page_hit_o` increments on every issued column op.
- `stat_page_miss_o` vs `stat_page_empty_o`: a single-bank precharge to a bank that is **not** timeout-closed sets `r_conflict_mark[bank]`. The next ACT to a marked bank is a miss; otherwise it is empty.
- `stat_row_hit_o[b]` counts a column op to bank `b` that arrives with no pending ACT for that bank.
- `stat_ref_busy_o` counts refreshes that fire while `demand_i` is high.

The conflict mark uses `w_is_pre1` (single-bank PRE only). PREA from refresh drain deliberately does **not** mark any bank, because its bank field is not an address and would otherwise make one arbitrary bank look like a conflict.

17.5 FSM policy

No FSM. The mode is a CSR and the rest is combinational priority plus flop counters.

17.6 Timing

All decision outputs are combinational from the registered bank state and the issued command. Telemetry counters update on the cycle the command is accepted.

17.7 Notes

- **CSR encoding translation lives in `scoria_top`.** The legacy `REFRESH_TUNING.page_policy_or` CSR uses the software encoding `0=build default, 1=OPEN, 2=CLOSE, 3=HYBRID`, while `page_policy_e` inside the RTL is `OPEN=0, CLOSE=1, HYBRID=2`. A raw cast swapped OPEN and CLOSE and corrupted the entire open-page / reorder config axis (issue #42). `scoria_top` now performs an explicit `unique case` translation before driving `page_policy_i`. The `policy_mode_i` port of `scoria_page_policy` is already the translated enum.
- **Mode 0 is not mode 1.** Mode 0 means "use the legacy `page_policy_i` input"; mode 1 means "force open page". They produce the same result only when the legacy input already requests open page.
- **`tr_init_i == 0` disables the timeout.** That matches the CSR definition of 0 as build default.

18 Bank Timer (scoria_bank_timer + scoria_bank_timers)

Module: scoria_bank_timer.sv, scoria_bank_timers.sv **Location:** rtl/fub/ **Category:** JEDEC timing **Parent:** scoria_scheduler_plane **Status:** complete, sim-verified

---

18.1 Purpose

`scoria_bank_timers` is a thin wrapper that stamps one `scoria_bank_timer` instance for every `(rank, bank)` pair and fans the scheduler's command-event strobes to the addressed instance. `scoria_bank_timer` tracks a single bank's JEDEC timing constraints with preset/decrement countdown timers and a row-open register. There is no FSM; the per-command `safe_*` outputs are combinational off the timer flops.

18.2 Parameters

18.2.1 scoria_bank_timers

Table 2.13.1: Bank timers wrapper parameters

Parameter	Type	Range	Default	Meaning
NUM_RANKS	int	1+	1	ranks per channel
NUM_BANKS	int	4..16	8	banks per rank
ROW_WIDTH	int	—	14	row address width
RKW	int	derived	$clog2(NUM_RANKS) or 1	rank index width
BKW	int	derived	$clog2(NUM_BANKS)	bank index width
BANK_LA	int	0+	0	advisory lookahead depth passed to each timer

18.2.2 scoria_bank_timer

Table 2.13.2: Single bank timer parameters

Parameter	Type	Range	Default	Meaning
ROW_WIDTH	int	—	14	row address width
TW	int	—	8	timer counter width
LA	int	0+	0	advisory lookahead depth in cycles

18.3 Interface

18.3.1 Wrapper event and readiness ports

Table 2.13.3: Bank timers wrapper interface

Signal	Direction	Width	Description
evt_act_i	in	1	activate event
evt_rd_i	in	1	read event
evt_wr_i	in	1	write event
evt_pre_i	in	1	precharge event
evt_ap_i	in	1	auto-precharge qualifier
evt_rank_i	in	RKW	event rank
evt_bank_i	in	BKW	event bank
evt_row_i	in	ROW_WIDTH	event row
bank_act_ready_o	out	NUM_RANKS*NUM_BANKS	per-bank safe to activate
bank_rdwr_ready_o	out	NUM_RANKS*NUM_BANKS	per-bank safe to read/write
bank_pre_ready_o	out	NUM_RANKS*NUM_BANKS	per-bank safe to precharge
bank_row_active_o	out	NUM_RANKS*NUM_BANKS	per-bank row-valid flag
bank_open_row_o	out	NUM_RANKS*NUM_BANKS*ROW_WIDTH	per-bank open row
bank_state_o	out	NUM_RANKS*NUM_BANKS	derived bank_state_e

The wrapper also exports lookahead readiness twins (bank_act_ready_la_o, bank_rdwr_ready_la_o, bank_pre_ready_la_o) and observability flags.

18.3.2 Single-timer ports

Table 2.13.4: Single bank timer command inputs

Signal	Direction	Width	Description
t_rcd_i	in	TW	ACT-to-RD/WR

Signal	Direction	Width	Description
			spacing
t_rp_i	in	TW	PRE-to-ACT spacing
t_ras_i	in	TW	min ACT-to-PRE spacing
t_rc_i	in	TW	same-bank ACT-to-ACT spacing
t_wr_i	in	TW	WR-to-PRE recovery
t_rtp_i	in	TW	RD-to-PRE recovery
set_act_i	in	1	load tRCD/tRAS/tRC and open row
set_rd_i	in	1	load tRTP into preblk timer
set_wr_i	in	1	load tWR into preblk timer
set_pre_i	in	1	explicit precharge
set_ap_i	in	1	auto-precharge qualifier on RD/WR
row_i	in	ROW_WIDTH	row to open

18.4 Microarchitecture internals

18.4.1 Five timers

Each timer is an 8-bit saturating down-counter that reloads on its trigger event and counts to zero.

Table 2.13.5: Bank timer countdowns

Timer	Loaded on	Value	Gates
r_rcd	set_act_i	t_rcd_i	safe_rd_o, safe_wr_o
r_ras	set_act_i	t_ras_i	safe_pre_o
r_rc	set_act_i	t_rc_i	safe_act_o
r_rp	set_pre_i or auto-PRE fire	t_rp_i	safe_act_o
r_preblk	set_rd_i (t_rtp_i) / set_wr_i (t_wr_i)	tRTP / tWR	safe_pre_o

18.4.2 Safe equations

```
safe_act_o = !r_row_valid && (r_rp == 0) && (r_rc == 0)
safe_rd_o  =  r_row_valid && (r_rcd == 0) && !r_ap_pending
safe_wr_o  = safe_rd_o
safe_pre_o =  r_row_valid && (r_ras == 0) && (r_preblk == 0) && !
r_ap_pending
```

safe_wr_o is literally assigned safe_rd_o. Read/write recovery is handled by r_preblk, not by the read/write-safe term.

18.4.3 Auto-precharge pending

A read or write with set_ap_i sets r_ap_pending. The auto-precharge fires when the recovery and tRAS windows have both elapsed:

```
w_ap_fire = r_ap_pending && (r_preblk == 0) && (r_ras == 0)
```

On w_ap_fire, the row closes, r_ap_pending clears, and r_rp reloads from t_rp_i — all without a separate scheduler PRE command.

18.4.4 Lookahead twins

With LA = 4 in the scheduler build, the advisory outputs predict readiness LA cycles ahead:

```
safe_act_la_o  = !r_row_valid && (r_rp <= LA) && (r_rc <= LA)
safe_rdwr_la_o =  r_row_valid && (r_rcd <= LA) && !r_ap_pending
```

```
safe_pre_la_o  =  r_row_valid && (r_ras <= LA) && (r_preblk <= LA)
&& !r_ap_pending
```

The lookahead deliberately never predicts `row_valid` or the auto-precharge close. It can only under-report safety; the arbiter re-checks the live `safe_*` outputs at its final stage.

18.4.5 Observability and derived state

`bank_state_e` is derived purely for observability:

```
if (r_row_valid)  state = (r_rcd != 0) ? BANK_ACTIVATING : BANK_ACTIVE
else              state = (r_rp  != 0) ? BANK_PRECHARGING : BANK_IDLE
```

No downstream logic depends on `state_o`. Additional observability flags expose `r_rcd != 0`, `r_preblk != 0`, `r_ras != 0`, and `r_ap_pending`.

18.5 FSM policy

There is no state machine. The block is timers plus a row-open register, with combinational safe outputs.

18.6 Timing

`safe_*` outputs are combinational from the timer flops, so they reflect the effect of a command issued on the previous cycle after one clock edge. The wrapper adds only wiring; there is no extra pipeline stage.

18.7 Notes

- **No FSM means no multi-cycle lag.** The previous design double-registered readiness behind a 3-state FSM, which let refresh and columns slip into stale bank state.
- **Lookahead is advisory.** It is safe to trust a “not safe” lookahead, because the arbiter re-checks the live outputs. A “safe” lookahead can be invalidated by a reload from a command the scheduler issues in the meantime.
- **Rank/bank decoding.** The wrapper uses a decoded select rather than an indexed array access, so an out-of-range rank cannot address past the stamped instances.

19 Global Timers (scoria_global_timers)

Module: scoria_global_timers.sv **Location:** rtl/fub/ **Category:** JEDEC timing **Parent:** scoria_scheduler_plane **Status:** complete, sim-verified, formal-proven

---

19.1 Purpose

`scoria_global_timers` tracks the command-spacing constraints that span banks or ranks: per-rank `tFAW` and `tRRD`, and global `tWTR`, `tRTW`, and `tCCD`. It is pure counters; there is no FSM. The outputs are registered readiness flags consumed by the command arbiter.

19.2 Parameters

Table 2.14.1: Global timers parameters

Parameter	Type	Range	Default	Meaning
NUM_RANKS	int	1+	1	ranks per channel
NUM_BANKS	int	4..16	8	banks per rank
RKW	int	derived	$clog2(NUM_RANKS) or 1	rank index width
BKW	int	derived	$clog2(NUM_BANKS)	bank index width

19.3 Interface

19.3.1 Timing CSRs and events

Table 2.14.2: Global timers inputs

Signal	Direction	Width	Description
t_faw_i	in	8	tFAW window
t_rrd_i	in	8	ACT-to-ACT spacing per rank
t_wtr_global_i	in	8	write-to-read turnaround
t_rtw_i	in	8	read-to-write turnaround
t_ccd_i	in	8	CAS-to-CAS spacing
evt_act_i	in	1	activate event
evt_act_rank_i	in	RKW	rank of the activate event
evt_rd_i	in	1	read event
evt_wr_i	in	1	write event

19.3.2 Readiness and observability

Table 2.14.3: Global timers outputs

Signal	Direction	Width	Description
tfaw_window_ok_o	out	NUM_RANKS	at least one tFAW slot is free
trrd_window_ok_o	out	NUM_RANKS	tRRD satisfied
twtr_global_ok_o	out	1	tWTR satisfied
trtw_window_ok_o	out	1	tRTW satisfied
tccd_window_ok_o	out	1	tCCD satisfied
obs_faw_nz_o	out	NUM_RANKS	tFAW counter non-zero
obs_trrd_nz_o	out	NUM_RANKS	tRRD counter non-

Signal	Direction	Width	Description
			zero
obs_twtr_nz_o	out	1	tWTR counter non-zero
obs_trtw_nz_o	out	1	tRTW counter non-zero
obs_tccd_nz_o	out	1	tCCD counter non-zero

19.4 Microarchitecture internals

19.4.1 tFAW

Each rank maintains a 4-deep sliding window of countdowns. On an ACT, the slot with the smallest remaining count is loaded with `t_faw_i`. `tfaw_window_ok_o[r]` is high when at least one slot in rank `r` is at zero.

19.4.2 tRRD

Each rank has a single countdown that reloads on every ACT to that rank.

19.4.3 tWTR, tRTW, tCCD

These are global single counters. `tWTR` reloads on a write, `tRTW` on a read, and `tCCD` on any column command (`evt_rd_i || evt_wr_i`).

19.4.4 Next-state alignment

The module computes one next-state function for each counter family and feeds both the counter flops and the readiness flops from that same function. This is the fixed-form structure inherited from pumice with the one-cycle-lag defect removed.

The readiness outputs are computed as early comparators rather than as `w_*_nxt == 0`, so the late event signals drive only one-bit mux selects:

```
tfaw_ok_nxt = on_ACT ? (any_free_other || (t_faw_i == 0)) :
any_free_all
trrd_ok_nxt = on_ACT ? (t_rrd_i == 0) : (r_trrd_cnt <= 1)
twtr_ok_nxt = on_WR ? (t_wtr_global_i == 0) : (r_twtr_cnt <= 1)
trtw_ok_nxt = on_RD ? (t_rtw_i == 0) : (r_trtw_cnt <= 1)
tccd_ok_nxt = on_COL ? (t_ccd_i == 0) : (r_tccd_cnt <= 1)
```

19.4.5 Output flops

The readiness flops reset to all-ones, so every window starts ready:

```
tfaw_window_ok_o <= '1;
trrd_window_ok_o <= '1;
twtr_global_ok_o <= 1'b1;
trtw_window_ok_o <= 1'b1;
tccd_window_ok_o <= 1'b1;
```

19.5 FSM policy

No FSM. The block is counters and comparators.

19.6 Timing

The outputs are registered, so a command accepted in cycle `N` closes the relevant window in cycle `N+1`. The next-state derivation guarantees that the readiness flop and the counter flop agree on the same cycle, not one cycle apart.

19.7 Notes

- **The fixed-form warning is mandatory.** pumice's original version published readiness one cycle late: the registered status flop sampled the current counter state instead of the next state, so the flags permitted `tCCD` and `tRTW` violations for one cycle after the command that should have closed them. The fix computes one next-state function and feeds both the counter and its status flop from it. A fresh behavioral reimplementation would reintroduce the defect, because the behavioral description does not mention the flop. Copy the module. This is the warning from HAS Chapter 3.1, restated here as a design note.
- **Consumers no longer compensate.** The arbiter previously added `!(w_fire_out && r_do_rd)` terms and stopped using `tccd_ok_i` to close its own blind spot. With the fixed global timers, those compensations are no longer required; the formal environment assumes only the published outputs.
- **Formal proof.** The block is proven in `formal/mc_common/mc_global_timers/global_timers.sby`.

20 Command Arbiter (scoria_cmd_arbiter)

Module: scoria_cmd_arbiter.sv **Location:** rtl/fub/ **Category:** scheduling / arbitration
Parent: scoria_scheduler_plane **Status:** complete and formal-proven (formal/scoria/cmd_arbiter/scoria_cmd_arbiter.sby; the nofix task runs with BUG001=0 to prove the rank-global fire-stage re-check matters)

20.1 Purpose

The single-issue pick core of the scheduler plane. Each cycle it reads the read and write CAMs as flat per-entry vectors, classifies every entry as column, activate, or precharge, applies the runtime scheduling policy, enforces JEDEC timing through the bank/global timer readiness inputs, and pushes one abstract DRAM command into the scheduler-to-DFI FIFO. `andesite_cmd_arbiter` inherits this mechanism and adds bank-group long/short spacing.

20.2 Parameters

Table 2.15.1: Command arbiter parameters

Parameter	Type	Range	Default	Meaning
NUM_RANKS	int	1+	1	ranks per channel (v1 is single-rank pick)
NUM_BANKS	int	8..16	8	banks per rank
ROW_WIDTH	int	—	14	row address width
COL_WIDTH	int	—	10	column address width
AXI_ID_WIDTH	int	—	8	AXI transaction ID width
NUM_ENTRIES	int	8..32	8	read and write CAM depth
AGE_WIDTH	int	—	16	age-counter width

20.3 Interface

20.3.1 CAM scheduler vectors

Table 2.15.2: CAM scheduler vector ports

Signal	Direction	Width	Description
rd_sch_valid_i	in	NUM_ENTRIES	read CAM per-entry valid
rd_sch_bank_i	in	NUM_ENTRIES*BKW	read candidate bank per entry
rd_sch_row_i	in	NUM_ENTRIES*ROW_WIDTH	read candidate row per entry
rd_sch_col_i	in	NUM_ENTRIES*COL_WIDTH	read candidate column per entry
rd_sch_older_i	in	NUM_ENTRIES^2	read CAM age-order matrix
rd_sch_age_exceed_i	in	NUM_ENTRIES	read entry age-boost flag
rd_sch_qos_i	in	NUM_ENTRIES*4	read entry AxQOS value
rd_sch_head_rel_i	in	AGE_WIDTH	read CAM oldest relative age
wr_sch_valid_i	in	NUM_ENTRIES	write CAM per-entry valid
wr_sch_bank_i	in	NUM_ENTRIES*BKW	write candidate bank per entry
wr_sch_row_i	in	NUM_ENTRIES*ROW_WIDTH	write candidate row per entry
wr_sch_col_i	in	NUM_ENTRIES*COL_WIDTH	write candidate column per entry
wr_sch_older_i	in	NUM_ENTRIES^2	write CAM age-order matrix
wr_sch_age_exceed_i	in	NUM_ENTRIES	write entry age-boost flag

Signal	Direction	Width	Description
wr_sch_qos_i	in	NUM_ENTRIES*4	write entry AxQOS value
wr_sch_head_rel_i	in	AGE_WIDTH	write CAM oldest relative age
wr_commit_ready_i	in	1	write CAM drain-FIFO has room
wr_commit_valid_o	out	1	write CAM entry committed this cycle
wr_commit_slot_o	out	PTRW	committed write CAM slot
rd_issue_ready_i	in	1	read CAM issue-FIFO has room
rd_issue_valid_o	out	1	read CAM entry issued this cycle
rd_issue_slot_o	out	PTRW	issued read CAM slot

20.3.2 Policy CSRs and QoS inputs

Table 2.15.3: Scheduling policy CSR ports

Signal	Direction	Width	Description
sched_order_mode_i	in	2	0/2 FR-FCFS, 1 in_order, 3 age_threshold
sched_row_sel_i	in	2	activate select: 0 oldest, 1 most_pending, 2 fewest_pending
sched_col_sel_i	in	2	column select: 0 oldest, 1 most_pending, 2 fewest_pending
sched_access_pref_i	in	2	0/1 column_first, 2 row_first, 3

Signal	Direction	Width	Description
			precharge_first
sched_wr_high_wm_i	in	8	write-batching drain arm threshold
sched_wr_batch_max_i	in	8	maximum consecutive write columns per drain; 0 = unbounded
sched_wr_low_wm_i	in	8	write-batching drain disarm threshold
sched_prio_sub_i	in	2	0/2 load_over_store, 1 none, 3 age_boost
sched_qos_en_i	in	1	narrow each class to max-AxQOS candidates

20.3.3 Page-policy inputs

Table 2.15.4: Page-policy input ports

Signal	Direction	Width	Description
page_policy_i	in	page_policy_e	legacy OPEN / CLOSE policy
ap_mode_en_i	in	1	runtime page-policy engine override enable
ap_close_i	in	NUM_BANKS	per-bank auto-precharge close mask
timeout_pre_req_i	in	1	idle-expired bank requests background precharge

Signal	Direction	Width	Description
timeout_pre_bank_i	in	BKW	idle-expired bank index

20.3.4 Timer readiness

Table 2.15.5: Timer readiness ports

Signal	Direction	Width	Description
bank_act_ready_i	in	NUM_RANKS x NUM_BANKS	per-bank activate ready
bank_rdwr_ready_i	in	NUM_RANKS x NUM_BANKS	per-bank read/write ready
bank_pre_ready_i	in	NUM_RANKS x NUM_BANKS	per-bank precharge ready
bank_act_ready_la_i	in	NUM_RANKS x NUM_BANKS	advisory lookahead activate ready
bank_rdwr_ready_la_i	in	NUM_RANKS x NUM_BANKS	advisory lookahead read/write ready
bank_pre_ready_la_i	in	NUM_RANKS x NUM_BANKS	advisory lookahead precharge ready
bank_row_active_i	in	NUM_RANKS x NUM_BANKS	per-bank row-active flag
bank_open_row_i	in	NUM_RANKS x NUM_BANKS x ROW_WIDTH	per-bank open row
tfaw_ok_i	in	NUM_RANKS	per-rank tFAW window ok
trrd_ok_i	in	NUM_RANKS	per-rank tRRD spacing ok
twtr_ok_i	in	1	write-to-read turnaround ok
trtw_ok_i	in	1	read-to-write turnaround ok

Signal	Direction	Width	Description
tccd_ok_i	in	1	column-to-column spacing ok
t_ccd_i	in	8	tCCD in controller-clock cycles

20.3.5 Maintenance request/grant

Table 2.15.6: Maintenance request/grant ports

Signal	Direction	Width	Description
refresh_req_i	in	1	refresh controller wants the bus
refresh_drain_i	in	1	refresh drain burst in progress
refresh_kind_i	in	1	0 all-bank REF, 1 per-bank REFpb
refresh_bank_i	in	BKW	REFpb rotor bank mirror
refresh_grant_o	out	1	bus granted to refresh
t_rfc_i	in	16	tRFC / tRFCab recovery
t_rfc_pb_i	in	8	tRFCpb recovery; 0 falls back to t_rfc_i
zq_req_i	in	1	ZQ calibration controller wants the bus
zq_grant_o	out	1	bus granted to ZQ
t_zqcs_i	in	16	tZQCS block window

20.3.6 Init passthrough

Table 2.15.7: Init passthrough ports

Signal	Direction	Width	Description
`init_done_i`	in	1	initialization complete; host commands admitted when high
`init_cmd_valid_i`	in	1	init sequencer has a command
`init_cmd_op_i`	in	`dram_op_e`	init command opcode
`init_cmd_bank_i`	in	`BKW`	init command bank
`init_cmd_row_i`	in	`ROW_WIDTH`	init command row

20.3.7 Event strobes

Table 2.15.8: Timer event-strobe ports

Signal	Direction	Width	Description
`evt_act_o`	out	1	activate fired
`evt_rd_o`	out	1	read column fired
`evt_wr_o`	out	1	write column fired
`evt_pre_o`	out	1	precharge fired
`evt_ap_o`	out	1	auto-precharge bit of fired column
`evt_rank_o`	out	`RKW`	fired command rank
`evt_bank_o`	out	`BKW`	fired command bank
`evt_row_o`	out	`ROW_WIDTH`	fired command

Signal	Direction	Width	Description
			row

20.3.8 Command output

Table 2.15.9: Command output ports

Signal	Direction	Width	Description
cmd_valid_o	out	1	command valid to scheduler-to-DFI FIFO
cmd_ready_i	in	1	FIFO can accept command
cmd_op_o	out	dram_op_e	abstract DRAM opcode
cmd_rank_o	out	RKW	rank
cmd_bank_o	out	BKW	bank
cmd_row_o	out	ROW_WIDTH	row
cmd_col_o	out	COL_WIDTH	column
cmd_ap_o	out	1	auto-precharge flag

20.3.9 Stall attribution

Table 2.15.10: Stall-attribution ports

Signal	Direction	Width	Description
stall_bp_o	out	32	DFI FIFO backpressure
stall_refresh_o	out	32	refresh owns the bus
stall_zq_o	out	32	ZQ request or tZQCS block
stall_noreq_o	out	32	no CAM entry pending

Signal	Direction	Width	Description
stall_turnaround_o	out	32	tWTR / tRTW
stall_tccd_o	out	32	tCCD spacing
stall_actlimit_o	out	32	tFAW / tRRD
stall_banktimer_o	out	32	per-bank tRCD / tRP / tRAS

20.4 Microarchitecture internals

20.4.1 Classification

Every valid read and write CAM entry is classified independently against the registered bank image:

- **Column** — the entry's bank is row-active and its row matches the open row.
- **Activate** — the bank is idle (no row open).
- **Precharge** — the bank is row-active but the entry's row does not match.

The three masks are mutually exclusive per entry. Column eligibility also requires `rd_issue_ready_i` or `wr_commit_ready_i` so a fired column never outruns the CAM retire/drain path.

20.4.2 Priority chain

The per-cycle pick is a fixed priority chain. Maintenance and init outrank all demand traffic; demand class ordering is selected by `sched_access_pref_i`.

```
if !init_done:
    init passthrough
else if w_zq_busy:
    idle (tZQCS block)
else if refresh_req or refresh_drain:
    if REFpb and rotor bank active: precharge rotor bank
    else if REFpb and rotor bank idle and refpb_safe: REFpb + grant
    else if any bank active: precharge lowest active ready bank
    else if ref_safe: REF + grant
else if zq_req:
    if any bank active: precharge lowest active ready bank
    else if zq_safe: ZQCS + grant
else:
    choose demand class by access_pref
    column -> read first unless write-batching / policy override
    activate -> bank-parallel oldest ready
    precharge -> wrong-row oldest ready
    timeout precharge -> lowest priority
```

: Table 2.15.11: Priority chain

20.4.3 Policy CSRs

`sched_order_mode_i` narrows the class masks without changing the branch chain:

- `0/2` — FR-FCFS (default).

- `1` — `in_order`: only the oldest entry in each CAM is eligible; global age order is available under `PUMICE_ENHANCED`.
- `3` — `age_threshold`: when any entry is age-boosted, every class narrows to boosted entries.

`sched_prio_sub_i` selects read/write ordering within a class:

- `0/2` — `load_over_store`: reads first (default).
- `1` — `none`: fair alternation on `r_dir_rr`.
- `3` — `age_boost`: reads first unless the write-class winner is boosted and the read-class winner is not.

`sched_row_sel_i` and `sched_col_sel_i` choose among column/activate candidates by oldest, most pending, or fewest pending, with age as tie-break. When `sched_qos_en_i` is set, each class is first narrowed to the maximum AxQOS value before the population/age select runs.

20.4.4 Write-batching hysteresis

`r_wr_drain` arms when write CAM occupancy reaches `sched_wr_high_wm_i` and disarms only when occupancy falls to `sched_wr_low_wm_i`. While armed, writes outrank reads in every demand class. `sched_wr_batch_max_i` bounds the drain: after that many write columns the drain yields until a read column fires, preventing unbounded read starvation. `sched_wr_high_wm_i == 0` disables batching entirely.

20.4.5 Pick pipeline

The pick pipeline was deliberately deepened by one stage (PUMICE-018) to close double-issue paths opened by the STAGE-1a snapshot register:

- **STAGE-1a** — snapshot the qualified class masks, age-order matrices, population arrays, and per-bank auto-precharge verdict into a coherent epoch.
- **STAGE-1b** — run the per-class `arg_sel` / `arg_oldest` on the snapshot.
- **pre-pick flop** — latch the selected slot and pre-mux the wide `{bank,row,col}` operands off the output-stage critical path.
- **output register** — hold the final command and retire it when the FIFO accepts.

The pipeline advances with `w_out_ready`, so it stalls coherently under FIFO backpressure. Because the snapshot and pre-pick registers create a 3–4 cycle window before the CAM `sch_valid` drop and bank timers reflect a pick, `w_if_preact_sel/pre/out` and `w_if_col_sel/pre/out` guard against the same slot or bank being re-selected while a command is mid-pipeline.

20.4.6 Final safety gate

w_out_safe re-checks the registered pick against live bank timers, rank-global windows, turnaround/tCCD live terms, and !w_zq_busy. Unsafe picks are **dropped**, not held: the CAM entry remains valid and is re-picked later. The command push is cmd_valid_o = r_pick_valid && w_out_safe, which is the BUG-003 fix; previously an unsafe pick could still push into the FIFO because cmd_valid_o was r_pick_valid unconditionally.

20.4.7 Fire-stage bug fixes

BUG-001 adds tfaw_ok_i[RK0] && trrd_ok_i[RK0] to the ACT case of w_out_safe because per-bank timers cannot see rank-global windows; an ACT already in the pipeline when tFAW/tRRD closed would otherwise issue. The STAGE-1b live ACT gate also squashes activate classes while the windows are closed, and the final gate catches any remaining race.

BUG-002 adds !w_zq_busy to w_out_safe. w_zq_busy loads on the accepted fire of a ZQCS, so during the ZQCS fire cycle the pick cone still sees it low and freely selects a follow-on command; the fire-stage re-check prevents that command from issuing inside the calibration window.

20.4.8 Column spacing and turnaround guards

tccd_ok_i reloads on the column fire, 3–4 pick-pipeline cycles after classification. The forward counter r_tccd_fwd reloads on column selection and blocks new column classifications while it is above one or while another column is being selected with t_ccd_i > 1, replacing the stale flopped tccd_ok_i without adding a full tCCD bubble.

The global turnaround ok flags are flopped and drop two cycles after a fired column. r_wrfire0/1 and r_rdfire0/1 block cross-direction column picks for two cycles, but they load on the cycle after fire while the pick is evaluated the cycle before its own command leaves, leaving a one-cycle blind spot. The live terms !(w_fire_out && r_do_wr) and !(w_fire_out && r_do_rd) close that seam.

20.4.9 Auto-precharge guard

Under close-page or per-bank ap_close_i, a fired read/write with auto-precharge closes the bank. r_apguard0/1 block same-bank columns for two cycles, and r_ap_closing is held until bank_row_active drops.

20.4.10 Refresh safety

w_ref_safe requires no possibly-open row, no in-flight precharges, clear guard registers, !w_rfc_busy, !r_grant, and !r_zq_grant. The !r_grant term prevents a double-REF when refresh_req drops one cycle late. w_refpb_safe requires only the rotor bank to be closed; other banks continue serving row hits. While a refresh is pending, columns to banks the refresh will close are masked (w_ref_col_block) so the refresh drain cannot livelock behind an in-flight column guard.

20.4.11 tRFC recovery and tZQCS block

`r_rfc_cnt` loads on a fired REF; while nonzero it blocks further activates and refreshes. `REFpb` uses `t_rfc_pb_i` if non-zero, otherwise `t_rfc_i`. `r_zqcs_cnt` loads on a fired ZQCS and blocks all command issue while nonzero. The two counters are keyed separately (`r_grant` for refresh, `r_zq_grant` for ZQ) so the two maintenance windows do not corrupt each other.

20.4.12 Stall attribution

Stall buckets are mutually exclusive and ordered so their deltas sum to the total stalled-cycle count. Priority order is: `w_out_reject` → `stall_banktimer`; else a picked command waiting on `cmd_ready_i` → `stall_bp`; else `w_zq_busy`/`zq_req` → `stall_zq`; else `refresh_req`/`refresh_drain` → `stall_refresh`; else no CAM entry → `stall_noreq`; else `!twtr_ok`/`!trtw_ok` → `stall_turnaround`; else `!tccd_ok` → `stall_tccd`; else `!tfaw_ok`/`!trrd_ok` → `stall_actlimit`; else `stall_banktimer`.

Table 2.15.12: Stall attribution priority

20.4.13 Maintenance request/grant semantics and event strobes

The arbiter keeps two inherited request/grant pairs — one for refresh and one for ZQ — rather than a single unified `maint_req`/`maint_tag` channel. The unified tagged channel remains the conceptual model; the formatter-side maintenance encoding derives from it downstream. The family doctrine applies: **request and wait, never preempt**. A maintenance source raises its `req` and holds it until `grant` arrives; it cannot yank the bus away from an in-flight host command.

On a safe fire (`w_fire_out`), `evt_act_o`, `evt_rd_o`, `evt_wr_o`, and `evt_pre_o` strobe the bank and global timers; `evt_rank_o`, `evt_bank_o`, and `evt_row_o` carry rank, bank, and row. `evt_ap_o` is `r_ap_out` and does not gate on fire.

20.5 FSM policy

The arbiter is priority logic plus pipeline registers, not an FSM. The “state” is the CAM contents, the timer inputs, and the pipeline registers; there is no explicit state machine governing the pick. The policy is selected by runtime CSRs and recombined every cycle.

20.6 Timing

The arbiter issues at most one grant per cycle. The critical path runs from CAM valid/row inputs through classification, the STAGE-1a snapshot, the STAGE-1b argmax, the pre-pick flop, and the final live safety gate to the FIFO push. The output-stage data muxes were moved into the pre-pick flop to keep the final stage small.

20.7 Notes

- **Dropped, not held:** `w_out_reject` discards an unsafe registered pick rather than holding it. Holding head-of-line blocks the pipeline behind one bank and was measured as a throughput regression; dropping is safe because every CAM commit/issue is qualified by `w_fire_out` and the entry remains schedulable.
- **Refresh grant hazard:** The `!r_grant` term on `w_ref_safe` prevents a second REF being picked while the first is still in the output register. This is especially dangerous for `REFpb`, because every command advances the device's internal bank rotor.
- **AP verdict snapshot:** `r_ap_snap` captures the per-bank auto-precharge verdict with the STAGE-1a masks. Deciding RD vs. RDA at the output stage would let a column see an `ap_close_i[b]` change and issue a row-stay-open command behind one that closed the same row.
- **ISSUE-002:** Under `row_first` (`sched_access_pref_i == 2`), ACT classification is still gated by `w_act_classify_gate`; under `column_first` activate candidates flow through the pipeline while tFAW/tRRD are closed. This preserves `row_first`'s measured throughput floor.
- **Formal path:** The landed SBY file is `formal/scoria/cmd_arbiter/scoria_cmd_arbiter.sby`, not `formal/scoria/scoria_cmd_arbiter.sby`; the `nofix` task sets `BUG001=0` and fails, proving the rank-global fire-stage re-check is load-bearing.

21 Refresh Controller (scoria_refresh_ctrl)

Module: scoria_refresh_ctrl.sv **Location:** rtl/fub/ **Category:** maintenance / refresh
Parent: scoria_scheduler_plane **Status:** complete and sim-verified; formal-retention proof green in formal/scoria/refresh_ctrl

---

21.1 Purpose

`scoria_refresh_ctrl` keeps the DRAM rows alive. It is a credit machine, not an FSM: a `tREFI` interval counter ticks, a pending accumulator tracks how many refreshes are owed, and a request line stays raised until the scheduler grants enough of them back. The block supports DDR3 all-bank refresh, LPDDR3 per-bank refresh with a device-mirror rotor, JEDEC's plus-or-minus-eight postpone/pull-in window, and the TASK-001 Mode A elastic and Mode B temperature-compensated refresh policies.

The core is inherited from pumice, proven there, and re-derived here for scoria's REFpb interval arithmetic and the TASK-001 modes. The RTL is the authority; this page documents the logic as it exists.

21.2 Parameters

Table 2.16.1: Refresh controller parameters

Parameter	Type	Range	Default	Meaning
NUM_BANKS	int	8..16	8	banks per rank; determines the REFpb rotor wrap
BA_W	int	—	$clog2(NUM_BANKS)	bank-address width, drives refresh_bank_o

21.3 Interface

21.3.1 Timing, mode, and configuration

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
enable_i	in	1	refresh engine enable; driven by the init sequencer once initialization completes
t_refi_i	in	16	tREFI reload value, runtime CSR
trefi_pb_i	in	16	per-bank tREFI for REFpb mode; 0 means derive t_refi_i >> 3
refresh_burst_i	in	4	refreshes to issue per drain window, 1..8
refpb_mode_i	in	1	0 = all-bank REFab, 1 = per-bank REFpb (LPDDR3)
refi_reload_i	in	1	immediate counter reload; DV/bring-up pulse only — tie 0 in production

21.3.2 Credit-window and demand-aware policy

Signal	Direction	Width	Description
postpone_limit_i	in	4	postpone bound from CSR; internally

Signal	Direction	Width	Description
			clamped to POSTPONE_MAX
pullin_limit_i	in	4	pull-in bound from CSR; internally clamped to 8
demand_i	in	1	scheduler has read/write work waiting
elastic_en_i	in	1	Mode A elastic refresh enable
pullin_idle_streak_i	in	8	consecutive no-demand cycles before pull-in is allowed
postpone_demand_streak_i	in	7	consecutive demand cycles before the postpone threshold engages
tcr_en_i	in	1	Mode B temperature-compensated refresh enable
trefi_derate_i	in	2	0 = 1x, 1 = 2x, 2 = 4x; encoding 3 clamps to 2

21.3.3 Arbiter handshake and refresh outputs

Signal	Direction	Width	Description
refresh_req_o	out	1	request to the scheduler; held until granted
refresh_grant_i	in	1	scheduler grant pulse

Signal	Direction	Width	Description
grant_was_pb_i	in	1	the granted command on the wire this cycle was a REFpb
refresh_drain_active_o	out	1	high during a burst-drain window; scheduler keeps granting REF back-to-back
refresh_kind_o	out	1	0 = REFab, 1 = REFpb
refresh_bank_o	out	BA_W	per-bank refresh target; the device-mirror rotor output
pending_refreshes_o	out	4	current pending backlog

21.3.4 Observability

Table 2.16.2: Refresh controller ports

Signal	Direction	Width	Description
obs_refi_cnt_o	out	16	live interval countdown
obs_drain_remaining_o	out	4	burst drain counter remaining
obs_bank_rotor_o	out	BA_W	rotor bank pointer
obs_grants_total_o	out	16	total refresh grants since reset
obs_pullin_credit_o	out	4	pull-in credits banked
obs_postpone_events_o	out	16	Mode A telemetry:

Signal	Direction	Width	Description
			refreshes actively postponed
obs_pullin_events_o	out	16	Mode A telemetry: pull-in grants

21.4 Microarchitecture internals

21.4.1 tREFI counter and effective interval

`r_refi_cnt` counts down from the effective interval. On expiry the machine either consumes one pull-in credit or increments the pending accumulator. The effective interval depends on the refresh mode and, in Mode B, the temperature derate:

```
w_refi_eff       = refpb_mode_i ? (trefi_pb_i != 0 ? trefi_pb_i :
t_refi_i >> 3)
                                : t_refi_i
w_derate_shift   = tcr_en_i ? (trefi_derate_i > 2 ? 2 :
trefi_derate_i) : 0
w_refi_eff_derated = w_refi_eff >> w_derate_shift
```

The derate shift is applied to the reload value only. A CSR change mid-interval takes effect on the next reload; the running counter is never rescaled.

21.4.2 Credit accumulator

`r_pending` is a saturating 4-bit counter with a ceiling of `MAX_PENDING = 8`. Each `tREFI` tick adds one unless a pull-in credit is available; each grant subtracts one. When `r_pending` reaches 8, further tREFI ticks are silently dropped — a data-retention hazard — so the request logic is deliberately biased to start asking before the ceiling.

`r_pullin` tracks refreshes performed ahead of their tREFI tick. A grant issued while `r_pending` is 0 and `r_pullin` is below 8 banks a credit instead of retiring a pending refresh.

21.4.3 Postpone and pull-in limits

The JEDEC window permits ±8 refreshes, but the controller clamps the programmed postpone limit to `POSTPONE_MAX = MAX_PENDING - 2 = 6`:

```
w_post_eff = (postpone_limit_i > 6) ? 6 : postpone_limit_i
w_pull_eff = (pullin_limit_i  > 8) ? 8 : pullin_limit_i
```

With the old clamp of 7, the busy-side request `r_pending > w_post_eff` first asserted at `r_pending == 8`, exactly the value at which the accumulator saturates and drops further ticks. Any latency between request and grant — finishing a burst, precharging banks, tRFC — then ate real refreshes. The deliberate 7→6 reduction gives one full `tREFI` of headroom before the JEDEC 8-postponed ceiling. The pull-in limit is clamped to 8, matching the JEDEC window.

21.4.4 Idle confirmation and the three-case request

Demand is CAM occupancy, which can blink off for a few cycles between bursts. Treating those micro-gaps as idle would release postponed refreshes mid-stream and trigger pull-ins, so idle is confirmed only after a sustained no-demand streak. Mode A makes the threshold sweepable; when disabled, the inherited 16-cycle confirmation is used.

The request equation has three cases:

```
if (w_idle)
    req = (r_pending > 0) || (r_pullin < w_pull_eff)
else if (elastic_en_i && r_demand_streak < postpone_demand_streak_i)
    req = (r_pending > 0)           // sporadic demand: strict
else
    req = (r_pending > w_post_eff)  // sustained demand: postpone
threshold
```

When `elastic_en_i` is low, the equation reduces bit-for-bit to the v3 behavior. The demand streak is a saturating 7-bit counter reset on any no-demand cycle.

21.4.5 Drain burst

When the previous burst is fully drained and `r_pending` is non-zero, `r_burst_remaining` loads `min(refresh_burst_i, r_pending)`. The drain window stays open while `r_burst_remaining > 0`, `r_pending > 0`, and `refresh_req_o` is asserted. Each grant decrements the remaining count. The drain is gated on the registered request so a postponed backlog does not accidentally open a preemption window.

21.4.6 REFpb rotor

`r_bank_rotor` mirrors the device's internal per-bank refresh counter. It advances only when `grant_was_pb_i` is true, and it holds through REFab mode changes. Keying the advance off the granted command rather than `refpb_mode_i` keeps the mirror synchronized across mode boundaries; the device's counter persists, and clearing the controller's copy would desynchronize it.

21.4.7 Why a credit machine and not an FSM

A refresh controller could be drawn as a state machine with states for "counting", "requesting", "draining", and "recovering". This block deliberately is not. The interval counter, pending accumulator, pull-in bank, drain quota, and rotor are all independent counters whose next-state functions are evaluated together each cycle. There is no central state to sequence them because their interaction is purely arithmetic: the request line is a combinational function of the counters, and the grant pulse is just another term in the counter updates. That structure matches the inherited pumice block, keeps the formal-property surface small, and makes Mode A/B additions pure data-path changes.

21.5 FSM policy

There is no FSM. The block is a credit/state-counter machine: one counter for the interval, one saturating accumulator for pending refreshes, one counter for pull-in credits, and a handful of comparators that decide when to raise `refresh_req_o`. Mode selects are data, not control — they change the values loaded into counters and the bank address driven to the scheduler, but they do not add states.

The only sequential state beyond the counters is the registered output stage, which copies the combinational request and counter values into the output flops each cycle.

21.6 Timing

The tREFI counter reloads with w_refi_eff_derated on every expiry and on an asserted refi_reload_i. Recovery timing after a refresh is owned by the arbiter: it loads t_rfc_i for REFab and t_rfc_pb_i for REFpb into its own recovery counter. Refresh is request-and-wait; it never preempts host traffic.

The request-to-grant latency is unbounded by design. The credit window absorbs scheduler back-pressure: the pending counter keeps ticking upward while the request is held, and the pull-in credit bank absorbs the opposite case where the bus is free before the next tREFI tick.

21.7 Notes

- **Formal proof:** The retention and accounting properties are green in `formal/scoria/refresh_ctrl`. The proof must be re-derived when the interval arithmetic changes — REFpb mode, Mode B derate, or any future ceiling change. Carrying the all-bank proof forward unchanged would prove the wrong thing while appearing green.
- **`refi_reload_i` is a DV/bring-up knob:** it immediately reloads the counter with the current CSR value, saving the one stale interval that otherwise occurs after a `t_refi_i` change. Tie it to 0 in production; when tied low it has no effect, so the default build is bit-identical to a design without it.
- **Refresh is an ordinary bank occupant:** bank timers own the banks during refresh. A refresh request waits its turn in the scheduler like any other command, and the drain window only biases scheduling within the maintenance class.
- **Postpone clamp trap:** A CSR value above 6 still clamps to 6. Firmware cannot program the controller all the way to the JEDEC ceiling; the headroom is intentional.
- **REFpb mode-change trap:** The rotor must advance only on `grant_was_pb_i`, not on `refpb_mode_i`. A grant decided as `REFab` but counted as per-bank, or vice versa, desynchronizes the mirror and causes the controller to precharge the wrong bank ahead of each device refresh.
- **Telemetry semantics:** `obs_postpone_events_o` counts interval ticks that add to `r_pending` while the elastic postpone branch is actively withholding the request; `obs_pullin_events_o` counts grants issued when `r_pending` is zero. Both saturate at 0xFFFF rather than wrapping.

22 ZQ Controller (scoria_zq_ctrl)

Module: scoria_zq_ctrl.sv **Location:** rtl/fub/ **Category:** maintenance / ZQ calibration
Parent: scoria_scheduler_plane **Status:** complete and sim-verified; DDR3-only at the current design point because the scheduler gates w_zq_run on memtype_i == MEMTYPE_DDR3

22.1 Purpose

`scoria_zq_ctrl` issues periodic ZQ short-calibration commands (`ZQCS`) as maintenance traffic. It is new in scoria: pumice's DDR2/LPDDR2 target has no ZQ calibration, and LiteDRAM's working DDR3 core implements its ZQCS executor inside the refresh path. scoria splits it out so the arbiter sees one maintenance request/grant channel per source, with a source tag rather than a special case.

The block is intentionally small. Its job is to wait the programmed interval, ask for the bus, hold the post-grant `tZQCS` window, and start the next interval. It does not block traffic during `tZQCS`; that obligation lives in the arbiter.

22.2 Parameters

`scoria_zq_ctrl` has no parameters; all configuration is runtime CSR.

Table 2.17.1: ZQ controller parameters (none — all ports)

22.3 Interface

22.3.1 Configuration and timing

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
enable_i	in	1	ZQCS enable; also gated on init_done upstream
t_zqcs_interval_i	in	32	cycles between calibrations; 0 disables
t_zqcs_i	in	16	post-grant tZQCS hold count

22.3.2 Mode C placement policy

Signal	Direction	Width	Description
placement_i	in	2	0 = request on expiry, 1 = defer under demand, 2 = reserved
overdue_max_i	in	13	maximum deferral cycles; 0 = no cap
demand_i	in	1	scheduler has read/write work waiting

22.3.3 Arbiter handshake and telemetry

Table 2.17.2: ZQ controller ports

Signal	Direction	Width	Description
zq_req_o	out	1	request to the scheduler; held until granted

Signal	Direction	Width	Description
zq_grant_i	in	1	scheduler grant pulse
obs_busy_o	out	1	high in the post-grant tZQCS window
obs_zqcs_total_o	out	16	total ZQCS commands issued since reset
obs_interval_cnt_o	out	32	live interval countdown
obs_overdue_o	out	1	interval expired and still no grant, or intentionally deferred

22.4 Microarchitecture internals

22.4.1 Interval counter

`r_interval` is a 32-bit down-counter loaded from `t_zqcs_interval_i`. A 32-bit width is required: a ~128 ms interval at 75 MHz is ~9.6M cycles, which does not fit the 16-bit counter `tREFI` uses. If `t_zqcs_interval_i == 0`, the block treats the interval as disabled rather than as "as fast as possible".

At reset the counter is seeded from the input rather than zeroed. A zero reset would make the first interval zero-length and issue a calibration immediately out of reset, along with a spurious `obs_overdue_o` if demand is already up. The upstream `enable_i` is normally low out of reset and additionally gated on `init_done`, but the seed removes a latent coupling on those reset values.

22.4.2 The four FSM states

`scoria_zq_ctrl` has a real FSM, unlike `refresh_ctrl`, because its behavior is genuinely sequential: it must wait, then ask, then hold, and it needs a separate deferred-wait state for the placement policy.

```
ZQ_IDLE  — counting down to the next calibration
ZQ_REQ   — requesting the arbiter, waiting for grant
ZQ_HOLD  — granted; holding tZQCS before reloading the interval
ZQ_DEFER — Mode C: interval expired but demand is high, deferring
```

22.4.3 Placement policy

`placement_i == 0` requests on expiry; this is bit-identical to the v1 behavior. `placement_i == 1` enters `ZQ_DEFER` if `demand_i` is high when the interval expires. `placement_i == 2` is reserved and should not be used.

`ZQ_DEFER` exits to `ZQ_REQ` when any of these occurs:

- `demand_i` goes low,
- `placement_i` is changed away from 1,
- `overdue_max_i != 0` and `r_defer_cnt >= overdue_max_i`.

`obs_overdue_o` is high while deferred, and also while in `ZQ_REQ` with demand still present. The telemetry exists to make starvation visible: a request that withdrew itself under load would starve silently.

22.4.4 Post-grant hold semantics

On `zq_grant_i`, the FSM moves to `ZQ_HOLD` and loads `r_hold` with `t_zqcs_i`. When `r_hold` reaches zero, the FSM returns to `ZQ_IDLE` and reloads `r_interval` from `t_zqcs_interval_i`. The hold simply delays the next interval reload; it does not itself block traffic. The arbiter enforces

the actual `tZQCS` command-spacing window through its `w_zq_busy` term and final safety gate, which is where BUG-002 was fixed.

22.5 FSM policy

The FSM is the right tool here because ZQCS has a non-arithmetic lifecycle: count, request, hold, repeat. The `ZQ_DEFER` state adds a conditional wait that cannot be expressed as a counter threshold alone. The state register is 2 bits; the combinational outputs are derived directly from it.

22.6 Timing

One grant per calibration. The interval reload happens at the end of ZQ_HOLD, so the effective spacing between calibrations is `t_zqcs_interval_i + t_zqcs_i` cycles measured from grant to the start of the next request. `t_zqcs_interval_i` is counted in controller-clock cycles; the CSR value is derived from the JEDEC specification and the desired calibration frequency.

22.7 Notes

- **ZQCS never preempts:** The block raises `zq_req_o` and waits. This matches LiteDRAM's behavior and is consistent with the family doctrine that maintenance is request-and-wait traffic.
- **DDR3-only today:** `scoria_scheduler_plane` computes `w_zq_run = zq_enable_i && init_done && (memtype_i == MEMTYPE_DDR3)`. For LPDDR3, ZQ calibration is handled differently and this block is not exercised.
- **Traffic blocking is the arbiter's job:** Do not rely on `ZQ_HOLD` to space commands. The arbiter's `w_zq_busy` term and final safety gate prevent any new command from issuing inside the `tZQCS` window; that is where BUG-002 was caught and fixed.
- **Placement policy trap:** `placement_i == 2` is reserved. If programmed, the block falls out of the defined policy and behavior is unspecified. Firmware should keep it at 0 or 1.
- **Overdue semantics trap:** `obs_overdue_o` is set intentionally in `ZQ_DEFER` and incidentally in `ZQ_REQ` while demand is present. A high value is not by itself an error; it is a starvation indicator that should be bounded by `overdue_max_i`.

23 Init Sequencer (scoria_init_sequencer)

Module: scoria_init_sequencer.sv **Location:** rtl/fub/ **Category:** control / initialization
Parent: scoria_scheduler_plane **Status:** landed and sim-verified

23.1 Purpose

scoria_init_sequencer owns the DRAM power-up handshake. It drives RESET#, raises CKE, programs the mode-register set in JEDEC order, issues ZQCL, waits out tDLLK and tZQinit, and then releases the scheduler to normal traffic. The block is NEW relative to pumice because DDR3 adds the RESET# pin and a shorter init sequence than DDR2; the LPDDR3 chain is inherited from pumice's LPDDR2 path, reusing the same {MA[5:0], OP[7:0]} MRW packing.

The sequencer issues real commands through the scheduler's init passthrough while init_busy_o is high. Each command state occupies one cycle, then the FSM parks in S_WAIT for the JEDEC inter-command delay. The mode-register shadow is updated in lockstep via mr_seq_we_o so live CL/CWL/BL decode tracks what was programmed.

23.2 Parameters

Table 2.18.1: Init sequencer build parameters

Parameter	Type	Range	Default	Meaning
ROW_WIDTH	int	—	14	width of init_cmd_row_o; carries MR data for MRS and {MA, OP} for LPDDR3 MRW
NUM_BANKS	int	—	8	number of DRAM banks; determines BKW
BKW	int	—	$clog2(NUM_BANKS)	bank/MR-index width on init_cmd_bank_o

All timing values are runtime CSRs, not parameters. That is the family rule: a timing compiled in cannot be swept, and a controller that cannot be swept cannot be characterized.

23.3 Interface

23.3.1 Clocks, reset, configuration, and DFI status

Table 2.18.2: Clocks, reset, configuration, and DFI status

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
memtype_i	in	memtype_e	MEMTYPE_DDR3 or MEMTYPE_LPDDR3
dram_reset_n_o	out	1	RESET# to top/PHY; latched released at S_D3_CKE
dfi_init_start_o	out	1	asserted from S_RESET until S_DONE
dfi_init_complete_i	in	1	PHY reports init complete; gates exit from S_DFI_INIT

23.3.2 JEDEC init-sequence waits (CSR-backed)

Table 2.18.3: JEDEC timing CSR inputs

Signal	Direction	Width	Description
t_init_wait_i	in	16	CKE / tINIT settle wait
t_dll_wait_i	in	16	tDLLK DLL lock wait
t_xpr_wait_i	in	16	tXPR = max(tXS, 5 tCK) after CKE high
t_zqinit_wait_i	in	16	tZQinit after ZQCL
t_mrd_wait_i	in	8	tMRD between MRS

Signal	Direction	Width	Description
			commands
t_rp_wait_i	in	8	tRP after precharge (unused on DDR3 init)
t_rfc_wait_i	in	8	tRFC after refresh (unused on DDR3 init)

23.3.3 Mode-register values, restart, and shadow port

Table 2.18.4: Mode-register inputs, restart, and shadow write port

Signal	Direction	Width	Description
mr0_i	in	16	MR0 base value (BL/CL/tWR)
mr1_i	in	16	MR1 value (ODT, ODS, DLL enable)
mr2_i	in	16	MR2 value
mr3_i	in	16	MR3 value
init_restart_i	in	1	rising edge re-runs the JEDEC chain from S_RESET
mr_seq_we_o	out	1	pulse: capture mr_seq_data_o into shadow MR[mr_seq_index_o]
mr_seq_index_o	out	5	MR index (0..3)
mr_seq_data_o	out	16	MR value to shadow

23.3.4 Command request and status

Table 2.18.5: Command request, legacy handshake, and status outputs

Signal	Direction	Width	Description
`init_cmd_valid_o`	out	1	single-cycle pulse: issue this init command
`init_cmd_op_o`	out	`dram_op_e`	command opcode (`OP_MRS`, `OP_ZQCL`, etc.)
`init_cmd_bank_o`	out	`BKW`	bank address; for MRS this is the MR index
`init_cmd_row_o`	out	`ROW_WIDTH`	MR data for MRS; `{MA[5:0], OP[7:0]}` for LPDDR3 MRW
`zqcl_req_o`	out	1	tied low; `ZQCL` is issued through the command stream
`zqcl_grant_i`	in	1	unused; tied off at source
`init_busy_o`	out	1	high while init is in progress (tied off in scheduler plane)
`init_done_o`	out	1	high when FSM reaches `S_DONE`

The `init_cmd_valid_o` pulse is accepted by the scheduler's init passthrough without a grant handshake. During init the scheduler stays in `S_IDLE` and `dfi_cmd_formatter` is always `cmd_ready`, so a single-cycle pulse issues exactly one command.

23.4 Microarchitecture internals

23.4.1 FSM state list

```
S_RESET
S_DFI_INIT
    DDR3 branch:
        S_D3_RSTN, S_D3_CKE, S_D3_XPR
        S_D3_MR2, S_D3_MR3, S_D3_MR1, S_D3_MR0
        S_D3_ZQCL, S_D3_LOCK
    LPDDR3 branch:
        S_L_RESET, S_L_ZQ, S_L_MR1, S_L_MR2, S_L_MR3
S_WAIT
S_DONE
```

23.4.2 DDR3 initialization sequence

```
S_RESET -> S_DFI_INIT -> S_D3_RSTN -> S_D3_CKE -> S_D3_XPR
S_D3_XPR -> S_D3_MR2 -> S_D3_MR3 -> S_D3_MR1 -> S_D3_MR0
S_D3_MR0 -> S_D3_ZQCL -> S_D3_LOCK -> S_DONE
```

The MR order is JESD79-3F's, not sorted. MR1 carries DLL enable and MR0 carries DLL reset, so the order encodes a dependency.

23.4.3 LPDDR3 initialization sequence

```
S_RESET -> S_DFI_INIT -> S_L_RESET -> S_L_ZQ
S_L_ZQ -> S_L_MR1 -> S_L_MR2 -> S_L_MR3 -> S_DONE
```

LPDDR3 MRW uses `init_cmd_row_o` packed as `{MA[5:0], OP[7:0]}` because the 3-bit bank path cannot reach MR10 or MR63. MR63 and MR10 are issued to the DRAM but not shadowed; only MR1, MR2, and MR3 update the shadow.

23.4.4 MR0 DLL reset handling

The MR0 command issued to the DRAM ORs in `DDR3_DLL_RESET = 16'h0100` (MR0 A8). The bit is self-clearing per JESD79-3F 3.4.2.4, so no second MR0 load is needed. The shadow is written with the unmodified `mr0_i` so the live CL/CWL/BL decode tracks the steady-state value rather than the reset pulse.

23.4.5 `dram_reset_n_o` latch

`dram_reset_n_o` is asserted from reset until the FSM reaches `S_D3_CKE`, then released and never re-asserted without a controller reset or an `init_restart_i` pulse. The latch avoids the one-cycle/low-window hazards that come from decoding the pin directly off the FSM state set. For LPDDR3, which has no RESET# pin, the output reads high throughout.

23.5 FSM policy

The block has **one FSM** and it is the init FSM. There are no nested state machines. Command states occupy exactly one cycle each; the inter-command waits live in `S_WAIT`, which decrements a CSR-loaded counter until zero and then resumes `r_next`. The `r_next` register captures the state to resume before entering `S_WAIT`.

23.6 Timing

Every wait state counts from a CSR-loaded value captured on entry, so a CSR written mid-wait does not affect the running count. Values are runtime CSRs initialized from the JESD79-3F or JESD209-2F speed bin at CSR-derivation time.

Table 2.18.6: Init-sequence timing windows

Wait	Symbol	Loaded from	Counted in
RESET# low to CKE high	`tINIT`	`t_init_wait_i`	MC clocks
CKE high to first MRS	`tXPR`	`t_xpr_wait_i`	MC clocks
Between MRS commands	`tMRD`	`t_mrd_wait_i`	MC clocks
`ZQCL` to ready	`tDLLK`, `tZQinit`	`t_dll_wait_i`, `t_zqinit_wait_i`	MC clocks, the larger of the two
LPDDR3 reset settle	`tINIT`	`t_init_wait_i`	MC clocks
LPDDR3 ZQ init	`tZQinit`	`t_dll_wait_i`	MC clocks

23.7 Notes

- **`zqcl_req_o` and `init_busy_o` are intentionally tied off** in the scheduler plane. `ZQCL` is issued through the ordinary command stream, and the scheduler derives init occupancy from the command stream rather than from `init_busy_o`.
- **Re-initialization:** A rising edge on `init_restart_i` forces the FSM back to `S_RESET` and replays the MRS chain with the current CSR MR values. `init_done_o` falls immediately so the scheduler stops admitting host commands.
- **Ordering checks** — for example, that MR2 precedes MR0 — are assertions in DV, not in RTL. The FSM encodes the order, and DV proves the FSM never strays.
- **`t_rp_wait_i` and `t_rfc_wait_i`** are accepted for compatibility with the broader timing CSR set but are unused during DDR3 or LPDDR3 initialization.

24 Mode Register (scoria_mode_register)

Module: scoria_mode_register.sv **Location:** rtl/fub/ **Category:** register / decode **Parent:** scoria_scheduler_plane **Status:** landed and sim-verified

24.1 Purpose

`scoria_mode_register` is the per-rank mode-register shadow plus the live decode of MR-derived timing values. The init sequencer writes the shadow during DRAM bring-up; a later CSR/APB hot-update path can write it without re-running init. Downstream consumers read the decoded values rather than polling raw MR bits, so the decode equations are the single place where JEDEC bit fields become controller-internal numbers.

The block is INHERITED from pumice with DDR3-specific additions: shadow sizing widened to cover LPDDR3 MR0..MR16, `wr_o` decoded from MR0[11:9], and `wrlvl_en_o` gated from MR1[7] for DDR3 only.

24.2 Parameters

Table 2.19.1: Mode-register build parameters

Parameter	Type	Range	Default	Meaning
NUM_RANKS	int	1+	1	number of ranks; one MR shadow bank per rank
MAX_MR_IDX	int	—	17	shadow index range 0..16; covers DDR3 MR0..MR3 and LPDDR3 MR0..MR16
RKW	int	—	(NUM_RANKS > 1) ? $clog2(NUM_RANKS) : 1	rank-index width

24.3 Interface

24.3.1 Clocks, reset, configuration, and write port

Table 2.19.2: Clocks, configuration, and shadow write port

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
memtype_i	in	memtype_e	MEMTYPE_DDR3 or MEMTYPE_LPDDR3
mr_we_i	in	1	shadow write enable
mr_index_i	in	5	MR index (0..MAX_MR_ID X-1)
mr_data_i	in	16	MR value to write
mr_rank_i	in	RKW	rank index

24.3.2 Scheduler request channel (v1 unused)

Table 2.19.3: Scheduler MR-update request channel

Signal	Direction	Width	Description
mr_req_o	out	1	tied low; no hot MR updates issued through the scheduler in v1
mr_grant_i	in	1	unused
mr_req_index_o	out	5	tied to zero
mr_req_data_o	out	16	tied to zero
mr_req_rank_o	out	RKW	tied to zero

24.3.3 Live decoded outputs

Table 2.19.4: Live decoded outputs

Signal	Direction	Width	Description
cl_o	out	4	CAS read latency
cwl_o	out	4	CAS write latency
bl_o	out	4	burst length
al_o	out	4	additive latency (tied off in scheduler plane for scoria v1)
drv_strength_o	out	2	output drive strength (tied off in scheduler plane)
odt_o	out	2	ODT rule (tied off in scheduler plane)
wr_o	out	5	write recovery, DDR3 MR0[11:9]
wrlvl_en_o	out	1	DRAM is in write-leveling mode (DDR3 MR1[7])

Live decode is taken from rank 0. Multi-rank designs are expected to use matching MR values across ranks.

24.4 Microarchitecture internals

24.4.1 Shadow array

The shadow is `r_mr_shadow[NUM_RANKS][MAX_MR_IDX]`, a 16-bit array. On `mr_we_i`, if `mr_index_i < MAX_MR_IDX`, `mr_data_i` is written into `r_mr_shadow[mr_rank_i][mr_index_i]`. The init sequencer drives the write port during the MRS chain.

24.4.2 Live decode equations

Decode is combinational off `r_mr_shadow[0][0]` (MR0), `r_mr_shadow[0][1]` (MR1), and `r_mr_shadow[0][2]` (MR2), then registered to the outputs.

Table 2.19.5: Live decode equations

Output	DDR3 equation	LPDDR3 equation
`bl_o`	MR0[1:0]: `00/01 -> 8`, `10 -> 4`	MR1[2:0]: `010 -> 4`, `011/100 -> 8`
`cl_o`	`MR0[6:4] + 4 + (MR0[2] ? 8 : 0)`	MR2[3:0] enum: `0001->3`, `0010->4`, `0011->5`, `0100->6`, `0101->7`, `0110->8`
`cwl_o`	`MR2[5:3] + 5`	write latency from the same MR2[3:0] enum
`al_o`	MR1[4:3]: `00->0`, `01->CL-1`, `10->CL-2`, `11->0`	`0`
`wr_o`	MR0[11:9] table: `000->16`, `001->5`, `010->6`, `011->7`, `100->8`, `101->10`, `110->12`, `111->14`	`0`
`wrlvl_en_o`	`(memtype == DDR3) && MR1[7]`	`0`

A reserved DDR3 MR0 encoding for `cl_o` returns 4, which is not a legal DDR3 CL and is intentionally unusable. LPDDR3 BL16 clips to BL8 because `bl_o` is 4-bit.

24.4.3 JESD79-3F MR0 field-map note

JESD79-3F section 3.4.2.2 states in prose that “CAS Latency is defined by MR0 (bits A9-A11)”, which contradicts the spec’s own Figure 9, where A11:A9 holds write recovery. The figure is self-consistent: the values tabulated against A11:A9 are 16/5/6/7/8/10/12, which are write-recovery cycle counts and cannot be CAS latencies. The RTL comment at `scoria_mode_register.sv:126-133` resolves the inconsistency deliberately in favor of the figure: CL at MR0[6:4], WR at MR0[11:9].

24.5 FSM policy

There is no FSM. The block is a shadow RAM plus combinational decode. Writes are single-cycle; all outputs are registered.

24.6 Timing

All decoded outputs are registered on `mc_clk`. A shadow write in cycle N is visible on `cl_o`/`cwl_o`/`bl_o`/etc. in cycle N+1.

24.7 Notes

- **`al_o`, `drv_strength_o`, `odt_o`, and `mr_req_o` are tied off in the scheduler plane** in scoria v1. The ports exist for family compatibility and future multi-rank or ODT-policy work.
- **LPDDR3 BL16 clips to BL8** because `bl_o` is 4-bit. Widening `bl_o` and updating downstream consumers is recorded as a v3 task.
- **Reserved MR0 encodings** return obviously wrong values rather than plausible-looking defaults, so a mis-programmed MR is visible immediately.

25 Write Leveling Interface (`scoria_wrlvl_ifc`)

Module: `scoria_wrlvl_ifc.sv` **Location:** `rtl/fub/` **Category:** training control / PHY handshake **Parent:** `scoria_scheduler_plane` **Status:** landed and sim-verified

25.1 Purpose

`scoria_wrlvl_ifc` is the controller-side write-leveling interface. It does **not** contain a delay-line search; that is decision D2. The block sequences the DFI v3.1 per-CS leveling handshake, enforces the JESD79-3F timing windows, captures the DRAM's prime-DQ answer, and exposes telemetry so firmware can distinguish four outcomes: never attempted, converged, timed out, and swept with no flip found.

The reasoning behind the no-search rule is the family's architectural boundary. A tap sweep is PHY calibration — step size, tap count, monotonicity, and temperature behavior are PHY properties, not DDR3 properties. Putting the search inside the controller would make a calibration bug a silicon respin instead of a script edit. The block owes the system the handshake, the windows, the capture, and the telemetry; firmware owns the algorithm.

25.2 Parameters

Table 2.20.1: Write-leveling interface parameters

Parameter	Type	Range	Default	Meaning
NUM_CS	int	1+	1	number of independent chip selects
CSW	int	—	(NUM_CS > 1) ? $clog2(NUM_CS) : 1	chip-select index width (cs_sel_i)

25.3 Interface

25.3.1 Mode and host controls

Table 2.20.2: Mode and host controls

Signal	Direction	Width	Description
`wrlvl_en_i`	in	1	DRAM is in write-leveling mode (from `scoria_mode_register.wrlvl_en_o`)
`strobe_i`	in	1	host pulse: emit one DQS edge
`cs_sel_i`	in	`CSW`	chip select under training

25.3.2 Timing windows (runtime CSRs)

Table 2.20.3: Write-leveling timing CSR inputs

Signal	Direction	Width	Description
`t_wldqsen_i`	in	16	`tWLDQSEN`: wait before DQS may be driven
`t_wlmrd_i`	in	16	`tWLMRD`: wait before the first DQS pulse
`t_wlmrd_max_i`	in	16	controller-defined `tWLMRD` timeout; `0` = no timeout
`t_wlo_i`	in	16	`tWLO`: result-return delay after DQS edge
`t_wloe_i`	in	16	`tWLOE`: inert in this implementation; samples prime DQ

Signal	Direction	Width	Description
			only

25.3.3 DFI v3.1 per-CS leveling handshake

Table 2.20.4: DFI v3.1 write-leveling handshake

Signal	Direction	Width	Description
`dfi_phylvl_req_cs_n_o`	out	`NUM_CS`	controller requests per-CS write leveling, active low
`dfi_phylvl_ack_cs_n_i`	in	`NUM_CS`	PHY acknowledges per CS, active low
`dfi_phy_wrlvl_cs_n_o`	out	`NUM_CS`	selects write leveling per CS, active low
`dfi_wrlvl_strobe_o`	out	1	DQS strobe forwarded by the PHY for the prime-DQ sample

25.3.4 DRAM answer and telemetry

Table 2.20.5: DRAM answer and host telemetry

Signal	Direction	Width	Description
`prime_dq_i`	in	1	sampled prime DQ returned by the PHY
`result_valid_o`	out	1	a sample completed
`result_o`	out	1	captured prime-DQ value
`obs_attempts_o`	out	16	number of strobes emitted
`obs_flips_o`	out	16	result-bit transitions

Signal	Direction	Width	Description
			observed across the sweep
obs_timeout_o	out	1	t_wlmrd_max_i expired
obs_ever_done_o	out	1	a pass has completed at least once
obs_state_o	out	3	FSM-state observability

The telemetry discipline is the four-state story: reset is never-attempted, obs_ever_done_o marks converged, obs_timeout_o marks timed out, and a completed sweep with obs_flips_o == 0 marks no result in window. The counters survive wrlvl_en_i going low so firmware can read the record of the pass that just finished.

25.4 Microarchitecture internals

25.4.1 FSM state list

```
WL_OFF
WL_DQSEN
WL_MRD
WL_READY
WL_WAIT_WLO
WL_TIMEOUT
```

25.4.2 State behavior

```
WL_OFF      -> WL_DQSEN      when wrlvl_en_i rises
WL_DQSEN    -> WL_MRD        after tWLDQSEN
WL_MRD      -> WL_READY      after tWLMRD
WL_READY    -> WL_WAIT_WLO   on strobe_i
WL_READY    -> WL_TIMEOUT    if t_wlmrd_max_i expires with no strobe
WL_WAIT_WLO -> WL_READY      after tWLO, result captured
WL_TIMEOUT  -> WL_TIMEOUT    sticky until wrlvl_en_i clears
```

`dfi_wrlvl_strobe_o` is generated only in `WL_READY` and only when `strobe_i` is high. A strobe arriving early is dropped, not deferred; deferring would report an attempt the DRAM never saw.

25.4.3 `t_wlmrd_max_i` timeout behavior

The timeout is armed when `t_wlmrd_max_i != 0`. It measures elapsed time since entering write-leveling mode, but it restarts on every accepted strobe. Without the restart, a host sweeping delay taps would blow through any sensible timeout part-way through a working pass and get `WL_TIMEOUT` spuriously.

25.4.4 DFI handshake outputs

`dfi_phylvl_req_cs_n_o` and `dfi_phy_wrlvl_cs_n_o` are driven active-low on the selected chip select for as long as `wrlvl_en_i` is high, and driven high (inactive) otherwise. The one-hot decode guards against `cs_sel_i` values past `NUM_CS`.

25.5 FSM policy

The block has one minimal window-sequencer FSM. There are no search states, no delay-walk states, and no tap-storage registers. The host walks the delay line by asserting `strobe_i` again with a new delay setting.

25.6 Timing

Table 2.20.6: Write-leveling timing windows

Window	Symbol	Source	Meaning
tWLDQSEN	t_wldqsen_i	JESD79-3F	wait after entering leveling mode before driving DQS
tWLMRD	t_wlmrd_i	JESD79-3F	wait before the first DQS pulse
tWLMRD max	t_wlmrd_max_i	controller-defined	timeout with distinct status
tWLO	t_wlo_i	JESD79-3F speed bin	result-return delay after DQS edge
tWLOE	t_wloe_i	JESD79-3F speed bin	inert: bounds mismatch across DQ bits, but only the prime bit is sampled

All values are runtime CSRs, initialized from the JESD79-3F speed bin at CSR-derivation time.

25.7 Notes

- **No on-chip tap search.** This is decision D2 and is the defining boundary of the block. A future system that needs autonomous leveling should add an external calibration engine driving this interface, not a search loop inside `scoria_wrlvl_ifc`.
- **`tWLOE` is inert.** The implementation samples the prime DQ bit only, so `tWLOE` exists in the CSR map for compatibility but does not gate the result capture. That choice is explicit and documented, not an accident.
- **The timeout status is distinct from "no flip found."** `obs_timeout_o` means the host stopped strobing before `t_wlmrd_max_i` elapsed. A sweep that completed but saw zero transitions is `obs_ever_done_o` high, `obs_timeout_o` low, and `obs_flips_o` zero.
- **Cleared only by reset.** `obs_attempts_o`, `obs_flips_o`, `obs_timeout_o`, and `obs_ever_done_o` are not cleared when `wrlvl_en_i` falls; they are cleared only by controller reset. That preserves the record of the pass for firmware inspection.

26 DFI Command Path (scoria_dfi_cmd_path)

Module: scoria_dfi_cmd_path.sv **Location:** rtl/fub/ **Category:** DFI datapath **Parent:** scoria_dfi_plane **Status:** implemented (TASK-007)

26.1 Purpose

`scoria_dfi_cmd_path` pops the abstract command stream from the CDC command FIFO, unpacks `{ap, col, row, bank, rank, op}`, and drives the multi-phase DFI command bus through `scoria_dfi_cmd_formatter`. It also emits `wr_fire_o` and `rd_fire_o` strobes so the write serializer and read aligner can align their data phases to the accepted command.

The most important thing this block does **not** do is pace commands. All JEDEC timing lives in the scheduler — the bank timers, global timers, and command arbiter enforce `tRCD`, `tRP`, `tRAS`, `tRFC`, `tCCD`, `tRTW`, and `tWTR` upstream of the CDC. A previous revision tried to re-enforce some of those windows inside this module; TASK-007 traced the resulting FIFO compression to board failures, and the only remaining gates here are structural (read-aligner slot availability and write-data staging). The command path is a constant-latency conduit, not a scheduler.

26.2 Parameters

Table 2.21.1: `scoria_dfi_cmd_path` parameters

Parameter	Type	Range	Default	Meaning
`NUM_RANKS`	int	1..4	1	rank count
`NUM_BANKS`	int	power of 2	8	bank count
`ROW_WIDTH`	int	—	14	row address width
`COL_WIDTH`	int	—	10	column address width
`BURST_LEN_WIDTH`	int	—	8	burst-length field width (unused by formatter)
`DFI_RATE`	int	power of 2	4	DFI frequency ratio
`CMD_HISTORY_EN`	int	0/1	0	enable DFI-wire command-history scoreboard
`HIST_T_RCD`	int	—	0	scoreboard `tRCD`
`HIST_T_RP`	int	—	0	scoreboard `tRP`
`HIST_T_RAS`	int	—	0	scoreboard `tRAS`
`HIST_T_RFC`	int	—	0	scoreboard `tRFC`
`HIST_T_WTR`	int	—	0	scoreboard `tWTR`
`HIST_T_RTW`	int	—	0	scoreboard `tRTW`
`N_SUBCMD`	int	1..	1	compile-time max packed sub-

Parameter	Type	Range	Default	Meaning
				commands
SUB_COL_STRIDE	int	—	1	compile-time max column stride
SUB_PHASE_STRIDE	int	—	1	compile-time max DFI-phase stride
DFI_ADDR_WIDTH	int	—	14	per-phase DFI address width
DFI_BANK_WIDTH	int	—	3	per-phase DFI bank width
DFI_CTRL_WIDTH	int	—	1	per-phase control width
DFI_CS_WIDTH	int	—	NUM_RANKS	per-phase chip-select width

26.3 Interface

26.3.1 Controller-side command in

Table 2.21.2: Command input group

Signal	Direction	Width	Description
dfi_clk	in	1	DFI / PHY clock
dfi_rstn	in	1	active-low synchronous reset
memtype_i	in	memtype_e	DDR3 or LPDDR3 selection
rd_phase_i	in	PHW	DFI phase carrying read commands
wr_phase_i	in	PHW	DFI phase carrying write commands
n_subcmd_i	in	SUBW_MAX	active sub-command count (runtime)
sub_col_stride_i	in	COL_WIDTH	active column stride (runtime)
sub_phase_stride_i	in	PHW	active phase stride (runtime)
cmd_valid_i	in	1	CDC command FIFO valid
cmd_ready_o	out	1	CDC command FIFO ready
cmd_data_i	in	CMD_DW	packed {ap, col, row, bank, rank, op}

26.3.2 DFI command bus

Table 2.21.3: DFI command output group

Signal	Direction	Width	Description
dfi_address_o	out	DFI_ADDR_BUS_W	multi-phase DFI address bus
dfi_bank_o	out	DFI_BANK_BUS_W	multi-phase DFI bank bus
dfi_cas_n_o	out	DFI_CTRL_BUS_W	multi-phase CAS#
dfi_ras_n_o	out	DFI_CTRL_BUS_W	multi-phase RAS#
dfi_we_n_o	out	DFI_CTRL_BUS_W	multi-phase WE#
dfi_cs_n_o	out	DFI_CS_BUS_W	multi-phase chip-select
dfi_odt_o	out	DFI_CS_BUS_W	multi-phase ODT

26.3.3 Fire strobes

Table 2.21.4: Data-path strobe group

Signal	Direction	Width	Description
wr_fire_o	out	1	WR/WRA accepted this cycle
rd_fire_o	out	1	RD/RDA accepted this cycle
fire_rank_o	out	RKW	rank of the fired command
rd_op_ready_i	in	1	read aligner has a free slot
wr_op_ready_i	in	1	write burst is staged in the CDC
wr_accept_o	out	1	pops the write-staged token

26.4 Microarchitecture internals

26.4.1 No command pacing

The accept gate `w_gate` is purely structural:

```
w_gate = (!read || rd_op_ready_i) && (!write || wr_op_ready_i)
```

`rd_op_ready_i` comes from the read aligner's outstanding-read slot count; `wr_op_ready_i` comes from the CDC write-staged token FIFO. Both are sized so they never deassert in steady state. If either does deassert, the in-order command stream stalls and compresses spacing for every command behind it — exactly the failure mode TASK-007 measured. The `r_wr_held_*` counters (simulation-only) make any such hold observable.

All real timing is enforced upstream:

Table 2.21.5: Timing enforcement remains upstream of the command path

Timing	Upstream owner
tCCD	`scoria_cmd_arbiter` `r_tccd_fwd`, clamped to burst length in `scoria_core`
tRTW / tWTR	`scoria_global_timers` `r_trtw_cnt` / `r_twtr_cnt`
tRFC	`scoria_cmd_arbiter` `r_rfc_cnt`
tRCD / tRP / tRAS / tRRD / tFAW	`scoria_bank_timer` and rank gates

26.4.2 Sub-DFI-word burst packing

When `N_SUBCMD > 1`, a single scheduled column command packs multiple JEDEC bursts into one DFI word. The canonical case is gear 1:4 with x16 BL4: each BL4 occupies only two of the four DFI phases, so two BL4 reads share one 128-bit DFI word. The path issues `n_subcmd_i` sub-column-commands in a single DFI cycle on phases `{base_phase, base_phase + sub_phase_stride_i, ...}` with columns `{col, col + sub_col_stride_i, ...}`. One formatter instance per sub-command places its decoded command on its own phase; the per-sub buses merge with OR for address/bank/ODT and AND for chip-select and the three command strobes, because inactive phases carry NOP (`cs_n`/`ras_n`/`cas_n`/`we_n` all ones, address/bank/ODT zero).

The whole packed group produces exactly one FIFO pop, one `wr_fire_o` or `rd_fire_o`, and one `wr_accept_o`. The data path therefore drives or captures the single DFI word once, even though `N_SUBCMD` DRAM commands were issued.

```
compile-time max:  N_SUBCMD, SUB_COL_STRIDE, SUB_PHASE_STRIDE
runtime active:    n_subcmd_i, sub_col_stride_i, sub_phase_stride_i
legacy path:       N_SUBCMD == 1, sub 0 only on the base phase
```

26.4.3 DFI-wire command-history scoreboard

When `CMD_HISTORY_EN != 0`, the block instantiates `scoria_cmd_history_checker` on the accepted command stream (`w_fire`). This checker sees the same spacing the DRAM sees, downstream of the CDC FIFO, so it catches compression that the scheduler-side checker cannot observe. The feature is simulation-only and off by default.

26.5 FSM policy

There is no FSM. The command path is combinational unpack, generate, and merge, followed by registered outputs and the registered fire strobes.

26.6 Timing

- Command accept-to-DFI-bus latency is one `dfi_clk` cycle (the formatter's registered output).
- `wr_fire_o` / `rd_fire_o` are registered one cycle to align with the formatter outputs.
- Sub-DFI-word packing adds no extra latency; all sub-commands issue in the same cycle.

26.7 Notes

- The elaboration assertions at `scoria_dfi_cmd_path.sv:403-413` require `DFI_RATE` to be a power of two and, when `N_SUBCMD > 1`, `N_SUBCMD * SUB_PHASE_STRIDE == DFI_RATE`. These mirror the DV BFM's anchored-slot-mask contract so the a7ddrphy de-interleaver packs every sub-burst into the same DFI word with no stale phases.
- The `BURST_LEN_WIDTH` parameter is carried for interface compatibility but `cmd_len_i` is tied off inside the formatter for this controller generation.
- Dormant operations `OP_SREFE`, `OP_SREFX`, and `OP_DPDE` fall through the formatter default arm and are driven as NOP on the DFI command pins; their real behavior is CKE sequencing handled elsewhere. See `ch02_blocks/26_dormant_powerdown_and_pack.md`.

27 DFI Command Formatter (scoria_dfi_cmd_formatter)

Module: scoria_dfi_cmd_formatter.sv **Location:** rtl/fub/ **Category:** DFI command encoding **Parent:** scoria_dfi_plane **Status:** implemented

27.1 Purpose

`scoria_dfi_cmd_formatter` is the last combinational step before the abstract command stream becomes DFI wires. It translates the scheduler's `dram_op_e` opcode plus `{rank, bank, row, col, ap}` into the pin patterns the DRAM expects: DDR3 uses the JESD79-3F `{ras_n, cas_n, we_n}` truth table with address and bank fields; LPDDR3 uses the bit-exact JESD209-2F Table 60 CA-bus encoding packed as `{w_ca_f, w_ca_r}`. The block does not sequence anything — it only encodes. Sequence lives in the scheduler and init sequencer, the same split pumice used.

27.2 Parameters

Table 2.22.1: `scoria_dfi_cmd_formatter` parameters

Parameter	Type	Range	Default	Meaning
`NUM_RANKS`	int	1..4	1	rank count
`NUM_BANKS`	int	power of 2	8	bank count
`ROW_WIDTH`	int	—	14	row address width
`COL_WIDTH`	int	—	10	column address width
`BURST_LEN_WIDTH`	int	—	8	burst-length field width (unused)
`DFI_RATE`	int	power of 2	2	DFI frequency ratio
`DFI_ADDR_WIDTH`	int	—	14	per-phase DFI address width
`DFI_BANK_WIDTH`	int	—	3	per-phase DFI bank width
`DFI_CTRL_WIDTH`	int	—	1	per-phase control width
`DFI_CS_WIDTH`	int	—	`NUM_RANKS`	per-phase chip-select width

27.3 Interface

Table 2.22.2: `scoria_dfi_cmd_formatter` ports

Signal	Direction	Width	Description
`mc_clk`	in	1	controller / DFI clock
`mc_rst_n`	in	1	active-low synchronous reset
`memtype_i`	in	`memtype_e`	DDR3 or LPDDR3 selection
`cmd_valid_i`	in	1	command valid
`cmd_ready_o`	out	1	always 1 after reset
`cmd_op_i`	in	`dram_op_e`	opcode
`cmd_rank_i`	in	`RKW`	target rank
`cmd_bank_i`	in	`BKW`	bank address
`cmd_row_i`	in	`ROW_WIDTH`	row address / MR payload
`cmd_col_i`	in	`COL_WIDTH`	column address
`cmd_len_i`	in	`BURST_LEN_WIDTH`	unused — tied off
`rd_phase_i`	in	`PHW`	DFI phase for read commands
`wr_phase_i`	in	`PHW`	DFI phase for write commands
`dfi_address_o`	out	`DFI_ADDR_BUS_W`	multi-phase address bus
`dfi_bank_o`	out	`DFI_BANK_BUS_`	multi-phase

Signal	Direction	Width	Description
		W	bank bus
dfi_cas_n_o	out	DFI_CTRL_BUS_W	multi-phase CAS#
dfi_ras_n_o	out	DFI_CTRL_BUS_W	multi-phase RAS#
dfi_we_n_o	out	DFI_CTRL_BUS_W	multi-phase WE#
dfi_cs_n_o	out	DFI_CS_BUS_W	multi-phase chip-select
dfi_odt_o	out	DFI_CS_BUS_W	multi-phase ODT

27.4 Microarchitecture internals

27.4.1 DDR3 command truth table

The DDR3 encoding follows JESD79-3F. `A10` selects auto-precharge inside read and write commands, precharge-all inside precharge, and long/short inside ZQ calibration. The table below is the citation anchor for the block.

```
CS_n=0:
  RAS_n  CAS_n  WE_n    Command / notes
    1      1      1     NOP
    0      1      1     ACT  (bank=BA, address=row)
    1      0      1     RD   (bank=BA, A10=0, address=col)
    1      0      1     RDA  (bank=BA, A10=1, address=col)
    1      0      0     WR   (bank=BA, A10=0, address=col)
    1      0      0     WRA  (bank=BA, A10=1, address=col)
    0      1      0     PRE  (bank=BA, A10=0)
    0      1      0     PREA (A10=1, bank don't-care)
    0      0      1     REF  (all-bank refresh)
    0      0      0     MRS  (bank=MR index, address=MR data)
    1      1      0     ZQCS (A10=0)
    1      1      0     ZQCL (A10=1)

CS_n=1:  NOP / deselect
```

: Table 2.22.3: DDR3 command truth table (JESD79-3F)

For `OP_MRS`, the MR data is carried on `cmd_row_i` rather than `cmd_col_i` because DDR3 mode-register payloads can need bit 10, which a 10-bit column field would truncate.

27.4.2 Marking against pumice

Most of the DDR3 table is inherited unchanged from pumice's DDR2 formatter. The new encodings relative to pumice are:

Table 2.22.4: MODIFIED command encodings versus pumice

Opcode	Status	Reason
`OP_ZQCL`	MODIFIED	DDR3 adds ZQ calibration long; pumice defined the opcode but drove NOP
`OP_ZQCS`	MODIFIED	DDR3 adds ZQ calibration short; pumice defined the opcode but

Opcode	Status	Reason
		drove NOP
OP_PREA	MODIFIED	DDR3 precharge-all uses the same RAS/CAS/WE as precharge with A10=1

27.4.3 LPDDR3 CA-bus encoding

For memtype_i == MEMTYPE_LPDDR3, the formatter builds a 20-bit CA word bit-exact to JESD209-2F Table 60:

```
w_lpddr2_ca[9:0]   = CA[9:0]  on the rising edge
w_lpddr2_ca[19:10] = CA[9:0]  on the falling edge
packed form:        {w_ca_f, w_ca_r}
```

The word is driven on dfi_address_o[19:0]; ras_n, cas_n, and we_n stay idle, and chip-select is asserted on phase 0. The CA encoding covers ACT, RD/RDA, WR/WRA, PRE/PREA, REF, REFPB, and MRW. MRW carries the MR index in cmd_row_i[13:8] and MR data in cmd_row_i[7:0] because the LPDDR3 MR range exceeds the 3-bit bank port.

Key CA field mappings:

```
ACT:    CA0r=0, CA1r=1 ; bank -> CA7r..CA9r ; row[12:8] -> CA2r..CA6r
        row[7:0] -> CA0f..CA7f ; row[13] -> CA8f ; row[14] -> CA9f
RD/RDA: CA0r=1, CA1r=0, CA2r=1 ; bank -> CA7r..CA9r ; col[2:1] ->
CA5r,CA6r
        AP -> CA0f ; col[11:3] -> CA1f..CA9f
WR/WRA: CA0r=1, CA1r=0, CA2r=0 ; bank -> CA7r..CA9r ; col[2:1] ->
CA5r,CA6r
        AP -> CA0f ; col[11:3] -> CA1f..CA9f
PRE:    CA0r=1, CA1r=1, CA2r=0, CA3r=1 ; bank -> CA7r..CA9r
PREA:   CA0r=1, CA1r=1, CA2r=0, CA3r=1, CA4r=1
REF:    CA0r=0, CA1r=0, CA2r=1, CA3r=1
REFPB:  CA0r=0, CA1r=0, CA2r=1, CA3r=0
MRW:    MA[5:0] -> CA4r..CA9r ; MA[7:6] -> CA0f,CA1f ; OP[7:0] ->
CA2f..CA9f
NOP:    CA0r..CA3r = 1
```

27.4.4 ODT

dfi_odt_o is declared on the interface but is driven to zero for all commands in this implementation. The ODT pin is part of the DFI command group per the HAS, but scoria's PHY target does not require dynamic ODT control at the formatter; any ODT behavior is handled elsewhere or left inactive.

27.4.5 Phase placement

Reads are placed on `rd_phase_i`, writes on `wr_phase_i`, and all other commands on phase 0. This matches the a7ddrphy rdphase/wrphase contract. Phases that do not carry the active command are driven as NOP.

27.5 FSM policy

There is no FSM. The formatter is combinational encoding followed by a single registered pipeline stage. `cmd_ready_o` is tied high after reset.

27.6 Timing

The formatter introduces one `mc_clk` cycle of latency from decoded inputs to DFI outputs. All command-to-command spacing is enforced upstream by the scheduler.

27.7 Notes

- `OP_SREFE`, `OP_SREFX`, and `OP_DPDE` are not encoded here. They fall through the default arm and are driven as NOP on the DFI command pins; the real behavior is CKE/power-down sequencing, which is described in `ch02_blocks/26_dormant_powerdown_and_pack.md`.
- `OP_REFPB` is LPDDR3-only and uses the same CA encoding branch as the per-bank refresh case.
- The module comment header still names DDR2/LPDDR2 because the formatter body was inherited from pumice and the file was not renamed; the actual `memtype_e` values and encoding targets are DDR3 and LPDDR3.

28 DFI Write Serializer (scoria_dfi_wr_serializer)

Module: scoria_dfi_wr_serializer.sv **Location:** rtl/fub/ **Category:** DFI write datapath
Parent: scoria_dfi_plane **Status:** implemented

28.1 Purpose

`scoria_dfi_wr_serializer` turns the write-data stream into the DFI write-data pins. On a WR command (`wr_fire_i`), it waits `t_phy_wrlat_i` DFI cycles and then streams one DFI word per cycle from the write-data FIFO onto `dfi_wrdata_o` until the burst's `last` word. The data is pre-staged in the CDC FIFO, so the drive path never stalls once it starts.

28.2 Parameters

Table 2.23.1: `scoria_dfi_wr_serializer` parameters

Parameter	Type	Range	Default	Meaning
`DFI_DATA_WIDTH`	int	—	128	DFI data bus width (`DRAM_BEAT_WIDTH * DFI_RATE`)
`DFI_RATE`	int	power of 2	2	DFI frequency ratio
`DFI_STRB_WIDTH`	int	—	`DFI_DATA_WIDTH/8`	byte-strobe width
`DFI_EN_WIDTH`	int	—	`DFI_RATE`	write-enable width
`WRLAT_W`	int	—	8	`t_phy_wrlat` width

28.3 Interface

Table 2.23.2: `scoria_dfi_wr_serializer` ports

Signal	Direction	Width	Description
`dfi_clk`	in	1	DFI / PHY clock
`dfi_rstn`	in	1	active-low synchronous reset
`t_phy_wrlat_i`	in	`WRLAT_W`	WR command to `dfi_wrdata_en` delay
`wr_fire_i`	in	1	WR command accepted strobe
`wd_valid_i`	in	1	write-data FIFO valid
`wd_ready_o`	out	1	write-data FIFO ready
`wd_data_i`	in	`DFI_DATA_WIDTH`	write data
`wd_strb_i`	in	`DFI_STRB_WIDTH`	byte strobe
`wd_last_i`	in	1	last word of burst
`dfi_wrdata_o`	out	`DFI_DATA_WIDTH`	DFI write data
`dfi_wrdata_en_o`	out	`DFI_EN_WIDTH`	DFI write data enable
`dfi_wrdata_mask_o`	out	`DFI_STRB_WIDTH`	DFI write data mask

28.4 Microarchitecture internals

28.4.1 Stateless multi-burst serializer

The serializer uses a delay line rather than an FSM. `wr_fire_i` shifts into `r_age[PIPE-1:0]`, where `PIPE = MAX_WRLAT + 1 = 32`. A fire matures when it reaches index `t_phy_wrlat_i`; `r_owed` counts matured-but-not-finished bursts. The path drives whenever `r_owed != 0` and the FIFO has a word.

```
PIPE       = MAX_WRLAT + 1 = 32
w_fired    = {r_age, wr_fire_i}
w_mature   = w_fired[t_phy_wrlat_i]
w_owed_now = r_owed + (w_mature ? 1 : 0)
w_drive    = (w_owed_now != 0) && wd_valid_i
wd_ready_o = w_drive
```

This handles every write cadence by construction: back-to-back bursts mature contiguously, `tCCD`-paced writes mature with the correct bubble, and multiple in-flight writes add to `r_owed` independently.

28.4.2 Owed-burst accounting

`r_owed` tracks bursts that have matured but have not yet finished. It increments when `w_mature` is true and decrements when a driven word is the last word of its burst:

```
w_burst_last = w_drive && wd_last_i
r_owed_next  = w_owed_now - (w_burst_last ? 1 : 0)
```

The counter width `OWEDW = 3` is small because the controller rate-matches write commits to the data path; only a few write bursts can be in flight at once. If `r_owed` saturates, the behavior is undefined, but the upstream staging invariant makes saturation unreachable.

28.4.3 Mask generation

DFI mask is active-high “mask this byte”, while the upstream strobe is active-high “write this byte”. The conversion is a simple inversion:

```
dfi_wrdata_mask_o = ~wd_strb_i
```

When `w_drive` is low, both `dfi_wrdata_en_o` and `dfi_wrdata_mask_o` are driven to zero.

28.5 FSM policy

There is no FSM. The serializer is a shift-register delay line plus an owed-burst counter.

28.6 Timing

- Latency from `wr_fire_i` to the first `dfi_wrdata_en_o` assertion is `t_phy_wrlat_i` DFI cycles.
- Each burst then runs one DFI word per cycle with no bubbles until `wd_last_i`.
- The `dfi_wrdata_o` bus is wired straight from `wd_data_i` because the upstream FIFO already holds the full DFI word.

28.7 Notes

- The prior implementation used an FSM that assumed seamless continuation and dropped `tCCD` bubbles, driving write data early (`scoria_dfi_wr_serializer.sv:62-64`). The current delay-line design removes that assumption.
- `MAX_WRLAT = 31` covers the DDR3 write-latency range. A build with a larger programmed `t_phy_wrlat` will see `w_mature` clamped to zero and the write data never driven.
- The write-data FIFO is assumed never empty while `r_owed != 0`; the CDC staged-token invariant and rate-matched commit upstream guarantee this in normal operation.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

29 DFI Read Aligner (scoria_dfi_rd_aligner)

Module: scoria_dfi_rd_aligner.sv **Location:** rtl/fub/ **Category:** DFI read datapath
Parent: scoria_dfi_plane **Status:** implemented

29.1 Purpose

`scoria_dfi_rd_aligner` admits read commands from the command path, drives `dfi_rddata_en_o` after `t_rddata_en_i` DFI cycles, captures returning read data, and pushes one DFI word per cycle into the read CDC FIFO. It supports multiple outstanding reads and backpressures the command path when its tracking slots are full.

29.2 Parameters

Table 2.24.1: `scoria_dfi_rd_aligner` parameters

Parameter	Type	Range	Default	Meaning
`DFI_DATA_WIDTH`	int	—	128	DFI data bus width
`DFI_RATE`	int	power of 2	2	DFI frequency ratio
`DFI_EN_WIDTH`	int	—	`DFI_RATE`	`dfi_rddata_en` width
`DFI_VALID_WIDTH`	int	—	`DFI_RATE`	`dfi_rddata_valid` width
`BL_WORDS`	int	—	4	DFI words captured per read
`EN_CYC`	int	`>= BL_WORDS`	`BL_WORDS`	`rddata_en` window width in DFI cycles
`MAX_OUTSTANDING`	int	—	8	outstanding-read tracking depth
`RDEN_W`	int	—	8	`t_rddata_en` width

29.3 Interface

Table 2.24.2: `scoria_dfi_rd_aligner` ports

Signal	Direction	Width	Description
`dfi_clk`	in	1	DFI / PHY clock
`dfi_rstn`	in	1	active-low synchronous reset
`t_rddata_en_i`	in	`RDEN_W`	RD command to `dfi_rddata_en` delay
`op_valid_i`	in	1	read-op admission valid
`op_ready_o`	out	1	read-op admission ready
`dfi_rddata_en_o`	out	`DFI_EN_WIDTH`	DFI read enable
`dfi_rddata_i`	in	`DFI_DATA_WIDTH`	DFI read data
`dfi_rddata_valid_i`	in	`DFI_VALID_WIDTH`	DFI read data valid
`rd_valid_o`	out	1	captured data valid to CDC FIFO
`rd_ready_i`	in	1	CDC FIFO ready
`rd_data_o`	out	`DFI_DATA_WIDTH`	captured read data
`rd_resp_o`	out	2	response code (`RESP_OKAY = 2'b00`)
`rd_last_o`	out	1	last word of burst

29.4 Microarchitecture internals

29.4.1 Admission and backpressure

A read is admitted on `op_valid_i && op_ready_o`. The ready signal deasserts when `r_outstanding == MAX_OUTSTANDING`:

```
op_ready_o = (r_outstanding < MAX_OUTSTANDING)
```

`r_outstanding` increments on admission and decrements when the last word of a read is captured. `MAX_OUTSTANDING` is sized at least as large as the read CAM depth so the ready signal does not deassert in steady state.

29.4.2 Stateless rddata_en delay line

The enable window is generated by a shift register `r_age[PIPE-1:0]`, where `PIPE = MAX_RDDATA_EN + EN_CYC`. An admission shifts in on each cycle; the enable output is the OR of `EN_CYC` consecutive taps starting at `t_rddata_en_i`:

```
w_en = OR over b in [0, EN_CYC-1] of w_fired[t_rddata_en_i + b]
dfi_rddata_en_o = w_en ? all-ones : 0
```

This handles contiguous or `tCCD`-spaced reads without an FSM: overlapping windows from multiple outstanding reads simply OR together.

29.4.3 EN_CYC versus BL_WORDS

`EN_CYC` and `BL_WORDS` are separate on purpose:

Table 2.24.3: EN_CYC versus BL_WORDS

Quantity	Meaning
`EN_CYC`	True DQ-bus occupancy: `ceil(DRAM_BL / DFI_RATE)` DFI cycles
`BL_WORDS`	Number of DFI words captured and pushed per read

`scoria_core` passes `RD_EN_CYC_CORE = ceil(DRAM_BL / DFI_RATE)` as `EN_CYC`. The old default of `BL_WORDS` would have halved the enable window for narrow devices and lost half of every burst. The build must satisfy `EN_CYC >= BL_WORDS`.

29.4.4 Enable-window credit

The PHY asserts a preamble `dfi_rddata_valid` one cycle before the real data window. A simple outstanding-read gate would capture that preamble and shift the whole burst. The aligner uses an

enable-window credit: one credit is minted for every enable cycle and spent for every captured word.

```
w_credit_avail = r_credit + (w_en ? 1 : 0)
w_word_valid   = |dfi_rddata_valid_i && (w_credit_avail != 0)
w_cap_fire     = w_word_valid && (r_outstanding != 0) && rd_ready_i
```

The credit **must be combinational**. A registered credit reads zero on the first enable cycle when data and enable arrive together (the a7_read_gated model), dropping the first word. That is why the credit includes the current cycle's enable contribution.

29.4.5 Capture and last-word marking

Captured data is pushed to the CDC FIFO with `rd_resp_o = RESP_OKAY`. `rd_last_o` is asserted when `r_rcnt == BL_WORDS - 1`. `r_rcnt` resets on the last word so the next read's capture starts at word 0.

The capture gate has three conditions:

```
1. at least one bit of dfi_rddata_valid_i is set
2. w_credit_avail is non-zero (inside the real data window)
3. at least one read is outstanding
4. rd_ready_i is high (downstream FIFO can accept)
```

`rd_valid_o` is asserted whenever conditions 1-3 hold; `rd_data_o` and `rd_resp_o` are driven combinationally. Only the actual push (condition 4) decrements credit and advances `r_rcnt`.

29.4.6 Debug probes

Under `!SYNTHESIS`, the block maintains counters for returned and captured beats, plus the two blocked-beat counters. A nonzero `dbg_blocked_real` means a real data beat was discarded, which shifts every later beat by one and corrupts the burst stream. The first-block snapshot records how many cycles the enable had run and how many words had already been captured, making it easy to distinguish a preamble from a credit-accounting bug.

29.5 FSM policy

There is no FSM. The aligner is a delay line, a credit counter, and an outstanding-read counter.

29.6 Timing

- `dfi_rddata_en_o` asserts `t_rddata_en_i` DFI cycles after admission.
- The enable window lasts `EN_CYC` DFI cycles.
- Captured words are pushed one per cycle starting in the first data-valid cycle inside the window.

29.7 Notes

- The non-synthesis assertion `assert (!(rd_valid_o && !rd_ready_i))` fires if a read beat is lost because the return FIFO is full. The return-path sizing contract makes this unreachable; the assertion turns any future sizing break into a hard failure.
- Debug probes `dbg_blocked_pre` and `dbg_blocked_real` count valid beats rejected before any read is outstanding (preamble/stray) and while a read is outstanding (dropped real data). `RD_ALIGN_TRACE` enables cycle-by-cycle `$display` logging under `!SYNTHESIS`.
- The module comment header and some internal names still say “LPDDR2” because the file was inherited from pumice; the operational targets are DDR3 and LPDDR3.

30 DFI Clock-Domain Crossing (`scoria_dfi_cdc`)

Module: `scoria_dfi_cdc.sv` **Location:** `rtl/fub/` **Category:** clock-domain crossing **Parent:** `scoria_dfi_plane` **Status:** implemented and formally proven (`formal/scoria/scoria_dfi_cdc.sby`)

30.1 Purpose

`scoria_dfi_cdc` is the single boundary between the controller clock domain (`ctl_clk`) and the PHY DFI clock domain (`dfi_clk`). All traffic between the two domains crosses through `gaxi_fifo_async` Gray-pointer FIFOs; there are no hand-rolled synchronizers and no open-loop bit crossings. The block is formally proven.

30.2 Parameters

Table 2.25.1: `scoria_dfi_cdc` parameters

Parameter	Type	Range	Default	Meaning
`CMD_DW`	int	—	32	command payload width
`WD_DW`	int	—	72	write-data payload width
`RD_DW`	int	—	66	read-data payload width
`CMD_DEPTH`	int	power of 2	8	command FIFO depth
`WD_DEPTH`	int	power of 2	16	write-data FIFO depth
`RD_DEPTH`	int	power of 2	16	read-data FIFO depth
`TOK_DEPTH`	int	power of 2	4	init-token FIFO depth
`N_FLOP_CROSS`	int	—	2	pointer synchronizer stages
`USE_JOHNSON`	int	0/1	0	0 = Gray pointers, 1 = Johnson pointers

30.3 Interface

30.3.1 Controller domain

Table 2.25.2: Controller-domain ports

Signal	Direction	Width	Description
`ctl_clk`	in	1	controller clock
`ctl_rstn`	in	1	active-low synchronous reset
`cmd_valid_i`	in	1	command FIFO push valid
`cmd_ready_o`	out	1	command FIFO push ready
`cmd_data_i`	in	`CMD_DW`	packed command payload
`wd_valid_i`	in	1	write-data FIFO push valid
`wd_ready_o`	out	1	write-data FIFO push ready
`wd_data_i`	in	`WD_DW`	packed write-data payload
`wd_last_i`	in	1	last word of write burst
`init_start_i`	in	1	init-start level
`rd_valid_o`	out	1	read-data FIFO pop valid
`rd_ready_i`	in	1	read-data FIFO pop ready
`rd_data_o`	out	`RD_DW`	packed read-

Signal	Direction	Width	Description
			data payload
init_complete_o	out	1	init-complete sticky latch

30.3.2 PHY domain

Table 2.25.3: PHY-domain ports

Signal	Direction	Width	Description
dfi_clk	in	1	DFI / PHY clock
dfi_rstn	in	1	active-low synchronous reset
pcmd_valid_o	out	1	command FIFO pop valid
pcmd_ready_i	in	1	command FIFO pop ready
pcmd_data_o	out	CMD_DW	packed command payload
pwd_valid_o	out	1	write-data FIFO pop valid
pwd_ready_i	in	1	write-data FIFO pop ready
pwd_data_o	out	WD_DW	packed write-data payload
pwr_staged_valid_o	out	1	write-burst-staged token valid
pwr_staged_pop_i	in	1	write-burst-staged token pop
pinit_start_o	out	1	init-start sticky latch

Signal	Direction	Width	Description
prd_valid_i	in	1	read-data FIFO push valid
prd_ready_o	out	1	read-data FIFO push ready
prd_data_i	in	RD_DW	packed read-data payload
pinit_complete_i	in	1	init-complete level

30.4 Microarchitecture internals

30.4.1 FIFO instances

Table 2.25.4: Clock-domain-crossing FIFOs

Instance	Direction	Payload	Depth	Purpose
u_cmd_fifo	ctl to phy	CMD_DW	CMD_DEPTH	commands
u_wd_fifo	ctl to phy	WD_DW	WD_DEPTH	write data
u_wtok_fifo	ctl to phy	1 bit	WD_DEPTH	write-burst-staged tokens
u_rd_fifo	phy to ctl	RD_DW	RD_DEPTH	read data
u_istart_tok	ctl to phy	1 bit	TOK_DEPTH	init-start rising-edge token
u_icmp_tok	phy to ctl	1 bit	TOK_DEPTH	init-complete rising-edge token

All six FIFOs use gaxi_fifo_async with N_FLOP_CROSS = 2 synchronizer stages.

30.4.2 Payload packing

```
CMD_DW = 32 : { op[3:0], rank, bank, row, col, ap }
WD_DW  = 72 : { data[63:0], strb[7:0], last }
RD_DW  = 66 : { data[63:0], resp[1:0], last }
```

The command payload is opaque to the CDC; unpacking happens in scoria_dfi_cmd_path. The write-data and read-data payloads include their respective last bits so the serializer and return ring can detect burst boundaries.

30.4.3 Pointer encoding

Gray pointers are the default (USE_JOHNSON = 0), which requires power-of-two depths. Johnson encoding is opt-in (USE_JOHNSON = 1) and supports arbitrary depths, but it costs DEPTH-bit pointers in both clock domains and every synchronizer stage.

30.4.4 Init tokens

init_start_i and pinit_complete_i are level signals, but initialization is monotonic: once asserted it stays asserted. The CDC converts each rising edge to a one-bit token, crosses it through a token FIFO, and sets a sticky latch on the far side. Re-runs are future work.

30.4.5 The staged-token invariant

Write data and write-staged tokens are accepted together on the controller side:

```
wd_ready_o  = w_wd_data_ready && w_wtok_ready
w_wtok_push = wd_valid_i && wd_last_i && w_wd_data_ready
```

A token is pushed only on the burst's last word, and only if the data FIFO also has room. On the PHY side, `scoria_dfi_cmd_path` pops one token for each accepted WR command. This guarantees that a WR command can never reach the PHY ahead of its data. With the controller's rate-matched commit and `CMD_DELAY`, the token gate is an invariant that never holds; the command path's `r_wr_held_*` counters prove it.

30.5 FSM policy

There is no FSM. The CDC is a structural assembly of async FIFOs plus the two token sticky-latch paths.

30.6 Timing

- Command and data FIFO crossings add the latency of the `gaxi_fifo_async` implementation plus `N_FLOP_CROSS` synchronizer stages on each pointer.
- Init-token crossing adds one rising-edge detector, one FIFO, and one sticky latch on each side.
- No combinational path exists between `ctl_clk` and `dfi_clk`.

30.7 Notes

- The staged-token FIFO depth equals `WD_DEPTH` because at most one token can be staged per data word.
- All six crossings go through `gaxi_fifo_async`; there are no custom synchronizers.
- The block is formally proven in `formal/scoria/scoria_dfi_cdc.sby` with `prove` and `cover` tasks at depth 20.

31 The Dormant Pair (`scoria_powerdown_ctrl` + `scoria_dfi_signal_pack`)

Module: `scoria_powerdown_ctrl.sv`, `scoria_dfi_signal_pack.sv` **Location:** `rtl/fub/` **Category:** power-down / DFI pack **Parent:** — `(dormant)` **Status:** RETAINED DORMANT — nothing in the tree instantiates either block, deliberately

---

31.1 Purpose

Both source files carry the same header: **RETAINED DORMANT — nothing in the tree instantiates this, deliberately**. They are kept as files because scoria's DDR3 design point does not need them, but a future LPDDR3 build does. This page documents what each block contains and why deleting them would be premature.

A grep across the scoria tree for instantiation patterns (`scoria_powerdown_ctrl #(`, `scoria_dfi_signal_pack #(`, `u_powerdown`, `u_dfi_signal_pack`) returns no matches. The only references are the module definitions themselves, their individual filelists, and the master `scoria_all.f` lint closure, which lists them explicitly for compile coverage.

31.2 Parameters

Table 2.26.1: Dormant pair parameters

Parameter	Module	Default	Meaning
NUM_BANKS	scoria_powerdown_ctrl	8	banks per rank (unused today)
CS_WIDTH	scoria_powerdown_ctrl	NUM_RANKS	chip-select width
NUM_RANKS	scoria_dfi_signal_pack	1	rank count
DFI_RATE	scoria_dfi_signal_pack	2	DFI clock ratio
DFI_ADDR_WIDTH	scoria_dfi_signal_pack	14	per-phase address width
DFI_BANK_WIDTH	scoria_dfi_signal_pack	3	per-phase bank width
DFI_DATA_WIDTH	scoria_dfi_signal_pack	64	write-data width

31.3 Interface

31.3.1 scoria_powerdown_ctrl

Table 2.26.2: scoria_powerdown_ctrl ports

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
idle_threshold_i	in	16	cycles of idle before arming
enable_pde_i	in	1	precharge power-down enable
enable_sref_i	in	1	self-refresh enable; takes priority over PDE
controller_idle_i	in	1	scheduler idle indication
pdn_req_o	out	1	power-down request to scheduler
pdn_kind_o	out	1	0 = PDE, 1 = SR
pdn_grant_i	in	1	scheduler grant
sref_active_o	out	1	high while in self-refresh
dfi_cke_o	out	CS_WIDTH	CKE output; low in S_ASLEEP
obs_state_o	out	3	FSM state observability
obs_idle_cnt_o	out	16	idle-cycle counter
obs_grants_pde_o	out	16	PDE grants since reset
obs_grants_sr_o	out	16	SR grants since reset

31.3.2 scoria_dfi_signal_pack

Table 2.26.3: scoria_dfi_signal_pack ports

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
i_address, i_bank, i_cas_n, i_ras_n, i_we_n, i_cs_n, i_cke, i_odt	in	various	pre-pack command inputs
i_wrdata, i_wrdata_en, i_wrdata_mask, i_rddata_en	in	various	pre-pack data inputs
dfi_address_o, dfi_bank_o, dfi_cas_n_o, dfi_ras_n_o, dfi_we_n_o	out	various	packed command bus
dfi_cs_n_o, dfi_cke_o, dfi_odt_o	out	DFI_CS_BUS_W	packed control outputs
dfi_wrdata_o, dfi_wrdata_en_o, dfi_wrdata_mask_o, dfi_rddata_en_o	out	various	packed data outputs
dfi_dram_clk_disable_o	out	DFI_CS_BUS_W	DRAM clock disable; the block's unique function

31.4 Microarchitecture internals

31.4.1 scoria_powerdown_ctrl

The block has a four-state FSM:

```
S_AWAKE  — CKE high, normal operation
S_ARMING — counting idle cycles toward entry
S_REQ    — pdn_req_o high, awaiting pdn_grant_i
S_ASLEEP — CKE low, in PDE or SR
```

Entry arms when `controller_idle_i` is true and either `enable_pde_i` or `enable_sref_i` is set. Self-refresh has priority over precharge power-down. On grant, the FSM moves to `S_ASLEEP` and drives `dfi_cke_o` low. It wakes on `!controller_idle_i`. The exit timing (`tXSDLL`, `tXSR`) is enforced by the scheduler, not this block.

The header also carries a v3 TODO list: deep-power-down entry, per-rank power-down, and a `dfi_init_complete` interlock.

31.4.2 scoria_dfi_signal_pack

This is a one-cycle registered pipeline stage on the DFI bus. Its reset values produce a safe NOP: `cs_n = 1`, `ras_n = cas_n = we_n = 1`, `cke = 0`, `odt = 0`, write/read enable = 0, `wrdata_mask = 1`, and `dfi_dram_clk_disable = 0`. In normal operation it simply latches inputs; `dfi_dram_clk_disable_o` is held at 0 with a TODO for power-state integration.

The block owns `dfi_dram_clk_disable`. That signal is the reason it is retained: every other DFI signal scoria uses is already phase-multiplied by construction and does not need a separate pack stage, but deep power-down does need the clock-disable control.

31.4.3 The four gaps

Table 2.26.4: Why the dormant pair is not wired in the DDR3 design point

Level	State today
`powerdown_ctrl` / `dfi_signal_pack`	not instantiated
`scoria_top` / `scoria_dfi_plane`	no `dfi_cke` or `dfi_dram_clk_disable` port
`scoria_dfi_cmd_formatter`	`OP_SREFE` / `OP_SREFX` / `OP_DPDE` fall to the `default:` arm and are driven as NOP
`scoria_cmd_arbiter`	never picks any of those three ops

31.5 FSM policy

`scoria_powerdown_ctrl` has a real FSM because power-down entry is a sequential lifecycle: detect idle, arm, request, sleep, wake. `scoria_dfi_signal_pack` has no FSM — it is a pure flop pipeline.

31.6 Timing

Neither block contributes to the active DDR3 timing path because neither is instantiated. If they were wired, `powerdown_ctrl` would add a request-to-sleep latency of `idle_threshold_i` cycles plus scheduler grant latency, and `dfi_signal_pack` would add one cycle of registered delay on every DFI output.

31.7 Notes

- **The DDR3 parity argument:** LiteDRAM's generated DDR3 core for the Genesys 2 board has no power-down or self-refresh engine. Its only `CKE` references resolve to a CSR bit that software raises during init and leaves high. scoria holding CKE high forever is therefore parity with a controller that passes memtest on this hardware, not a shortfall.
- **That argument does not transfer to LPDDR3:** Self-refresh is a mobile DRAM's normal idle state, and LPDDR3 adds Deep Power Down, which DDR3 does not have. DPD requires stopping the DRAM clock, which is exactly `dfi_dram_clk_disable_o`. The pair is load-bearing for LPDDR3, which is why both files stay.
- **Retention trap:** Keeping dormant code in the lint closure is only safe because the files are not instantiated. If someone adds an instance, all four gaps in Table 2.26.4 must be closed at the same time or the feature will silently issue NOPs.
- **TODO inheritance:** The v3 TODOs in `powerdown_ctrl` and v2 TODOs in `dfi_signal_pack` are preserved verbatim. They name the dependencies that an LPDDR3 integration would have to address.

---

32 AXI4 Slave and APB CSR at Pin Level

The host-side boundary of `scoria_top` is the family-standard AXI4 slave plus the PeakRDL register cpuif. Both are inherited from pumice unchanged at the protocol level; the widths below are the scoria design point. The AXI4-plane logic is described block-by-block in AXI4 Plane; this page names the pins that cross the top-level shell.

32.1 Clocks and resets

Table 3.1.1: Clocks, resets, and init-done

Pin	Dir	Width	Source / sink	Notes
`aclk`	in	1	external	controller clock; AXI + scheduler + CAMs
`aresetn`	in	1	external	controller reset, active low
`dfi_clk`	in	1	external	PHY DFI clock
`dfi_rstn`	in	1	external	PHY reset, active low
`init_done_o`	out	1	`scoria_top`	initialization complete (also echoed into `STATUS`)

32.2 AXI4 slave signals

The design-point widths are `AXI_ID_WIDTH = 8`, `AXI_ADDR_WIDTH = 32`, and `AXI_DATA_WIDTH = 64`. `DW = DRAM_BEAT_WIDTH * DFI_RATE` sizes the data buses and equals 64 at the default build; `SW = DW / 8` is 8. The AXI4-plane details are in AXI4 Plane.

Table 3.1.2: AXI4 slave pin groups and widths

Group	Pin	Dir	Width
Write address	s_axi_awid	in	8
	s_axi_awaddr	in	32
	s_axi_awlen	in	8
	s_axi_awsize	in	3
	s_axi_awburst	in	2
	s_axi_awlock	in	1
	s_axi_awcache	in	4
	s_axi_awprot	in	3
	s_axi_awqos	in	4
	s_axi_awregion	in	4
	s_axi_awuser	in	1
	s_axi_awvalid	in	1
	s_axi_awready	out	1
Write data	s_axi_wdata	in	64
	s_axi_wstrb	in	8
	s_axi_wlast	in	1
	s_axi_wuser	in	1
	s_axi_wvalid	in	1
	s_axi_wready	out	1
Write response	s_axi_bid	out	8
	s_axi_bresp	out	2
	s_axi_buser	out	1
	s_axi_bvalid	out	1
	s_axi_bready	in	1

Group	Pin	Dir	Width
Read address	s_axi_arid	in	8
	s_axi_araddr	in	32
	s_axi_arlen	in	8
	s_axi_arsize	in	3
	s_axi_arburst	in	2
	s_axi_arlock	in	1
	s_axi_arcache	in	4
	s_axi_arprot	in	3
	s_axi_arqos	in	4
	s_axi_arregion	in	4
	s_axi_aruser	in	1
	s_axi_arvalid	in	1
	s_axi_arready	out	1
Read data	s_axi_rid	out	8
	s_axi_rdata	out	64
	s_axi_rresp	out	2
	s_axi_rlast	out	1
	s_axi_ruser	out	1
	s_axi_rvalid	out	1
	s_axi_rready	in	1

32.3 PeakRDL register cpuif

The CSR block is generated by PeakRDL and exposed through a simple req/ack cpuif. Doctrine 3 of the family docs applies unchanged: the host side is AXI4 + APB, and the register map comes from RDL. The APB-to-cpuif CDC is an integration concern, not a `scoria_top` port; see the HAS integration note. The generated register reference is `regs/generated/docs/scoria_csr.md`.

Table 3.1.3: PeakRDL cpuif pins

Pin	Dir	Width	Description
`s_cpuif_req`	in	1	request valid
`s_cpuif_req_is_wr`	in	1	1 = write, 0 = read
`s_cpuif_addr`	in	12	byte address into the 4 KB map
`s_cpuif_wr_data`	in	32	write data
`s_cpuif_wr_biten`	in	32	per-byte write enables
`s_cpuif_req_stall_wr`	out	1	write request stalled
`s_cpuif_req_stall_rd`	out	1	read request stalled
`s_cpuif_rd_ack`	out	1	read acknowledge
`s_cpuif_rd_err`	out	1	read error
`s_cpuif_rd_data`	out	32	read data
`s_cpuif_wr_ack`	out	1	write acknowledge
`s_cpuif_wr_err`	out	1	write error

32.4 CSR address-map summary

The map is 4 KB, addressed by `s_cpuif_addr[11:0]`. The full generated description is in `regs/generated/docs/scoria_csr.md`; the table below gives the group layout.

Table 3.1.4: CSR address-map groups

Group	Base	Registers
Control/status	0x000	`CTRL`, `STATUS`, `STATUS_HISTORY`
Core timings	0x010	`TIMINGS_RC_RCD_RP_RAS`, `TIMINGS_RFC_REFI`, `TIMINGS_RRD_FAW_WTR_CCD`, `TIMINGS_CL_CWL_WR`, `TIMINGS_RTP_RTW`
Mode registers	0x020	`MR0`, `MR1`, `MR2`, `MR3`
PASR / temp	0x030	`PASR_BANK_MASK_RANK0`, `PASR_SEG_MASK_RANK0`, `TEMP_DERATE_RANK0`
Scheduler policy	0x040	`SCHED_TUNING` (retired), `REFRESH_TUNING`, `ADDR_MAP`, `INIT_TUNING`
DFI/PHY timing	0x058	`INIT_TIMING0`, `INIT_TIMING1`, `INIT_TIMING2`, `DFI_PHASE`, `PHY_TIMING`
Scheduler/page	0x068	`SCHED_POLICY`, `SCHED_WR_WM`, `PAGE_POLICY_CFG`, `PAGE_TIMEOUT_CFG`
Per-bank observation	0x080	`OBS_ROW_HIT[0..7]`
ZQ calibration	0x0C4	`ZQ_CFG`, `ZQ_INTERVAL`, `ZQ_STATUS`, `ZQ_OBS_INTERVAL`
Write leveling	0x0D0	`WRLVL_CFG`, `WRLVL_TIMING0`, `WRLVL_TIMING1`, `WRLVL_TIMING2`,

Group	Base	Registers
		WRLVL_STATUS0, WRLVL_STATUS1
Refresh modes	0x140	REF_CTRL, REF_TIMING_PB
Page/sched/refresh stats	0x148	PAGE_STATS_HIT, PAGE_STATS_MISS, PAGE_STATS_EMPTY, SCHED_STATS_ACT, SCHED_STATS_PRE, REF_STATS_REF, REF_STATS_REF_BUSY, REF_STATS_POSTPONE, REF_STATS_PULLIN
Stall attribution	0x160	STALL_BP, STALL_REFRESH, STALL_TURNAROUND, STALL_TCCD, STALL_ACTLIMIT, STALL_BANKTIMER, STALL_NOREQ, STALL_ZQ
ID/build	0xFF0	ID, BUILD

32.5 Register liveness classes

Table 3.1.5: CSR liveness classes

Class	Registers	Notes
Continuously live	all core timings (`tRCD`, `tRP`, `tRAS`, `tRC`, `tWR`, `tRTP`, `tRTW`, `tRRD`, `tFAW`, `tWTR`, `tCCD`, `tRFC`, `tREFI`), `SCHED_POLICY`, `SCHED_WR_WM`, `PAGE_POLICY_CFG`, `PAGE_TIMEOUT_CFG`, `REF_CTRL`, `ZQ_CFG`, `ADDR_MAP`	Sampled every cycle by the core. Safe to change only when the effect is intended.
Live but should not change mid-traffic	`DFI_PHASE.bl`, `DFI_PHASE.gear_ratio`, `PHY_TIMING.memtype`	No protection; wrong values are bad config, not synthesis errors.
Consumed at init	`INIT_TIMING0/1/2`, `MR0..MR3.VAL`	New values take effect only when `CTRL.init_force_restart` replays the MRS chain.
Read-only status/telemetry	`STATUS`, `WRLVL_STATUS*`, `ZQ_STATUS`, `ZQ_OBS_INTERVAL`, `PAGE_STATS_*`, `SCHED_STATS_*`, `REF_STATS_*`, `STALL_*`, `OBS_ROW_HIT[*]`, `ID`, `BUILD`	Driven by hardware.

32.6 The timing convention: exact per bank, N+1 global

Every JEDEC timing field in the core-timing registers is a count of MC cycles. The per-bank windows (tRC, tRP, tRCD, tRAS, tRTP, tWR, in `mc_bank_timer`) are JEDEC-exact: the counter loads `N-1` and the gate opens exactly `N` cycles after the command (b409f364e, 2026-10-17; before that they were `N+1` too), pinned by the `c_t*_exact` covers in `formal/mc_common/mc_bank_timer`. The global windows (tRRD, tCCD, tWTR, tRTW, in `mc_global_timers`) still enforce `N+1`: the counter loads `N` and the gate opens the cycle after it would reach zero, pinned by `a_*_bound_n1` in `formal/mc_common/mc_global_timers`. Program the JEDEC value from the part datasheet; do not subtract one to compensate.

The generated register reference with full field descriptions is `regs/generated/docs/scoria_csr.md`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

33 DFI v3.1 Pin-Level Table

This page pins every DFI signal that appears on `scoria_top`. The widths below are expressed in the design-point parameters (`ROW_WIDTH = 14`, `NUM_BANKS = 8`, `NUM_RANKS = 1`, `DRAM_BEAT_WIDTH = 64`, `DFI_RATE = 2`) and therefore evaluate to the default values shown in parentheses. The DFI-plane internals are in DFI Plane; command encoding is in DFI Command Formatter; write leveling is in Write Leveling Interface.

The “Version” column marks whether a signal is part of the v2.1.1 base surface or added by v3.1. Scoria targets v3.1 (HAS decision D1), so the v3.1-added signals are required at the boundary even when this PHY family does not consume them.

33.1 Per-phase packed buses

DFI v3.1 generalises the phase-specific notation that v2.1.1 enumerated as `_p0`, `_p1`, … into `_pN`. Scoria's buses are packed: phase `p` occupies bits `[p*W +: W]` of the bus. A command bus with `DFI_RATE = 2` therefore carries two independent command slots per DFI cycle; the data buses carry one DRAM beat per phase. The formatter places non-column commands on phase 0, READ on `rd_phase`, and WRITE on `wr_phase`.

33.2 Reset convention

DFI outputs reset to the de-asserted, command-inactive level: chip selects high, command pins high, data enables low, and address/bank buses at 0. DFI inputs are qualified by their valid/enable companions and carry no reset value. `dfi_reset_n_o` is the exception: it is driven active from reset and released by the init sequencer when the sequence reaches CKE high.

33.3 Clock, reset, and init

Table 3.2.1: Clock, reset, and init handshake

Pin	Dir	Width	Version	Notes
dfi_clk	in	1	v2.1.1 base	PHY clock
dfi_rstn	in	1	v2.1.1 base	PHY reset, active low
dfi_reset_n_o	out	1	v2.1.1 base (DDR3)	DRAM RESET#; gated on memory type, not DFI version
dfi_init_start_o	out	1	v3.1 added	MC init / frequency-change request
dfi_init_complete_i	in	1	v3.1 added	PHY init complete / frequency-change acknowledge

dfi_reset_n_o is driven by the init sequencer and is latched released at S_D3_CKE; it is never re-asserted without an init_force_restart. dfi_init_start_o and dfi_init_complete_i form a self-timed handshake: the controller raises start, the PHY returns complete when ready, and the controller does not wait on the return at this design point. The init_done_o pin is the host-visible completion flag.

33.4 Command bus

Table 3.2.2: DFI command bus

Pin	Dir	Width	Version	Notes
dfi_address_o	out	ROW_WIDTH * DFI_RATE (28)	v2.1.1 base	address; row on ACT, column on RD/WR, MR data on MRS
dfi_bank_o	out	$clog2(NUM_BANKS) * DFI_RATE (6)	v2.1.1 base	bank address
dfi_cas_n_o	out	1 * DFI_RATE (2)	v2.1.1 base	command pin; A15 in non-ACT commands
dfi_ras_n_o	out	1 * DFI_RATE (2)	v2.1.1 base	command pin; A16 in non-ACT commands
dfi_we_n_o	out	1 * DFI_RATE (2)	v2.1.1 base	command pin; A14 in non-ACT commands
dfi_cs_n_o	out	NUM_RANKS * DFI_RATE (2)	v2.1.1 base	chip select; low asserts the rank
dfi_odt_o	out	NUM_RANKS * DFI_RATE (2)	v2.1.1 base	on-die termination

The DDR3 command truth table is NOP/ACT/RD/RDA/WR/WRA/PRE/PREA/REF/MRS/ZQCS/ZQCL; LPDDR3 uses the bit-exact JESD209-2F Table 60 CA-bus encoding packed into dfi_address_o. See DFI Command Formatter for the decode.

dfi_odt_o is driven according to the mode-register ODT settings and the direction of the current transaction; it is not a static tie-off. At init it is held low (or off) until the MRS chain has programmed the termination values.

At `NUM_RANKS = 1` the `dfi_cs_n_o` bus carries one active-low select repeated across `DFI_RATE` phases. There is no `dfi_cid` because chip ID is meaningful only for 3DS multi-rank stacks.

33.5 Write data

Table 3.2.3: DFI write data

Pin	Dir	Width	Version	Notes
dfi_wrdata_o	out	DRAM_BEAT_WIDTH * DFI_RATE (128)	v2.1.1 base	write data, per-phase packed
dfi_wrdata_en_o	out	DFI_RATE (2)	v2.1.1 base	write data enable, one bit per phase
dfi_wrdata_mask_o	out	(DRAM_BEAT_WIDTH * DFI_RATE) / 8 (16)	v2.1.1 base	byte mask; ~strb from the write data stream

The write serializer drives dfi_wrdata_en_o after t_phy_wrlat DFI cycles and streams one DFI word per cycle until the burst's last word. dfi_wrdata_mask_o is the inverse of the AXI write strobe.

The write-data path is rate-matched to the command path by the staged-token invariant in the CDC: a WR command cannot reach the PHY ahead of its data. See DFI Clock-Domain Crossing for the token mechanism.

33.6 Read data

Table 3.2.4: DFI read data

Pin	Dir	Width	Version	Notes
dfi_rddata_en_o	out	DFI_RATE (2)	v2.1.1 base	read data enable, one bit per phase
dfi_rddata_i	in	DRAM_BEAT_WIDTH * DFI_RATE (128)	v2.1.1 base	read data, per-phase packed
dfi_rddata_valid_i	in	DFI_RATE (2)	v2.1.1 base	read data valid, one bit per phase

The read aligner drives dfi_rddata_en_o after t_rddata_en cycles and captures BL_WORDS DFI words. dfi_rddata_valid_i is fire-and-forget: the aligner accepts every asserted beat and pushes it into the read CDC FIFO.

dfi_rddata_en_o is masked to the active phases by gear_ratio: at the default gear the mask is all-ones, and the outputs are unchanged. See DFI Read Aligner.

33.7 Per-data-phase chip selects (v3.1 added)

Table 3.2.5: Per-data-phase chip selects

Pin	Dir	Width	Version	Notes
dfi_wrdata_cs_n_o	out	NUM_RANKS * DFI_RATE (2)	v3.1 only	which rank owns the DQ bus during the write window
dfi_rddata_cs_n_o	out	NUM_RANKS * DFI_RATE (2)	v3.1 only	which rank owns the DQ bus during the read window

At NUM_RANKS = 1 both buses are driven constant '0; the only legal value is rank 0. The CS-under-training role of dfi_wrdata_cs_n_o is degenerate at one rank because the rank being leveled is always CS0. A multi-rank build must drive them from the granted command's rank, in step with dfi_cs_n_o.

33.8 Write-leveling handshake (v3.1 added)

Table 3.2.6: Write-leveling handshake

Pin	Dir	Width	Version	Notes
`dfi_phylvl_req_cs_n_o`	out	`NUM_CS` (1)	v3.1 only	per-CS leveling request to PHY
`dfi_phylvl_ack_cs_n_i`	in	`NUM_CS` (1)	v3.1 only	PHY acknowledge per CS
`dfi_phy_wrlvl_cs_n_o`	out	`NUM_CS` (1)	v3.1 only	which CS is under write leveling
`dfi_wrlvl_strobe_o`	out	1	v3.1 only	one-cycle DQS strobe to PHY
`dfi_prime_dq_i`	in	1	v3.1 only	prime DQ result from PHY

This PHY family does not implement the DFI leveling handshake, so on the Genesys 2 build these pins are unused; write leveling there is the PHY's own. They exist for a PHY that exposes the v3.1 interface. The crossing from controller clock to `dfi_clk` lives in DFI Plane; the protocol state machine is in Write Leveling Interface. The `dfi_wrlvl_strobe_o` pulse is generated only in the `WL_READY` state.

33.9 Signals not pinned out

The following DFI v3.1 signals are defined by the specification but are **not present on `scoria_top`** at this design point:

Table 3.2.7: DFI v3.1 signals not pinned out

Signal	Why absent
`dfi_lp_ctrl_req` / `dfi_lp_data_req`	The dormant `powerdown_ctrl` / `dfi_signal_pack` pair is retained uninstantiated; power-down is absent by decision. See The Dormant Pair.
`dfi_error` / `dfi_error_info`	DDR4-era CA-parity error surface; scoria uses the error indication only conceptually and ignores the DDR4-specific encodings.

The absence of the low-power handshake means the HAS's never-wedge timeout behavior has no RTL driver at the pin level in this build. If a future PHY consumes `dfi_lp_ctrl_req` / `dfi_lp_data_req`, the handshake must be added at the top level and the timeout rule becomes binding.

33.10 References

The DFI v3.1 specification delta is argued in the HAS Chapter 4. The RTL port list is `rtl/top/scoria_top.sv`; the DFI-plane internal surface is `rtl/macro/scoria_dfi_plane.sv`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

34 Core Signal Contracts

34.1 What a contract is here

A signal contract in this repo is a machine-checkable statement of what a signal is allowed to do: a term list with its defining RTL citation, the invariants between terms, and a decision table whose forbidden rows are marked `ILLEGAL`. The canonical methodology is `vault/handbook/design/signal-contracts-and-kmaps.md`; the shared generator machinery is `bin/kmaps/`. A future scoria kmap book will build its contract sheets from the anchors this chapter names.

34.2 Post-RTL citation posture

This MAS is written after the RTL, so every contract below cites the RTL file and a named fence, assertion, or state list rather than a pre-RTL MAS page. When the kmap generator lands, its citation gate will diff the workbook against these anchors; a drift fails the run. The anchor map gives the contract domain, the RTL anchor, the formal suite where one exists, and the current verdict.

Table 4.1: Anchor map — RTL anchors and formal verdicts

Contract domain	RTL anchor	Formal suite	Verdict
Address decode	`scoria_addr_mapper.sv` — `{rank,bank,row,col}` decode	`scoria_addr_mapper.sby`	PROVEN
DDR3 command decode	`scoria_dfi_cmd_formatter.sv` — DDR3 truth-table fence	none	RTL fence
LPDDR3 CA encoding	`scoria_dfi_cmd_formatter.sv` — JESD209-2F Table 60 CA-bus block	none	RTL fence
Maintenance request/grant	Family doctrine 2 (quoted below); `scoria_cmd_arbiter.sv` maintenance ports	`scoria_cmd_arbiter.sby`	PROVEN
Refresh interval/credit	`scoria_refresh_ctrl.sv` — `POSTPONE_MAX = MAX_PENDING - 2` derivation	`scoria_refresh_ctrl.sby`	PROVEN
ZQ calibration window	`scoria_cmd_arbiter.sv` — `w_out_safe` ZQ quiet term	`scoria_cmd_arbiter.sby`	PROVEN
Bank-timer spacing gates	`scoria_bank_timer.sv` — `safe_act_o/safe_`	`scoria_bank_timer.sby`	PROVEN

Contract domain	RTL anchor	Formal suite	Verdict
	rd_o/safe_wr_o/safe_pre_o		
Global-timers next-state alignment	scoria_global_timers.sv — next-state derivation	scoria_global_timers.sby	PROVEN
Read-CAM issue contract	scoria_rd_cmd_cam.sv — issue-of-invalid-slot assertion	scoria_rd_cmd_cam.sby	ASSERTED
Return-ring input contracts	scoria_rd_return_ring.sv — return-without-ticket / issue-while-empty assertions	scoria_rd_return_ring.sby	ASSERTED
Write-CAM fill/commit/snarf	scoria_wr_data_cam.sv — fill/commit/drain queues	scoria_wr_data_cam.sby	PROVEN
CDC staged-token invariant	scoria_dfi_cdc.sv — write-burst-staged token FIFO	scoria_dfi_cdc.sby	PROVEN
Command-path no-pacing	scoria_dfi_cmd_path.sv — structural-holds-only comment	none	MEASURED
Arbiter final-gate drop-not-hold	scoria_cmd_arbiter.sv — w_out_safe rank-global terms	scoria_cmd_arbiter.sby (BUG001=0)	PROVEN
Init MR ordering	scoria_init_sequencer.sv — DDR3/LPDDR3 state-list fences	none	RTL fence
Write-leveling four-state telemetry	scoria_wrlvl_ifc.sv — state encoding and	none	RTL fence

Contract domain	RTL anchor	Formal suite	Verdict
	status outputs		

The line numbers are not pinned here; the generator’s citation gate will pin them when it runs. Until then the named fence or assertion is the anchor.

34.3 Contract: address decode

Terms. `axi_addr_i`, `bank_lsb_i`, `hash_en_i`, `hash_seed_i`, `rank_o`, `bank_o`, `row_o`, `col_o`.

Invariants.

1. The decode is injective inside the mappable range: two distinct word addresses never decode to the same `{rank,bank,row,col}` tuple (`a_injective` in `scoria_addr_mapper.sby`).
2. Equal word addresses decode identically (`a_deterministic`).
3. `row_o` and `rank_o` are invariant across legal `bank_lsb` values; only the bank/column split moves (`a_row_invariant`, `a_rank_invariant`).
4. An out-of-range `bank_lsb` clamps to `COL_WIDTH` (ROW_MAJOR) rather than producing an illegal slice (`a_clamp_*`).

Decision table. Rows where two different addresses alias, or where row/rank move with `bank_lsb`, or where an out-of-range `bank_lsb` produces a non-ROW_MAJOR decode, are `ILLEGAL`.

34.4 Contract: DDR3 command decode and LPDDR3 CA encoding

Terms. `dfi_cs_n_o`, `dfi_ras_n_o`, `dfi_cas_n_o`, `dfi_we_n_o`, `dfi_bank_o`, `dfi_address_o`; `memtype_i` selects the encoding family.

Invariants.

1. For DDR3, the anchored truth table in `scoria_dfi_cmd_formatter.sv` enumerates exactly `NOP/ACT/RD/RDA/WR/WRA/PRE/PREA/REF/MRS/ZQCS/ZQCL`; every command the formatter issues is one of these rows.
2. Auto-precharge variants (`RDA`, `WRA`, `PREA`) are distinguished by the `AP` bit carried on `dfi_address_o[10]`, not by a separate pin.
3. For LPDDR3, the CA-bus word is bit-exact JESD209-2F Table 60, packed as `{w_ca_f, w_ca_r}` into `dfi_address_o`; `dfi_ras_n_o/cas_n_o/we_n_o` are held idle.
4. `dfi_odt_o` is driven from the mode-register ODT settings and transaction direction; it is not a static value.

Decision table. One row per (op, memtype) pair; rows that decode to a command outside the anchored set are `ILLEGAL`. The minimized qualifier SOPs for this table will be the kmap book's command table when the generator lands.

34.5 Contract: maintenance request/grant

Terms. `refresh_req_o`/`refresh_grant_o` from `scoria_refresh_ctrl`, `zq_req_i`/`zq_grant_o` from `scoria_zq_ctrl`, and the arbiter's maintenance gating.

Invariants.

1. Family doctrine 2 applies in full: "Refresh, ZQ calibration, and their relatives are guests on the command bus. They raise a request, the scheduler grants them a window, and they wait their turn — they never preempt an in-flight host transaction."
2. `gnt` is exclusive: at most one maintenance source holds the bus in any cycle.
3. A refresh or ZQ calibration occupies the banks it touches for the operation's duration; the bank timers and the arbiter's `w_out_safe` quiet window own those banks.

Decision table. One row per (source, state) pair; rows where two sources hold `gnt`, or a source issues without `gnt`, or any command fires inside the ZQCS window, are `ILLEGAL`.

34.6 Contract: refresh interval/credit

Terms. `t_refi_i`, `postpone_limit_i`, `pullin_limit_i`, `pending_refreshes_o`, `obs_pullin_credit_o`, `refresh_req_o`.

Invariants.

1. JEDEC permits at most 8 postponed refreshes; the backlog saturates at that ceiling (`a_pending_ceiling` in `scoria_refresh_ctrl.sby`).
2. `POSTPONE_MAX = MAX_PENDING - 2 = 6` gives two refreshes of headroom: the request asserts at a backlog of 7 before the accumulator saturates at 8 (`a_headroom`).
3. Pull-in credit stays inside the JEDEC +-8 window (`a_pullin_ceiling`).
4. The REFpb bank rotor advances by exactly one per accepted REFpb and only then, mirroring the device's internal counter (`a_rotor_advances_on_refpb`, `a_rotor_holds_without_refpb`).

Decision table. Rows where `pending_refreshes_o > 8`, or where the rotor steps without a REFpb grant, or where a backlog past `POSTPONE_MAX` does not raise `refresh_req_o`, are `ILLEGAL`.

34.7 Contract: ZQ calibration window

Terms. `zq_req_i`, `zq_grant_o`, `t_zqcs_i`, `stall_zq_o`, the arbiter's `w_out_safe` final gate.

Invariants.

1. JESD79-3F 3.10 forbids every command during the `tZQCS` window.
2. The arbiter owns this obligation alone; `scoria_zq_ctrl` does not hold the window.
3. `scoria_cmd_arbiter.sby` proves `a_zqcs_quiet`: no ACT/RD/WR/PRE fires while `age_zq <= t_zqcs`.

Decision table. Any row with a command issued inside the ZQCS window is `ILLEGAL`.

34.8 Contract: bank-timer spacing gates

Terms. `safe_act_o`, `safe_rd_o`, `safe_wr_o`, `safe_pre_o`, and the timing CSR inputs `t_rcd_i`, `t_rp_i`, `t_ras_i`, `t_rc_i`, `t_wr_i`, `t_rtp_i`.

Invariants.

1. The gate cannot open early: `a_trc`, `a_trp`, `a_trcd`, `a_tras`, `a_trtp`, `a_twr` in `scoria_bank_timer.sby` enforce the JEDEC windows.
2. The enforced bound is `N+1`, not `N`: the counter is loaded with the programmed value and opens the cycle after it would reach zero (`a_trc_bound_n1`, `a_trcd_bound_n1`, `a_tras_bound_n1`).
3. Structural implications used by the arbiter: `safe_act => !row_valid`; `safe_rd || safe_wr => row_valid`; `safe_pre => row_valid`; `ap_pending => row_valid`.
4. `safe_rd == safe_wr` in this RTL; a consumer that treats them differently is wrong.

Decision table. Rows where a command fires while the matching `safe_*` is low, or where the spacing is less than `N+1`, are `ILLEGAL`.

34.9 Contract: global-timers next-state alignment

Terms. `tfaw_window_ok_o`, `trrd_window_ok_o`, `twtr_global_ok_o`, `trtw_window_ok_o`, `tccd_window_ok_o`, and their `obs_*` shadows.

Invariants.

1. The block derives its next state once and feeds both the counter flops and the readiness flops from it; there is no second derivation to fall out of step (the FIXED-FORM property that closed pumice ISSUE-018).
2. The environment contract is exactly the published flags plus one cycle of the consumer's own compensation; no extra hidden term is required.
3. `scoria_global_timers.sby` proves `a_trrd`, `a_tfaw`, `a_tccd`, `a_twtr`, `a_trtw`, and the `N+1` bound variants.
4. The debug view matches the scheduler view cycle-for-cycle (`a_obs_faw`, `a_obs_trrd`, `a_obs_twtr`, `a_obs_trtw`, `a_obs_tccd`).

Decision table. Rows where the consumer obeys the flags and a JEDEC window is still violated are `ILLEGAL`.

34.10 Contract: read-CAM issue contract

Terms. `issue_valid_i`, `issue_slot_i`, `sch_valid_o`, `r_valid[issue_slot_i]`.

Invariants.

1. This is an input contract, not a property of the block: it constrains whoever issues, not the CAM.
2. `scoria_rd_cmd_cam.sv` carries a simulation-only assertion at the issue-of-invalid-slot check (`w_issue_fire && !r_valid[issue_slot_i]`).
3. `scoria_rd_cmd_cam.sby` takes the same statement as an assumption (`assume (!issue_valid_i || sch_valid_o[issue_slot_i])`) and proves ticket integrity and slot lifecycle under it.

Decision table. A row where `issue_valid_i` is high and the targeted slot is not valid is `ILLEGAL`.

34.11 Contract: return-ring input contracts

Terms. `dfi_ret_valid_i`, `w_iq_rd_valid`, `issue_valid_i`, `w_empty`.

Invariants.

1. `assert (!(dfi_ret_valid_i && !w_iq_rd_valid))` — a return beat never arrives without a ticket in flight.
2. `assert (!(issue_valid_i && w_empty))` — a ticket is never issued while the ring is empty.
3. Both are environment contracts on the driver; `scoria_rd_return_ring.sby` restates them as assumptions and proves accounting, no-fabrication, and data integrity under them.

Decision table. Rows violating either invariant are `ILLEGAL`.

34.12 Contract: write-CAM fill/commit/snarf ordering

Terms. `ins_valid_i/ready_o`, `wd_valid_i/ready_o`, `commit_valid_i/ready_o`, `cm_rd_valid_o/ready_i`, `snarf_probe_valid_i`, `snarf_hit_o`, `snarf_accept_i`.

Invariants.

1. A slot is not schedulable until it is fully filled; the scheduler view excludes not-yet-filled entries.
2. Commit is rate-matched: at most `WR_DRAIN_AHEAD` bursts sit in the drain queue, so a WR command cannot fire far ahead of its data.
3. Snarf returns the youngest matching filled entry; the snarf and commit streams share one BRAM read port with commit priority and a starvation limit.
4. `scoria_wr_data_cam.sby` proves burst framing (`a_burst_len`) and slot lifecycle (`a_ins_ready_implies_free`).

Decision table. Rows where a commit fires before fill completion, where snarf returns a stale entry, or where burst framing is violated are `ILLEGAL`.

34.13 Contract: CDC staged-token invariant

Terms. `wd_valid_i`, `wd_last_i`, `wd_ready_o`, `pwr_staged_valid_o`, `pwr_staged_pop_i`.

Invariants.

1. A write burst crosses as data words plus, on the last word, a one-bit “burst staged” token in a second FIFO.
2. The command path pops one token per WR it accepts, so a WR command can never reach the PHY ahead of its data.
3. `scoria_dfi_cdc.sby` proves `a_no_extra_staged`: the PHY never pops more staged bursts than the controller completed.

Decision table. Rows where a token crosses without its data, or data crosses without its token, are `ILLEGAL`.

34.14 Contract: command-path no-pacing

Terms. `cmd_valid_i`, `cmd_ready_o`, `wr_op_ready_i`, `rd_op_ready_i`.

Invariants.

1. `scoria_dfi_cmd_path.sv` does not pace commands; all timing lives in the scheduler.
2. The only remaining holds are structural: a read may not issue unless the read aligner has a free slot, and a write may not issue unless its data has crossed and is staged.
3. TASK-007 measured the structural-hold profile; intentional pacing in the command path was removed because it compressed JEDEC spacing behind the hold.

Verdict. MEASURED, not formally proven as a contract. The property is that no timing-dependent hold exists in this path.

34.15 Contract: arbiter final-gate drop-not-hold

Terms. `w_out_safe`, `bank_act_ready_i`, `tfaw_ok_i`, `trrd_ok_i`, `bank_rdwr_ready_i`, `twtr_ok_i`, `trtw_ok_i`, `tccd_ok_i`.

Invariants.

1. The final gate re-validates both per-bank readiness and rank-global windows (`tfaw_ok`, `trrd_ok`) at the cycle the command fires.
2. BUG-001 was the failure mode: an earlier sampling of the rank-global terms let two cross-bank ACTs violate `tRRD`. The fix adds the rank-global terms to `w_out_safe`.
3. `scoria_cmd_arbiter.sby` proves `a_trrd_spacing` and `a_tfaw_window` with `BUG001 = 1`; the `nofix` task runs with `BUG001 = 0` to show the failure mode.
4. BUG-003 is the same shape for ZQ: the final gate drops the command rather than holding it, preserving spacing.

Decision table. Rows where a command fires while any relevant `*_ok` or per-bank ready is low are `ILLEGAL`.

34.16 Contract: init MR ordering

Terms. `init_cmd_valid`, `init_cmd_op`, `init_cmd_bank`, `init_cmd_row`, the init FSM state list.

Invariants.

1. DDR3 order is fixed by the state list: `S_D3_RSTN` → `S_D3_CKE` → `S_D3_XPR` → `S_D3_MR2` → `S_D3_MR3` → `S_D3_MR1` → `S_D3_MR0` → `S_D3_ZQCL` → `S_D3_LOCK` → `S_DONE`.
2. LPDDR3 order is `S_L_RESET` → `S_L_ZQ` → `S_L_MR1` → `S_L_MR2` → `S_L_MR3` → `S_DONE`.
3. MR0 load ORs in `DDR3_DLL_RESET = 16'h0100`; the shadow is written with the unmodified `mr0_i`.

Decision table. Rows where the MR order is violated, or where RESET# is re-asserted without `init_force_restart`, are `ILLEGAL`.

34.17 Contract: write-leveling four-state telemetry

Terms. `wrlvl_result`, `wrlvl_result_valid`, `wrlvl_timeout`, `wrlvl_ever_done`, `wrlvl_state`, `strobe_i`, `prime_dq_i`.

Invariants.

1. The four observable outcomes are {never attempted, converged, timed out, swept with no flip}; each has a distinct encoding in the status registers.
2. "Never attempted" is the reset state (`WL_OFF`); a detector that has never fired must be representable.
3. Timeout is distinguishable from no-result: `t_wlmrd_max` is a controller-defined timeout with its own status bit (`wrlvl_timeout`).
4. The state machine has no delay-walk states; firmware performs the search. `tWLOE` is inert because only the prime DQ bit is sampled.

Decision table. Rows where timeout and no-result share an encoding, or where a search state appears in the state list, are `ILLEGAL`.

34.18 Regenerating the workbook

There is no scoria kmap generator run yet. This chapter is the citation-ready anchor map for when one lands. Do not hand-edit a future `scoria_signal_contracts.xlsx`; it must be a build artifact of the generator, and the citation gate must fail the run if any RTL anchor drifts from the anchors named above.